

SRT Cashflow Modelling & mgen Library Introduction

Securitized Products & Structured Finance

22 October, 2019

Abstract

This paper provides an introduction to the Mizuho R library mgen developed on the Securitized Products & Structured Finance desk specifically for SRT structuring. The focus of the paper is on mgen use in the context of SRT pricing as requested by model risk. Information is provided relating to how to setup mgen and mlib to co-exist on a desktop machine and the necessary environment variables. The location of the source code is provided and the method for cloning outlined. The paper sets out specific illustrations of how to use the mgen objects for an illustrative SRT structure, starting from the basic building blocks to explain the approach from the ground up. It should be noted that this document is in draft form and subject to change, it will be finalised if the approach is accepted on completion of the validation.

Draft Version 1.22

Contents

1	Introduction to mgen	4
1.1	RStudio	4
1.1.1	Source Code Access	4
1.1.2	mllib dependency	4
1.1.3	Environment variables	4
1.2	Jupyter Notebook	5
1.3	SmaRT portal	5
2	Models & Regulation	7
2.1	ABS Modelling	7
2.2	Regulation	7
2.2.1	CRR	7
2.2.2	CRR Revisions	7
2.2.3	Discussion Paper	7
2.2.4	PRA	7
2.2.5	A&O Review	7
2.3	Basel III	7
3	SRT Modelling	8
3.1	Loading mgen	8
3.2	Loading mllib	8
3.3	Abstractions	10
3.4	Loan	10
3.4.1	Projecting Cash-flows	10
3.4.2	Bullet Loans	11
3.4.3	Recovery Lag	16
3.4.4	Comparsion with Schouten, Jonsson 2009	19
3.4.5	Amortising Loans	21
3.4.6	Floating Rate Loans	29
3.5	Portfolios	33
3.5.1	Portfolio Uploads	33
3.5.2	Portfolio Object Instantiation	36
3.5.3	Reinvestments	38
3.5.4	Reinvestment replines	39
3.6	Ratings Migration	42
3.7	Tranche	42
3.8	Triggers	43
3.9	Cashflow Engine	43
3.9.1	Waterfall	43
3.9.2	Example	44
3.9.3	Cash-flow output	45
3.9.4	Visualisations	54
3.9.5	Reinvestments	65
3.9.6	Triggers	75
3.9.7	Portfolio Upload	81
3.9.8	Pricing	82
3.9.9	PV Cashflows	86
3.9.10	IRR	87
3.9.11	Portfolio Data	87
4	SRT Structuring	88
4.1	Data Tapes	88
4.1.1	Client tape	88

4.1.2	Generate tape	88
4.2	Portfolio Optimisation	89
4.2.1	Simplex Algorithm	89
4.2.2	Greedy Algorithm	91
4.3	Capital Relief	91
4.3.1	SEC_IRBA	91
5	Jupyter Platform	92
5.1	mgen on Jupyter	92
5.1.1	Loans	92
5.1.2	Cashflow Engine	92
5.1.3	Visualisation	93
6	SmaRT Portal	95
6.1	Upload or generate tape	95
6.1.1	Upload	95
6.1.2	Generate	96
6.1.3	Tape stratifications	97
6.2	Portfolio Creation	98
6.2.1	Specify constraints	98
6.2.2	Select a portfolio	99
6.3	Liabilities	100
6.3.1	Define a capital structure	100
6.3.2	Calculate Capital Efficiency	101
6.3.3	Cash-flow Projections	101
6.4	Transaction Report	102

1 Introduction to mgen

mgen is an R based library that contains functions and objects developed on desk by Securitized Products & Structured Finance in consideration of structuring SRT transactions to replace the existing spread sheets. The SmaRT portal currently under development for structuring SRT transactions is built on top of the mgen library.

There are 3 ways to use mgen

1. RStudio - for source code level access
2. Jupyter Notebooks - for web based flexible access
3. SmaRT web portal - for web access

Accessing via RStudio is the lowest level access where source code may be accessed and inspected, Jupyter Notebooks offers an interactive web based approach where mgen can be used but the underlying code remains hidden, finally using the portal is a web based client friendly platform that requires no coding knowledge and is point and click. Importantly all 3 methodologies utilize the same library consequently consistency is assured by design, there is no duplication of code.

In this paper guidance is given firstly on how to setup the environment to use mgen and secondly in respect of the relevant functions and objects used for SRT modelling.

1.1 RStudio

RStudio is packaged and can be requested from x6666, it is also required to be a member of group Ustr_Web3 in order that r packages can be accessed from CRAN

1.1.1 Source Code Access

The library source is hosted on bitbucket at url

<https://bitbucket.uk.mizuho-sc.com:8449/projects/SRTPOR/repos/mgen/browse>

The repository can be cloned by opening a command prompt and typing git clone <https://USERID@bitbucket.uk.mizuho-sc.com:8449/scm/srtpor/mgen.git>

where USERID is replaced by the user credentials.

1.1.2 mlib dependency

To provide forward rates and discounting, from a previously validated source, the Mizuho mlib library is used and therefore mlib must be installed locally. For production machines this requires that the MizuhoBento framework is configured.

Once Bento has been configure (RJP can assist with setup) it is necessary to install mlib, the current latest copy is available here “X:\Secure\DIR_MLib_API\2019-07-04.1\Win64”

The mlib library should be installed into location ‘C:\MizuhoBento\Bento\QuantRapi’

1.1.3 Environment variables

It is necessary to configure two windows environment variables as follows:

MLIB_R_PATH=‘C:\MizuhoBento\Bento\QuantRapi’

MLIB_CURVES_PATH='I:\Group\SRT Private\Analytics\mlib\curves'

The curves path requires localisation by the user, this can be anywhere but the mlib JSON files will need to be copied into this directory.

Ideally mlib will in future source market data files directly from cdw, this has been requested

1.2 Jupyter Notebook

The mgen library can be accessed via Jupyter notebook, this can be via either using anaconda and local installation or via a Jupyter server.

If server Jupyter access is required, please contact Andrei Preda x1504

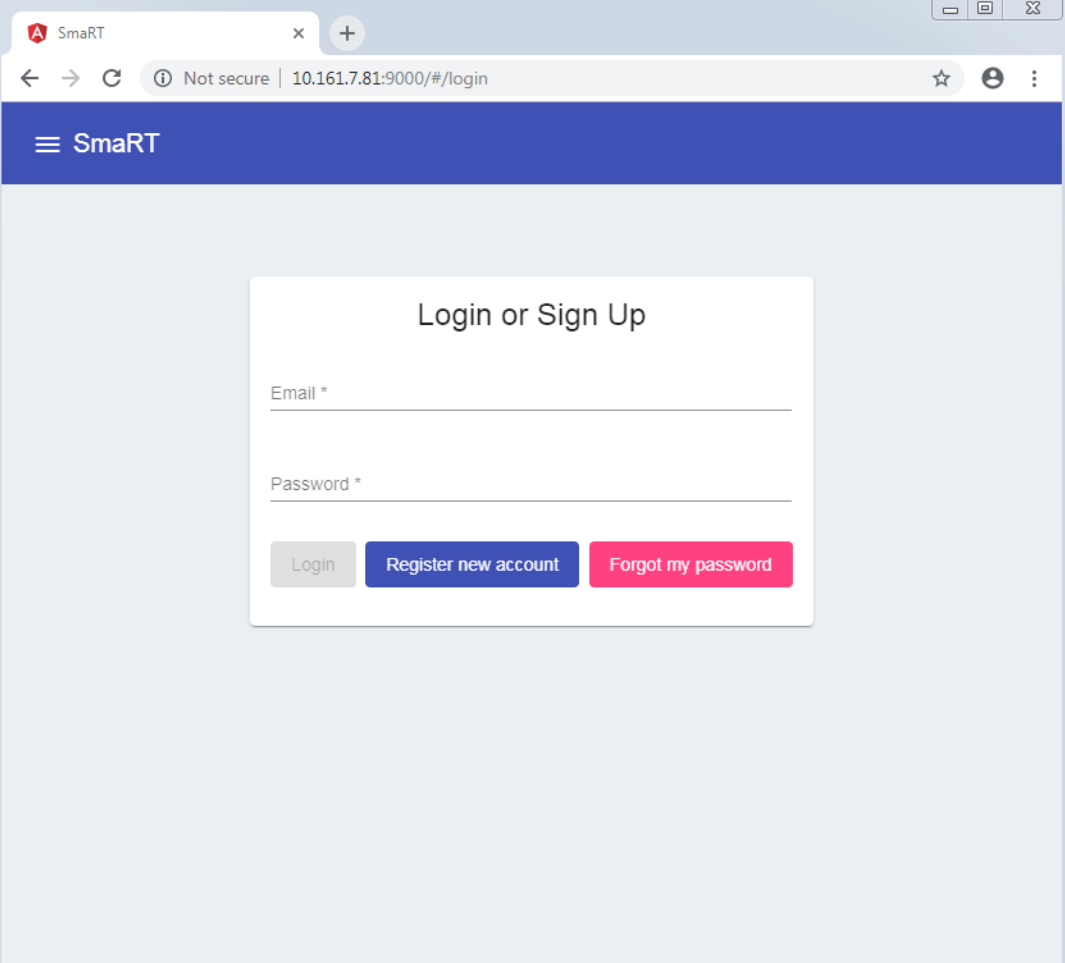
1.3 SmaRT portal

The SmaRT portal makes use of the mgen library to produce projections for SRT structuring, it can be accessed at the following URL

<http://10.161.7.81:9000>

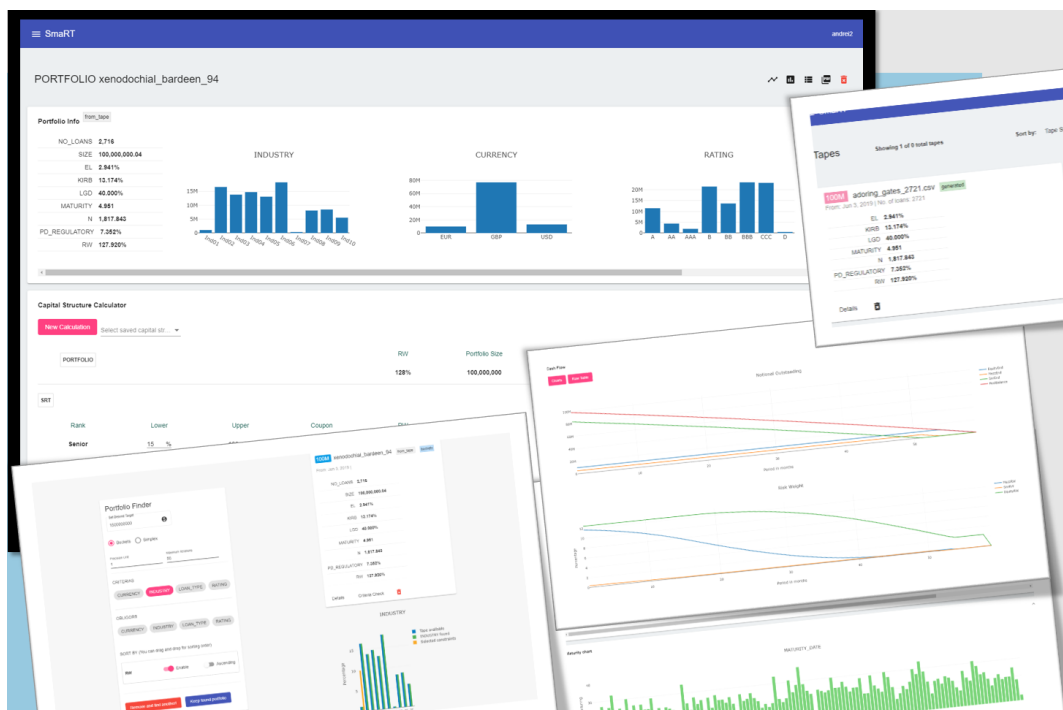
Contact Andrei Preda x 1540 for further information.

The SmaRT portal login is shown below



The screenshot shows a web browser window with the SmaRT portal login page. The browser's address bar displays the URL `10.161.7.81:9000/#/login`. The page features a blue header bar with the SmaRT logo on the left. The main body of the page is light gray and contains a white rectangular login form. The form is titled "Login or Sign Up" and includes two input fields: "Email *" and "Password *". Below these fields are three buttons: a gray "Login" button, a blue "Register new account" button, and a pink "Forgot my password" button.

A snapshot of a selection of user screens from the Smart portal is shown below



2 Models & Regulation

2.1 ABS Modelling

Asset backed securities: Risks, Ratings and Quantitative Modelling A good introduction to asset & liability modelling in cashflow transactions can be found here <https://www.eurandom.tue.nl/reports/2009/050-report.pdf>. It should be noted that whilst not detailing SRT mechanics the underlying asset side modelling is comparable.

2.2 Regulation

2.2.1 CRR

The Capital Requirements Regulation, now superceded in respect of the supervisory formula <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32013R0575&from=EN>

2.2.2 CRR Revisions

<https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32017R2401&from=en>

2.2.3 Discussion Paper

On the Significant Risk Transfer in Securitisation <https://eba.europa.eu/documents/10180/1963391/Discussion+Paper+on+the+Significant+Risk+Transfer+in+Securitisation+%28EBA-DP-2017-03%29.pdf/228098e3-29ba-473f-9e4c-680ce32e1869>

2.2.4 PRA

Securitisation: The new EU framework and SRT <https://www.bankofengland.co.uk/-/media/boe/files/prudential-regulation/policy-statement/2018/ps2918>

2.2.5 A&O Review

A&O review of revised EU securitisation framework and SRT [http://www.allenoverly.com/SiteCollectionDocuments/PRA_consultation_paper_published_on_the_revised_EU_securitisation_framework_and_significant_risk_transfer_\(Email\).pdf](http://www.allenoverly.com/SiteCollectionDocuments/PRA_consultation_paper_published_on_the_revised_EU_securitisation_framework_and_significant_risk_transfer_(Email).pdf)

2.3 Basel III

Finalising post-crisis reforms <https://www.bis.org/bcbs/publ/d424.pdf>

3 SRT Modelling

The stages of structuring an SRT can be considered sequentially as

- Receive data-tape
- Select/optimize portfolio
- Project cash-flows of portfolio
- Process cash-flows through capital structure subject to waterfall logic
- PV projected flows of the liability

The portfolio selection algorithm approaches are out of scope and will be detailed in a separate notebook. This paper assumes a final portfolio has been provided in the form of a reference registry, as is the case in an investor report for an existing transaction.

This notebook illustrates a simple cash-flow engine in R, the engine requires functionality that can be abstracted into the following objects.

1. Loan
2. Portfolio
3. Tranche
4. Triggers
5. Cash-flow Engine
6. Reg Capital Calculation

The capital requirements regulation as published by the EBA for section 6 forms the basis of the Reg Capital Calculation, to be covered in separate mgen documentation.

3.1 Loading mgen

First mgen must be loaded to obtain access to the library

3.2 Loading mlib

Mlib, Mizuho International's interest rate library is used for generation of discount factors and forward rates, it is loaded via an adapter creating an instance object from which wrapped functionality can be utilized.

```
# open mlib adapter
m <- mlib_adapter$new()
```

```
## in method for 'copyToR' with signature '"_p_swig::NelsonSiegelSvenssonParameters"': no definition for
## in method for 'copyToR' with signature '"_p_swig::NelsonSiegelSvenssonCalibrationResults"': no defin
## in method for 'copyToR' with signature '"_p_swig::PolynomialCalibrationResults"': no definition for
## INFO [2019-22-10 14:21:13] MLIB Successfully loaded and ready to GO!
## INFO [2019-22-10 14:21:15] Curve EUR_OIS_CURVE@24 loaded from 20190101!
## INFO [2019-22-10 14:21:15] Curve EUR_OIS_CURVE@30 loaded from 20190101!
## INFO [2019-22-10 14:21:15] Curve EUR_OIS_CURVE_MARKETDATA@23 loaded from 20190101!
## INFO [2019-22-10 14:21:15] Curve EUR_SWAP_3M_CURVE@18 loaded from 20190101!
## INFO [2019-22-10 14:21:15] Curve EUR_SWAP_3M_CURVE@22 loaded from 20190101!
## INFO [2019-22-10 14:21:15] Curve EUR_SWAP_3M_CURVE_MARKETDATA@21 loaded from 20190101!
## INFO [2019-22-10 14:21:15] mlib_adapter configured!
```

Note the messages above are thrown directly by mlib.

Date schedules can be created using mlib by providing key value pairs

```
# mlib connection

date_schedule_generator_input_list_names <- c("EffectiveDate",
      "MaturityDate", "AccrualDayCount", "AccrualFrequency", "AccrualBusinessDayAdjustment",
      "AccrualCalendar", "PaymentFrequency", "PaymentBusinessDayAdjustment",
      "PaymentCalendar", "PaymentLag", "FixingBusinessDayAdjument",
      "FixingCalendar", "FixingLag", "FirstStubDate", "LastStubDate",
      "RollDay")

date_schedule_generator_input_list_values <- c("20160501", "20200522",
      "30/360", "Semi-Annual", "NO_CHANGE", "TGT", "Semi-Annual",
      "NO_CHANGE", "TGT", "OD", "NO_CHANGE", "TGT", "OD", "", "",
      "")

date_schedule_generator_input_list <- as.list(date_schedule_generator_input_list_values)
names(date_schedule_generator_input_list) <- date_schedule_generator_input_list_names

mlib_dates <- m$date_schedule_generator(date_schedule_generator_input_list,
      hyphens = FALSE)
pander_split(mlib_dates)
```

FixingDate	AccrualStart	AccrualEnd	AccrualDays	AccrualYearFractions	PaymentDate
20160501	20160501	20160522	21	0.05833	20160522
20160522	20160522	20161122	184	0.50000	20161122
20161122	20161122	20170522	181	0.50000	20170522
20170522	20170522	20171122	184	0.50000	20171122
20171122	20171122	20180522	181	0.50000	20180522
20180522	20180522	20181122	184	0.50000	20181122
20181122	20181122	20190522	181	0.50000	20190522
20190522	20190522	20191122	184	0.50000	20191122
20191122	20191122	20200522	182	0.50000	20200522

Curves that have been put in the MLIB_CURVES_PATH location are available, in this case we have made available EUR_OIS_CURVE@30.JSON EUR_SWAP_3M_CURVE@18.JSON EURYC_EUR3ML.JSON

We can extract both discount factors and forward rates from the available curves as follows

```
discount <- m$curve_discounts("20210203", "EURYC", "OIS")
discount

## [1] 1.004377936

mlib forward rate for a supplied date.

discount <- m$curve_forward_rates("20210203", "EURYC", "EUR3ML")
discount

## [1] -0.004389776025

Further we can query for discount factors for a schedule of dates

mlib_dates$FixingDate

## [1] "20160501" "20160522" "20161122" "20170522" "20171122" "20180522"
## [7] "20181122" "20190522" "20191122"
```

```
discount_vec <- m$curve_discounts(mlib_dates$FixingDate, "EURYC",
  "OIS")
discount_vec

## [1] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
## [7] 0.000000000 0.000000000 1.000311051
```

We observe historical dates discount factors as 0 which is desired behavior.

The mlib adapter is unit tested in test-loan-by-loan.

3.3 Abstractions

To make the implementation more maintainable a class based approach has been adopted, the R language has a number of class based options s3, s4, R6. R6 is selected for performance reasons. Information in relation to the class systems can be found <https://adv-r.hadley.nz/r6.html>

3.4 Loan

Loans are described by the parameters as per the table below.

Parameter	Description
Term	Term of the loan in months
LoanRate	Coupon payable on the loan
OriginalBalance	Loan balance outstanding at observation date
DefaultVector	Default rates vector, element per month, expressed as an annualised rate
SeverityVector	Recover rate vector, element per month, expressed as a %
PrepaymentVector	Prepayments rates vector, element per month, expressed as an annualised rate
RiskWeight	Regulatory risk weight parameter
KIRB	Regulatory CRR term (8% RWA + EL)
LoanType	Amortising 'A' or Bullet 'B'
LoanFreq	Loan frequency in months

Let us now consider the loan cash-flow data-frame. We can create a loan cash-flow data-frame object via the `loan_amortization_default_prepayment` function subject to a parameterisation as described in the table above.

Additional regulatory parameters are specified such as Risk Weight and KIRB but these can also be calculated based on loan definition found in the CRR. For simplicity in this instance Risk Weight and KIRB have been defined to be constant through the life of the loan, regulatory formulations exist for derivation when PD, LGD, Maturity and Asset type are provided, see CRR article 142.

In addition Vectors for CPR and CDR can be supplied per period to term of the loan, these drive prepayment and loss rates, for simplicity in the example we have illustrated a constant rate in this example.

3.4.1 Projecting Cash-flows

We now examine how the loan model projects cash-flows

Loans are projected with the following field definitions

Parameter	Description
BalanceBegin	Balance of loan at start of projection
DefaultBalance	Default balance of loan in period
BalanceAfterDefault	BalanceBegin - DefaultedBalance
Severity	Loss given default
LossGivenLoss	DefaultBalance * Severity
SeverityRecovered	DefaultBalance * (1 - Severity)
Prepayment	Prepayment of loan in period
BalanceAfterDefaultAndPrepayment	BalanceAfterDefault - Prepayment
LoanPayment	Scheduled payment of loan in period
InterestComponent	InterestComponent of LoanPayment
PrincipalComponent	PrincipalComponent of LoanPayment
BalanceEnd	BalanceAfterDefaultAndPrepayment - PrincipalComponent
RiskWeight	Regulatory risk weight
KIRB	CRR defined KIRB

A **DefaultBalance** is produced subject to the following formulation

$$DefaultedBalance = BalanceBegin * (1 - (1 - CDR)^{(MonthsInPeriod/PeriodsInYear)})$$

The Defaulted balance has two components, **LossGivenLoss** and **SeverityRecovered**. The split is determined by the **Severity** parameter which is analogous to Loss Given Default.

$$LossGivenLoss = DefaultedBalance * Severity$$

$$SeverityRecovered = DefaultedBalance * (1 - Severity)$$

A **Prepayment** for a monthly compounding default-able loan can be formulated as

$$Prepayment = BalanceAfterDefault * (1 - (1 - CPR)^{(MonthsInPeriod/PeriodsInYear)})$$

A scheduled **LoanPayment** can be determined using a PMT function for amortizing loans or zero for bullet excepting payment of principal at maturity.

The **PeriodRate** is described as

$$PeriodRate = LoanRate / (12 / LoanFreq)$$

The pmt factor as

$$pmt.factor = (1 + (PeriodRate))^{Term}$$

The **LoanPayment** therefore follows as

$$LoanPayment = (LoanBalance * pmt.factor) * (PeriodRate / (pmt.factor - 1))$$

3.4.2 Bullet Loans

Let us project an amortising loan having zero default rate amortizing at scheduled payments rate only

```

# create an illustrative loan of term 12 months
loan_test_1Y_b <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0,
    360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanFreq = 1, LoanType = "B")

pander_split((loan_test_1Y_b))

```

Table 4: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100	0	0	0	0
2	100	0	0	0	0
3	100	0	0	0	0
4	100	0	0	0	0
5	100	0	0	0	0
6	100	0	0	0	0
7	100	0	0	0	0
8	100	0	0	0	0
9	100	0	0	0	0
10	100	0	0	0	0
11	100	0	0	0	0
12	100	0	0	0	0

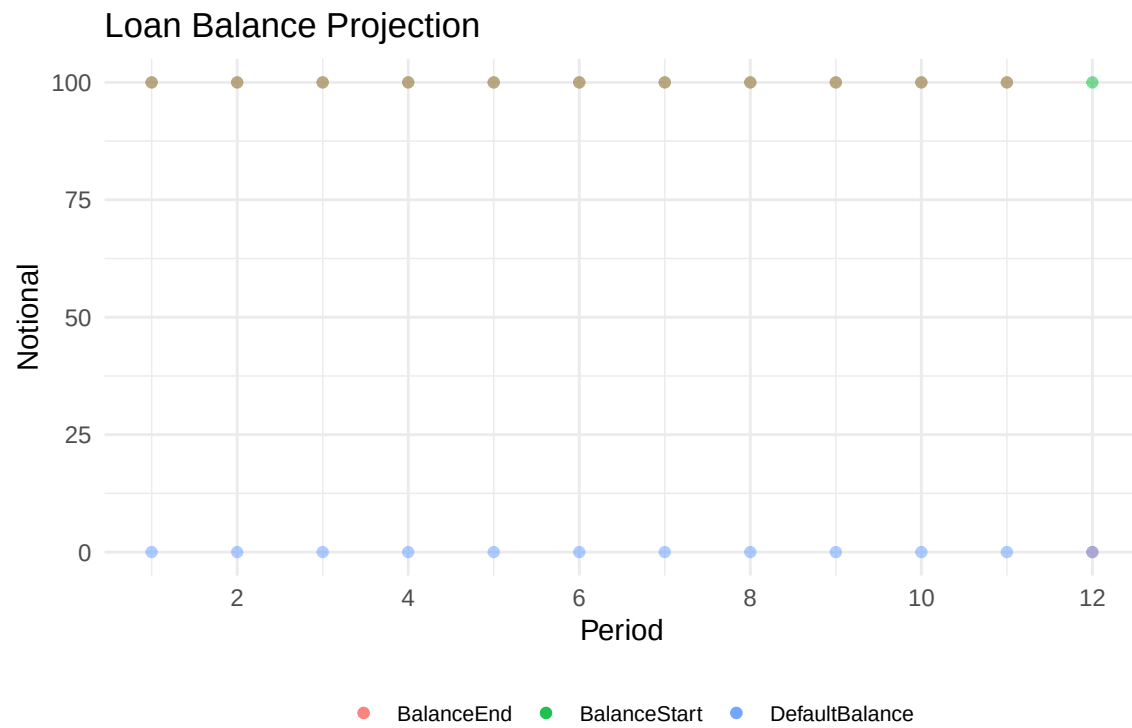
Table 5: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	0
100	0	0.4167	100

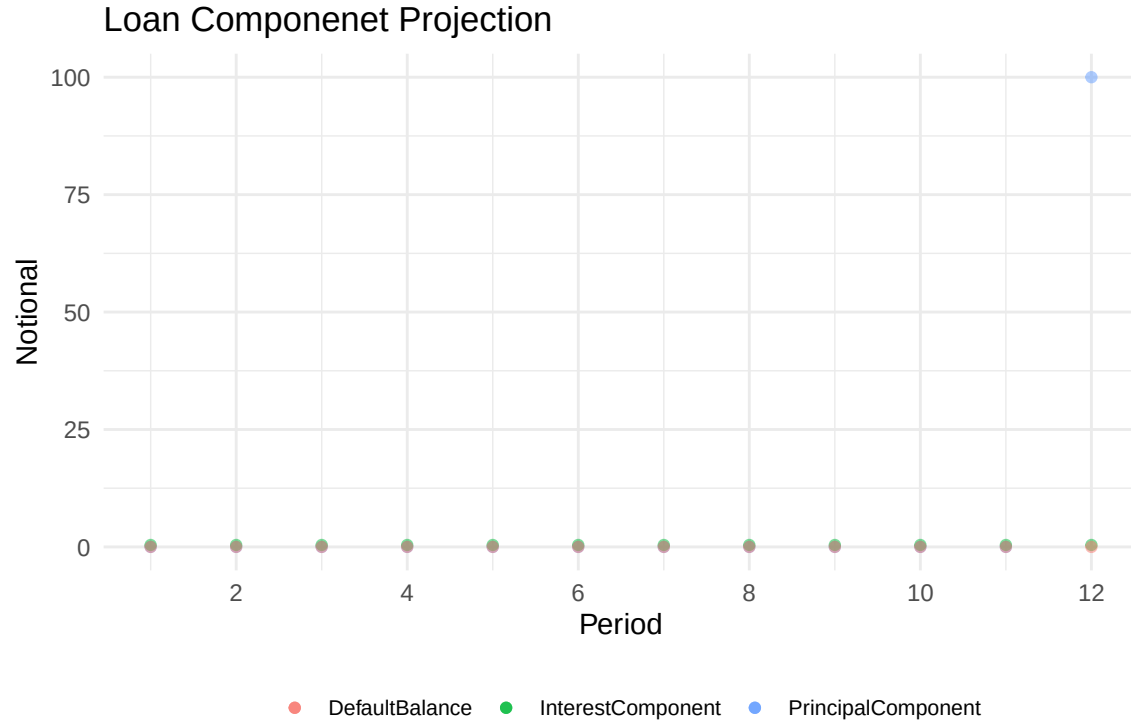
BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
100	0	0.6	0.08
0	0	0.6	0.08

The loan balances can be visualized



The proportion attributed to principal and interest can also be visualized



It is clear from the visualizations above that the bullet loan has principal outstanding until maturity (1Y = 12 months) at which time the full notional is repaid.

3.4.2.1 Non-zero default rate

Let us now introduce a **non zero** level of default (5%) and visualize again.

```
# create an illustrative loan of term 12 months
loan_test_1Y_b_d <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.05,
    360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanType = "B", LoanFreq = 1)

pander_split(loan_test_1Y_b_d)
```

Table 7: Table continues below

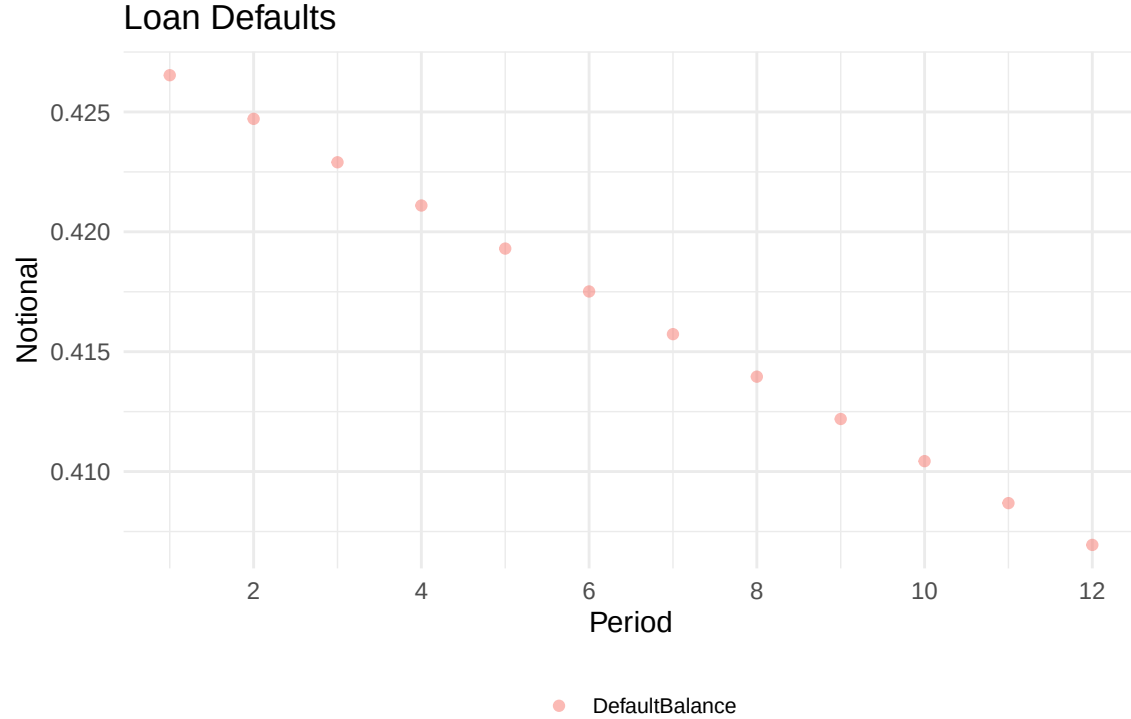
Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0.4265	0.2133	0.2133	0
2	99.57	0.4247	0.2124	0.2124	0
3	99.15	0.4229	0.2115	0.2115	0
4	98.73	0.4211	0.2105	0.2105	0
5	98.30	0.4193	0.2097	0.2097	0
6	97.89	0.4175	0.2088	0.2088	0
7	97.47	0.4157	0.2079	0.2079	0
8	97.05	0.4140	0.2070	0.2070	0
9	96.64	0.4122	0.2061	0.2061	0

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
10	96.23	0.4104	0.2052	0.2052	0
11	95.82	0.4087	0.2043	0.2043	0
12	95.41	0.4069	0.2035	0.2035	0

Table 8: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
99.57	0	0.4167	0
99.15	0	0.4149	0
98.73	0	0.4131	0
98.30	0	0.4114	0
97.89	0	0.4096	0
97.47	0	0.4079	0
97.05	0	0.4061	0
96.64	0	0.4044	0
96.23	0	0.4027	0
95.82	0	0.4009	0
95.41	0	0.3992	0
95.00	0	0.3975	95

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
99.57	0.2133	0.6	0.08
99.15	0.2124	0.6	0.08
98.73	0.2115	0.6	0.08
98.30	0.2105	0.6	0.08
97.89	0.2097	0.6	0.08
97.47	0.2088	0.6	0.08
97.05	0.2079	0.6	0.08
96.64	0.2070	0.6	0.08
96.23	0.2061	0.6	0.08
95.82	0.2052	0.6	0.08
95.41	0.2043	0.6	0.08
0.00	0.2035	0.6	0.08



Note that the default balance is declining in response to the outstanding balance falling as a consequence of the defaults being experienced

For the first entry, recalling the expression for DefaultBalance, therefore we have

$$100 * (1 - (1 - 0.05)^{(1/12)}) = 0.4265318778$$

3.4.3 Recovery Lag

It is sometimes required to lag the realisation of recovery from a defaulted obligation by a specified period, the section below illustrates how this can be achieved. Note that by default recoveries beyond the term of the loan will be lost, however by using the optional parameter KeepPostTermRecoveries as TRUE, the loan data frame is extended and the after term recoveries preserved.

```
# create an illustrative loan of term 12 months
loan_test_1Y_lag <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.05,
    360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanType = "B", LoanFreq = 1, RecoveryLag = 2)

pander_split(loan_test_1Y_lag)
```

Table 10: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0.4265	0.2133	0.0000	0
2	99.57	0.4247	0.2124	0.0000	0
3	99.15	0.4229	0.2115	0.2133	0

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
4	98.73	0.4211	0.2105	0.2124	0
5	98.30	0.4193	0.2097	0.2115	0
6	97.89	0.4175	0.2088	0.2105	0
7	97.47	0.4157	0.2079	0.2097	0
8	97.05	0.4140	0.2070	0.2088	0
9	96.64	0.4122	0.2061	0.2079	0
10	96.23	0.4104	0.2052	0.2070	0
11	95.82	0.4087	0.2043	0.2061	0
12	95.41	0.4069	0.2035	0.2052	0

Table 11: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
99.57	0	0.4167	0
99.15	0	0.4149	0
98.73	0	0.4131	0
98.30	0	0.4114	0
97.89	0	0.4096	0
97.47	0	0.4079	0
97.05	0	0.4061	0
96.64	0	0.4044	0
96.23	0	0.4027	0
95.82	0	0.4009	0
95.41	0	0.3992	0
95.00	0	0.3975	95

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
99.57	0.2133	0.6	0.08
99.15	0.2124	0.6	0.08
98.73	0.2115	0.6	0.08
98.30	0.2105	0.6	0.08
97.89	0.2097	0.6	0.08
97.47	0.2088	0.6	0.08
97.05	0.2079	0.6	0.08
96.64	0.2070	0.6	0.08
96.23	0.2061	0.6	0.08
95.82	0.2052	0.6	0.08
95.41	0.2043	0.6	0.08
0.00	0.2035	0.6	0.08

```
# create an illustrative loan of term 12 months
loan_test_1Y_lag_keep <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.05,
    360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanType = "B", LoanFreq = 1, RecoveryLag = 2, KeepPostTermRecoveries = TRUE)
```

`pander_split(loan_test_1Y_lag_keep)`

Table 13: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0.4265	0.2133	0.0000	0
2	99.57	0.4247	0.2124	0.0000	0
3	99.15	0.4229	0.2115	0.2133	0
4	98.73	0.4211	0.2105	0.2124	0
5	98.30	0.4193	0.2097	0.2115	0
6	97.89	0.4175	0.2088	0.2105	0
7	97.47	0.4157	0.2079	0.2097	0
8	97.05	0.4140	0.2070	0.2088	0
9	96.64	0.4122	0.2061	0.2079	0
10	96.23	0.4104	0.2052	0.2070	0
11	95.82	0.4087	0.2043	0.2061	0
12	95.41	0.4069	0.2035	0.2052	0
13	0.00	0.0000	0.0000	0.2043	0
14	0.00	0.0000	0.0000	0.2035	0

Table 14: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
99.57	0	0.4167	0
99.15	0	0.4149	0
98.73	0	0.4131	0
98.30	0	0.4114	0
97.89	0	0.4096	0
97.47	0	0.4079	0
97.05	0	0.4061	0
96.64	0	0.4044	0
96.23	0	0.4027	0
95.82	0	0.4009	0
95.41	0	0.3992	0
95.00	0	0.3975	95
0.00	0	0.0000	0
0.00	0	0.0000	0

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
99.57	0.2133	0.6	0.08
99.15	0.2124	0.6	0.08
98.73	0.2115	0.6	0.08
98.30	0.2105	0.6	0.08
97.89	0.2097	0.6	0.08
97.47	0.2088	0.6	0.08
97.05	0.2079	0.6	0.08
96.64	0.2070	0.6	0.08
96.23	0.2061	0.6	0.08
95.82	0.2052	0.6	0.08
95.41	0.2043	0.6	0.08

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
0.00	0.2035	0.6	0.08
0.00	0.0000	0.0	0.00
0.00	0.0000	0.0	0.00

3.4.4 Comparision with Schouten, Jonsson 2009

We align the approach to the model paper referenced <https://www.eurandom.tue.nl/reports/2009/050-report.pdf>. Considering the table in section 4.1.1. Replication can be made of the figures given by appropriately parmaterisign the mgen function as follows.

Month	Pool balance (beginning)	Defaulted principal	SMM (%)	Cumulative default rate (%)
1	100,000,000	200,000	0.20	0.2000
2	99,800,000	199,600	0.20	0.3996
3	99,600,400	199,201	0.20	0.5988
⋮	⋮	⋮	⋮	⋮
58	89,037,182	178,431	0.20	10.9628
59	88,859,108	178,074	0.20	11.1409
60	88,681,390	177,718	0.20	11.3186
61	88,504,027	177,363	0.20	11.4960
62	88,327,019	177,008	0.20	11.6730
⋮	⋮	⋮	⋮	⋮
119	78,801,487	157,919	0.20	21.1985
120	78,643,884	157,603	0.20	21.3561

Table extract from Jonsson & Schoutens, 2009, page 14

```
# default at 5%, recovery 50%
smm = 0.002
cdr = 1 - (1 - smm)^12

# create an illustrative loan of term 12 months
loan_test_eib1 <- loan_amortization_default_prepayment(Term = 120,
  LoanRate = 0.05, OriginalBalance = 1e+08, DefaultVector = rep(cdr,
    360), SeverityVector = rep(1, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanType = "B", LoanFreq = 1)

pander_split(loan_test_eib1[c(1:3, 58:62, 119:120), c("Period",
  "BalanceStart", "BalanceEnd", "DefaultBalance")])
```

	Period	BalanceStart	BalanceEnd	DefaultBalance
1	1	100,000,000	99,800,000	200,000

	Period	BalanceStart	BalanceEnd	DefaultBalance
2	2	99,800,000	99,600,400	199,600
3	3	99,600,400	99,401,199	199,201
58	58	89,215,611	89,037,180	178,431
59	59	89,037,180	88,859,105	178,074
60	60	88,859,105	88,681,387	177,718
61	61	88,681,387	88,504,024	177,363
62	62	88,504,024	88,327,016	177,008
119	119	78,959,406	78,801,487	157,919
120	120	78,801,487	0	157,603

It should be noted that Month 1-3 are in exact agreement, however 58-62, 119-120 in the referenced report appears to give the BalanceEnd figure rather than balance start.

We can also make comparison to the report section 4.1.2, by using the flag CumulativeDefault=TRUE in loan_amortization_default_prepayment it indicates that the original balance should be used for all calculations.

Month	Pool balance (beginning)	Defaulted principal	SMM (%)	Cumulative default rate (%)
1	100,000,000	200,000	0.2000	0.20
2	99,800,000	200,000	0.2004	0.40
3	99,600,000	200,000	0.2008	0.60
⋮	⋮	⋮	⋮	⋮
58	88,600,000	200,000	0.2257	11.60
59	88,400,000	200,000	0.2262	11.80
60	88,200,000	200,000	0.2268	12.00
61	88,000,000	200,000	0.2273	12.20
62	87,800,000	200,000	0.2278	12.40
⋮	⋮	⋮	⋮	⋮
119	76,400,000	200,000	0.2618	23.8
120	76,200,000	200,000	0.2625	24.0

Table extract from Jonsson & Schoutens, 2009, page 15

Calculating in mgen we observe

```
# default at 5%, recovery 50%
smm = 0.002
cdr = 1 - (1 - smm)^12

# create an illustrative loan of term 12 months
loan_test_eib2 <- loan_amortization_default_prepayment(Term = 120,
  LoanRate = 0.05, OriginalBalance = 1e+08, DefaultVector = rep(cdr,
```

```
360), SeverityVector = rep(1, 360), PrepaymentVector = rep(0,
360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
LoanType = "B", LoanFreq = 1, CumulativeDefault = TRUE)
```

```
pander_split(loan_test_eib2[c(1:3, 58:62, 119:120), c("Period",
"BalanceStart", "BalanceEnd", "DefaultBalance")])
```

	Period	BalanceStart	BalanceEnd	DefaultBalance
1	1	100,000,000	99,800,000	200,000
2	2	99,800,000	99,600,000	200,000
3	3	99,600,000	99,400,000	200,000
58	58	88,600,000	88,400,000	200,000
59	59	88,400,000	88,200,000	200,000
60	60	88,200,000	88,000,000	200,000
61	61	88,000,000	87,800,000	200,000
62	62	87,800,000	87,600,000	200,000
119	119	76,400,000	76,200,000	200,000
120	120	76,200,000	0	200,000

3.4.5 Amortising Loans

Let us project an amortizing loan having zero default rate amortizing at scheduled payments rate only

```
# create an illustrative loan of term 12 months
loan_test_1Y <- loan_amortization_default_prepayment(Term = 12,
LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0,
360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
LoanType = "A", LoanFreq = 1)
```

```
pander_split(loan_test_1Y)
```

Table 18: Table continues below

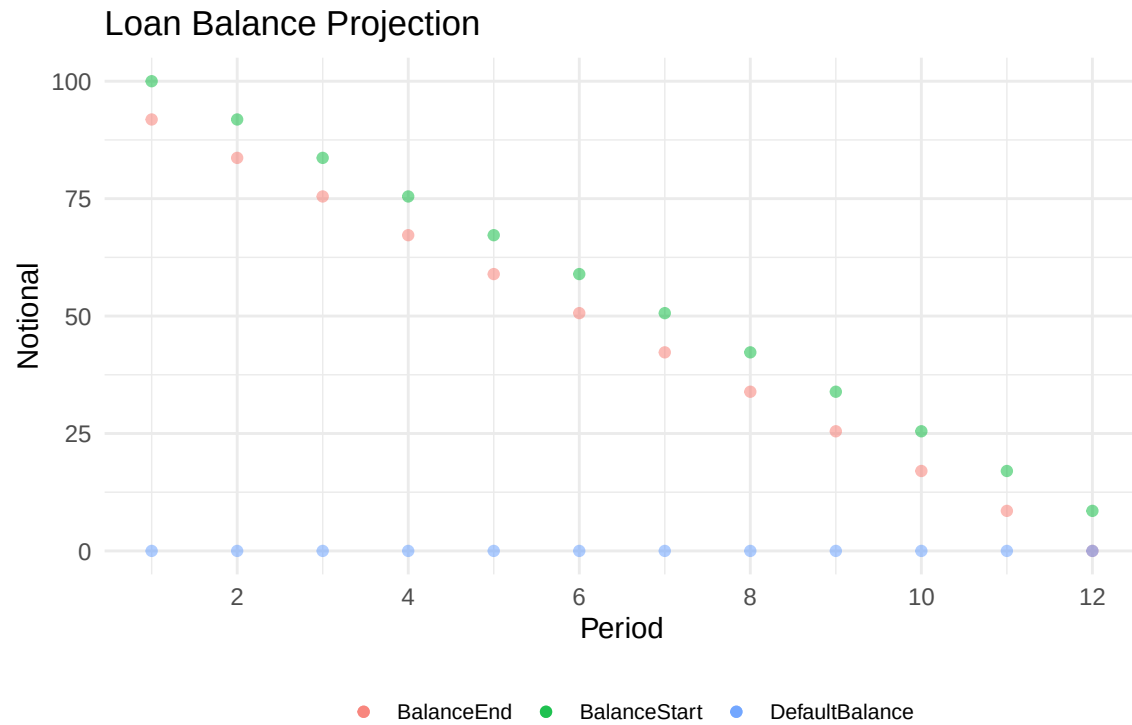
Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.000	0	0	0	0
2	91.856	0	0	0	0
3	83.678	0	0	0	0
4	75.466	0	0	0	0
5	67.220	0	0	0	0
6	58.939	0	0	0	0
7	50.624	0	0	0	0
8	42.274	0	0	0	0
9	33.889	0	0	0	0
10	25.470	0	0	0	0
11	17.015	0	0	0	0
12	8.525	0	0	0	0

Table 19: Table continues below

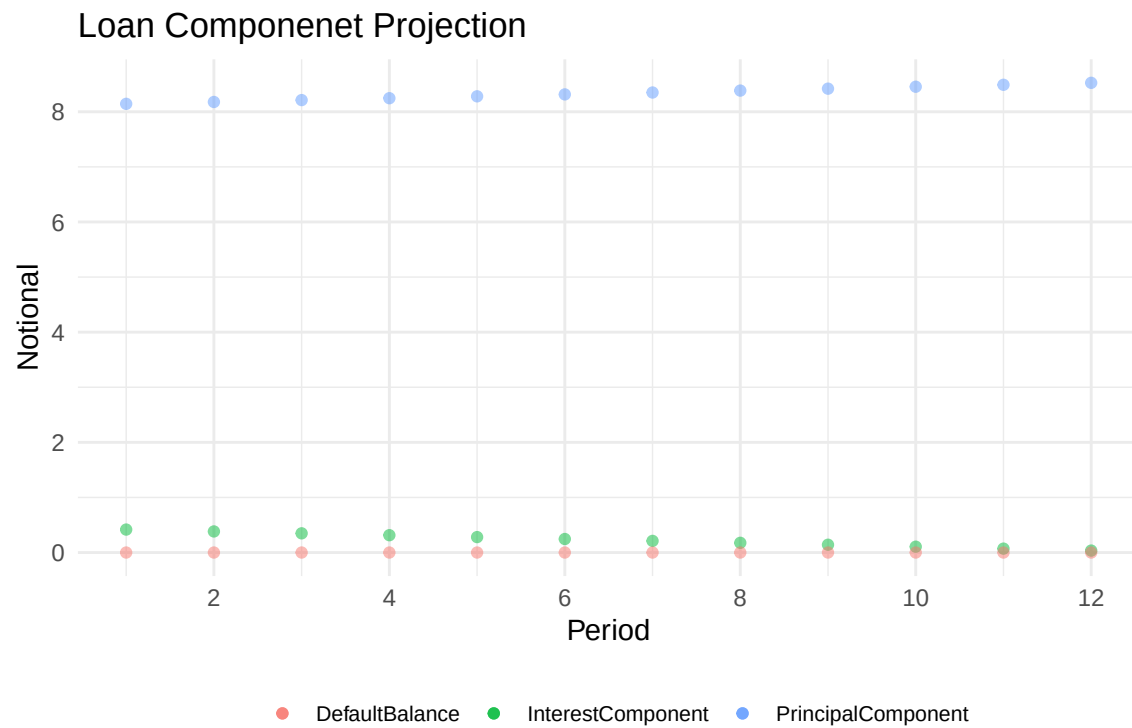
BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
100.000	8.561	0.41667	8.144
91.856	8.561	0.38273	8.178
83.678	8.561	0.34866	8.212
75.466	8.561	0.31444	8.246
67.220	8.561	0.28008	8.281
58.939	8.561	0.24558	8.315
50.624	8.561	0.21093	8.350
42.274	8.561	0.17614	8.385
33.889	8.561	0.14121	8.420
25.470	8.561	0.10612	8.455
17.015	8.561	0.07090	8.490
8.525	8.561	0.03552	8.525

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
91.8559184878199204	0	0.6	0.08
83.6779033026724477	0	0.6	0.08
75.4658130542535019	0	0.6	0.08
67.2195057631328297	0	0.6	0.08
58.9388388582991567	0	0.6	0.08
50.6236691746953298	0	0.6	0.08
42.2738529507432119	0	0.6	0.08
33.8892458258579623	0	0.6	0.08
25.4697028379522763	0	0.6	0.08
17.0150784209303438	0	0.6	0.08
8.5252264021709490	0	0.6	0.08
-0.00000000000000302	0	0.6	0.08

We can visualize the projected flows



The proportion attributed to principal and interest can also be visualized



3.4.5.1 Non zero default rates

The impact of introducing non zero default rates can be calculated, this parameter is specified in absolute level for a single loan, if using a portfolio there it can be used to introduced stress factors, this is explained in the portfolio section.

Let us now consider a 5% annualized loss rate with loss severity 50%

```
loan_test_1Y_stress <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.05,
    360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0,
    360), RiskWeight = rep(0.6, 360), KIRB = rep(0.08, 360),
  LoanType = "A", LoanFreq = 1)

pander_split(loan_test_1Y_stress)
```

Table 21: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.000	0.42653	0.21327	0.21327	0
2	91.464	0.39012	0.19506	0.19506	0
3	82.966	0.35387	0.17694	0.17694	0
4	74.504	0.31778	0.15889	0.15889	0
5	66.080	0.28185	0.14093	0.14093	0
6	57.693	0.24608	0.12304	0.12304	0
7	49.342	0.21046	0.10523	0.10523	0
8	41.028	0.17500	0.08750	0.08750	0
9	32.750	0.13969	0.06984	0.06984	0
10	24.508	0.10454	0.05227	0.05227	0
11	16.303	0.06954	0.03477	0.03477	0
12	8.134	0.03469	0.01735	0.01735	0

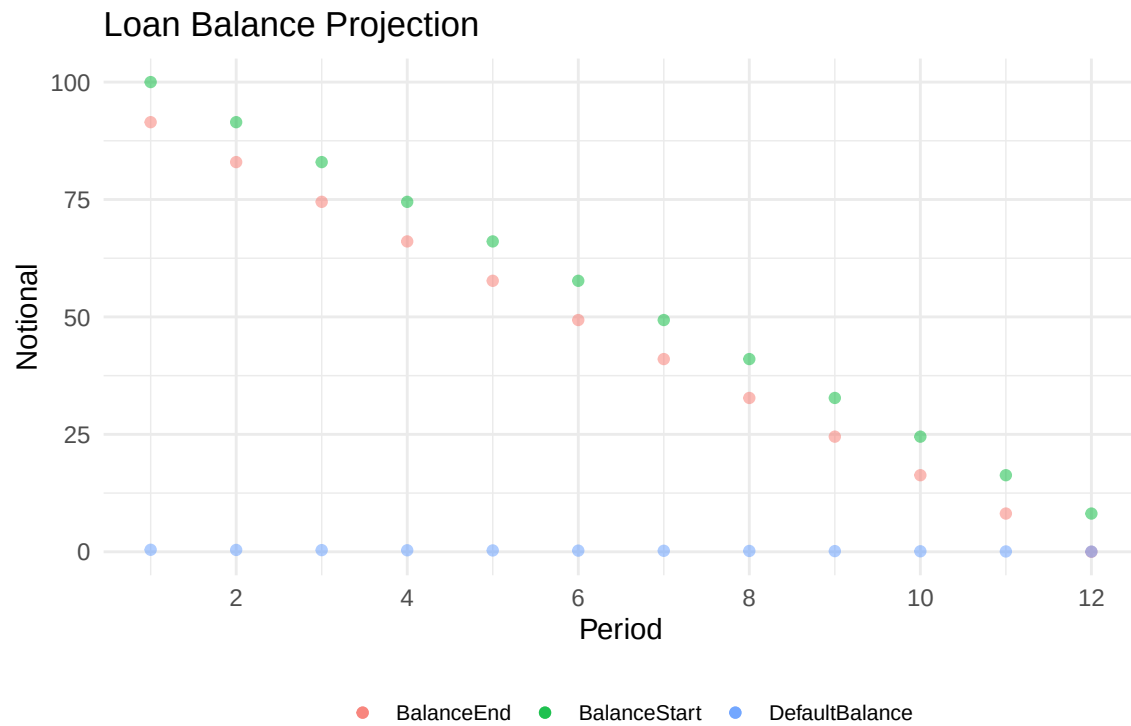
Table 22: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
99.573	8.524	0.41667	8.109
91.074	8.488	0.38110	8.108
82.612	8.452	0.34569	8.107
74.186	8.416	0.31043	8.107
65.798	8.380	0.27533	8.106
57.446	8.344	0.24039	8.105
49.131	8.308	0.20559	8.104
40.853	8.273	0.17095	8.103
32.610	8.238	0.13646	8.102
24.404	8.203	0.10212	8.101
16.234	8.168	0.06793	8.100
8.099	8.133	0.03389	8.099

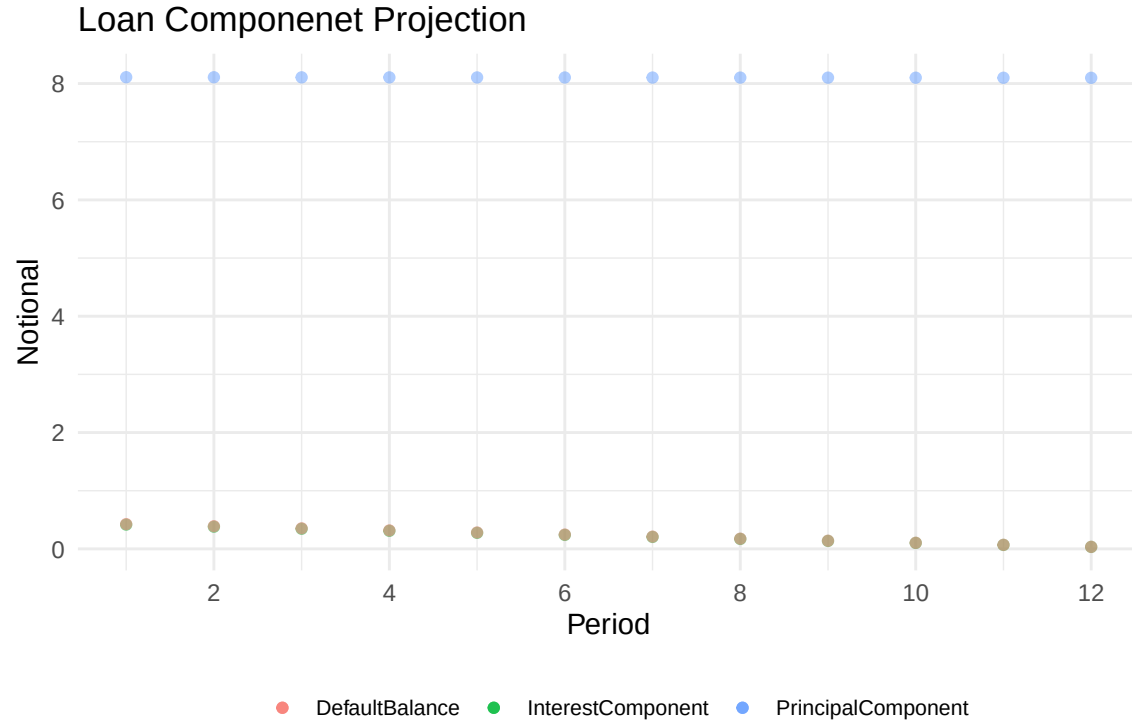
BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
91.46412371386374218	0.21327	0.6	0.08
82.96559978565663585	0.19506	0.6	0.08
74.50426878574594980	0.17694	0.6	0.08

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
66.07997196453007405	0.15889	0.6	0.08
57.69255124953775749	0.14093	0.6	0.08
49.34184924254005011	0.12304	0.6	0.08
41.02770921667449500	0.10523	0.6	0.08
32.74997511358132840	0.08750	0.6	0.08
24.50849154055221035	0.06984	0.6	0.08
16.30310376769119785	0.05227	0.6	0.08
8.13365772508747398	0.03477	0.6	0.08
-0.00000000000002842	0.01735	0.6	0.08

Similarly this can now be visualized accordingly, recalling that each period corresponds to 1 month.



The non zero default balance reflects the annualized 5% loss rate with recovery of 50%.



3.4.5.2 Prepayments

Let us now introduce prepayments to the loan projection. We assume a level of 10% (annualized) prepayment for the loan

```
loan_test_1Y_prep <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0,
    120), SeverityVector = rep(0.5, 120), PrepaymentVector = rep(0.1,
    120), RiskWeight = rep(0.6, 120), KIRB = rep(0.08, 120),
  LoanType = "A", LoanFreq = 1)

first_prepay = loan_test_1Y_prep$Prepayment[1]

pander_split(loan_test_1Y_prep)
```

Table 24: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0	0	0	0.87416
2	91.05	0	0	0	0.79595
3	82.22	0	0	0	0.71875
4	73.50	0	0	0	0.64254
5	64.90	0	0	0	0.56733
6	56.41	0	0	0	0.49309
7	48.03	0	0	0	0.41982
8	39.75	0	0	0	0.34751
9	31.59	0	0	0	0.27615
10	23.53	0	0	0	0.20573
11	15.58	0	0	0	0.13624

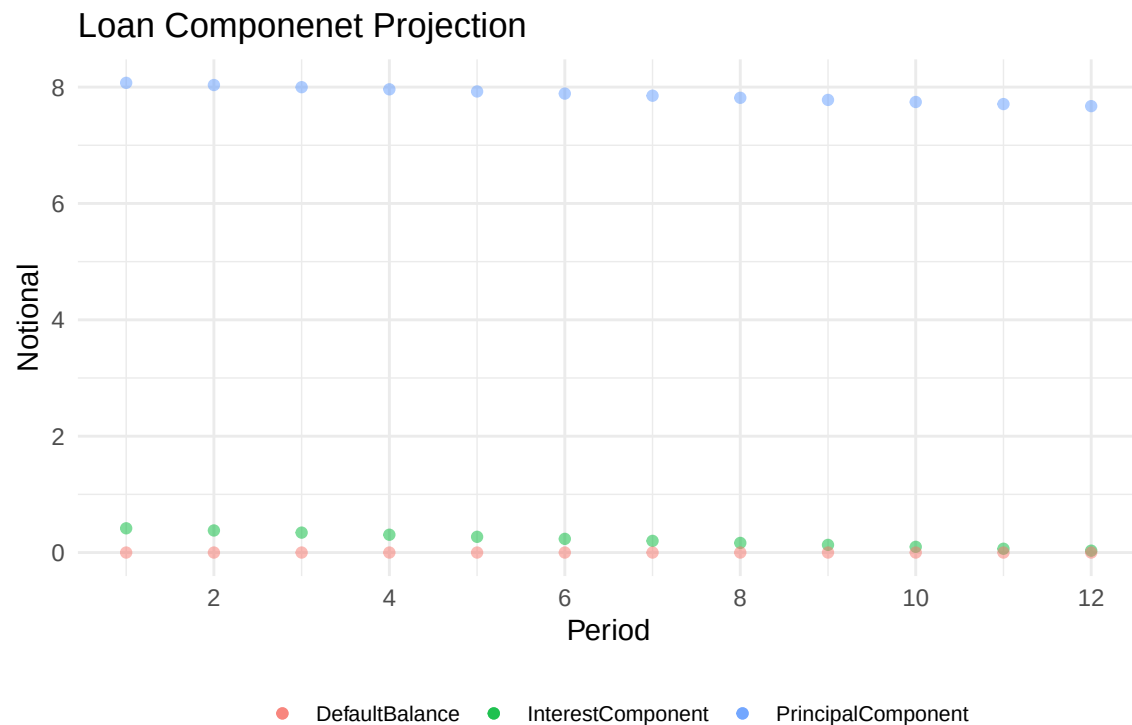
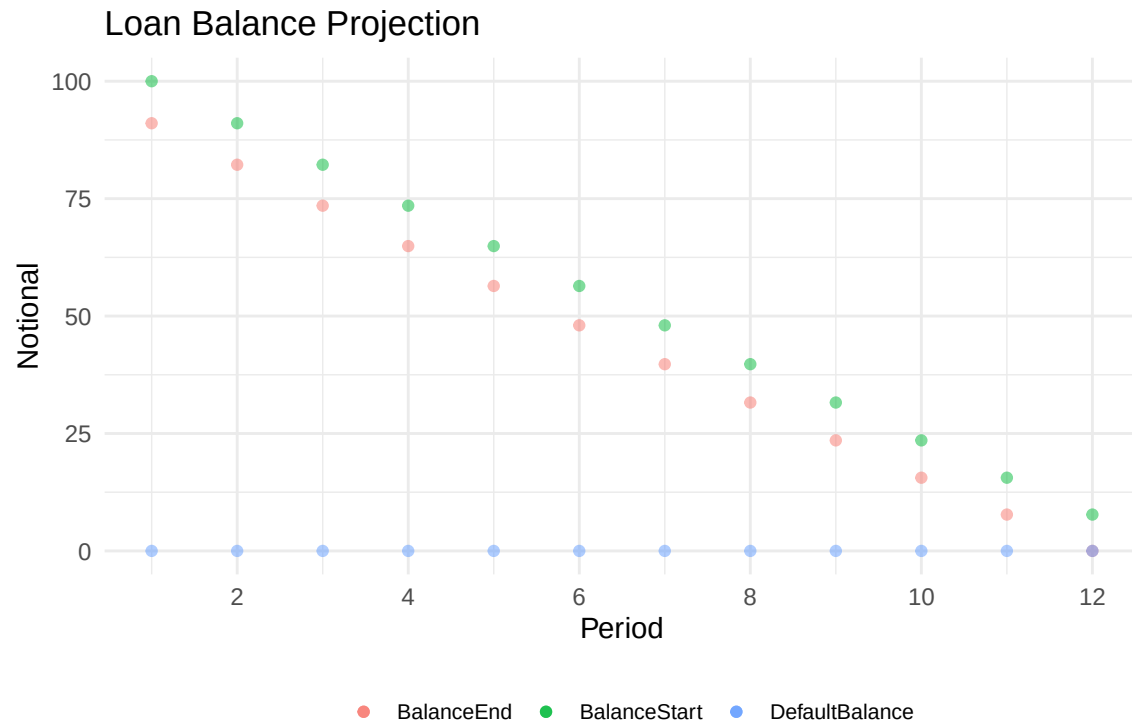
Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
12	7.74	0	0	0	0.06766

Table 25: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
99.126	8.486	0.41667	8.073
90.257	8.412	0.37939	8.036
81.503	8.338	0.34259	7.999
72.861	8.265	0.30627	7.962
64.332	8.193	0.27042	7.925
55.914	8.121	0.23503	7.888
47.606	8.050	0.20011	7.852
39.406	7.980	0.16564	7.816
31.314	7.910	0.13163	7.780
23.329	7.841	0.09806	7.744
15.449	7.773	0.06494	7.708
7.673	7.705	0.03225	7.673

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
91.05294978451306065	0	0.6	0.08
82.22133826106946231	0	0.6	0.08
73.50398464187398417	0	0.6	0.08
64.89971925274170417	0	0.6	0.08
56.40738343229656948	0	0.6	0.08
48.02582943206810029	0	0.6	0.08
39.75392031747824007	0	0.6	0.08
31.59052986971009602	0	0.6	0.08
23.53454248845146068	0	0.6	0.08
15.58485309550507125	0	0.6	0.08
7.74036703925760161	0	0.6	0.08
-0.00000000000002753	0	0.6	0.08

Visualizing



Note that even at a 10% prepayment rate the quantum of prepayment is small. This is because in a loan amortizing over a one year period the loan balance will quickly fall to zero through scheduled amortization. If we look at the loan data-frame we can see that the prepayment in period 1 is 0.87 on a balance of 100. If this level were maintained for all 12 months you would experience $12 \times 0.87 \sim 10$

Recall for a monthly compounding loan we can describe the payment due as

$$Prepayment = BalanceAfterDefault * (1 - (1 - CPR)^{(MonthsInPeriod/PeriodsInYear)})$$

For the first entry therefore we have

$$100 * (1 - (1 - 0.05)^{(1/12)}) = 0.8741610955$$

Which can be compared with the value in the loan data frame of `loan_test_1Y_prep$Prepayment[1] = 0.8741610955`

```
pander_split(loan_test_1Y_prep[c("Period", "Prepayment")])
```

Period	Prepayment
1	0.87416
2	0.79595
3	0.71875
4	0.64254
5	0.56733
6	0.49309
7	0.41982
8	0.34751
9	0.27615
10	0.20573
11	0.13624
12	0.06766

3.4.6 Floating Rate Loans

To use projected forwards connection to mlib is required, the loan forwards can be extracted from mlib as previously detailed.

We can apply these forwards to loans as follows

```
# TODO: floating rate loan using spr+forwards Mlib fields are
# 'ANNUAL', 'SEMI_ANNUAL', 'QUARTERLY', 'MONTHLY', 'WEEKLY', 'DAILY' or 'AT_MATURITY'
```

```
mlob_loan_dates <- m$date_schedule_generator_defaults(effective_date = "2019-05-01",
  maturity_date = "2020-05-01", frequency = "MONTHLY", hyphens = FALSE)
```

```
mlob_loan_dates
```

```
##      FixingDate AccrualStart AccrualEnd AccrualDays AccrualYearFractions
## 1  20190502    20190501    20190603         33      0.09166666667
## 2  20190603    20190603    20190701         28      0.07777777778
## 3  20190701    20190701    20190801         31      0.08611111111
## 4  20190801    20190801    20190902         32      0.08888888889
## 5  20190902    20190902    20191001         29      0.08055555556
## 6  20191001    20191001    20191101         31      0.08611111111
## 7  20191101    20191101    20191202         31      0.08611111111
## 8  20191202    20191202    20200102         31      0.08611111111
## 9  20200102    20200102    20200203         32      0.08888888889
## 10 20200203    20200203    20200302         28      0.07777777778
```

```
## 11 20200302 20200302 20200401 30 0.08333333333
## 12 20200401 20200401 20200504 33 0.09166666667
## PaymentDate
## 1 20190603
## 2 20190701
## 3 20190801
## 4 20190902
## 5 20191001
## 6 20191101
## 7 20191202
## 8 20200102
## 9 20200203
## 10 20200302
## 11 20200401
## 12 20200504
```

```
discount_vec <- m$curve_discounts(mlib_dates$FixingDate, "EURYC",
  "OIS")
pander_split(fwd_vec <- m$curve_discounts(mlib_dates$FixingDate,
  "EURYC", "EUR3ML"))
```

0, 0, 0, 0, 0, 0, 0, 0 and 1 The discount rates are

```
discount_vec
```

```
## [1] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
## [7] 0.000000000 0.000000000 1.000311051
```

The forward rates are

```
fwd_vec
```

```
## [1] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
## [7] 0.000000000 0.000000000 1.000387813
```

Note historical forwards are currently shown as 0, the mlb team is adding the fixings table access functionality to be exposed to R, thereafter historical fixings will be given for historical forwards.

3.4.6.1 Quarterly loans

Let us now illustrate quarterly loans, the accompanying workbook in “c:\dev\mgen\files\pmt functions.xls” gives some examples for comparison in excel.

```
loan_test_1Y_prep_3m <- loan_amortization_default_prepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0,
    120), SeverityVector = rep(0.5, 120), PrepaymentVector = rep(0,
    120), RiskWeight = rep(0.6, 120), KIRB = rep(0.08, 120),
  LoanType = "A", LoanFreq = 3)

pander_split(loan_test_1Y_prep_3m)
```

Table 28: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0	0	0	0
2	100.00	0	0	0	0

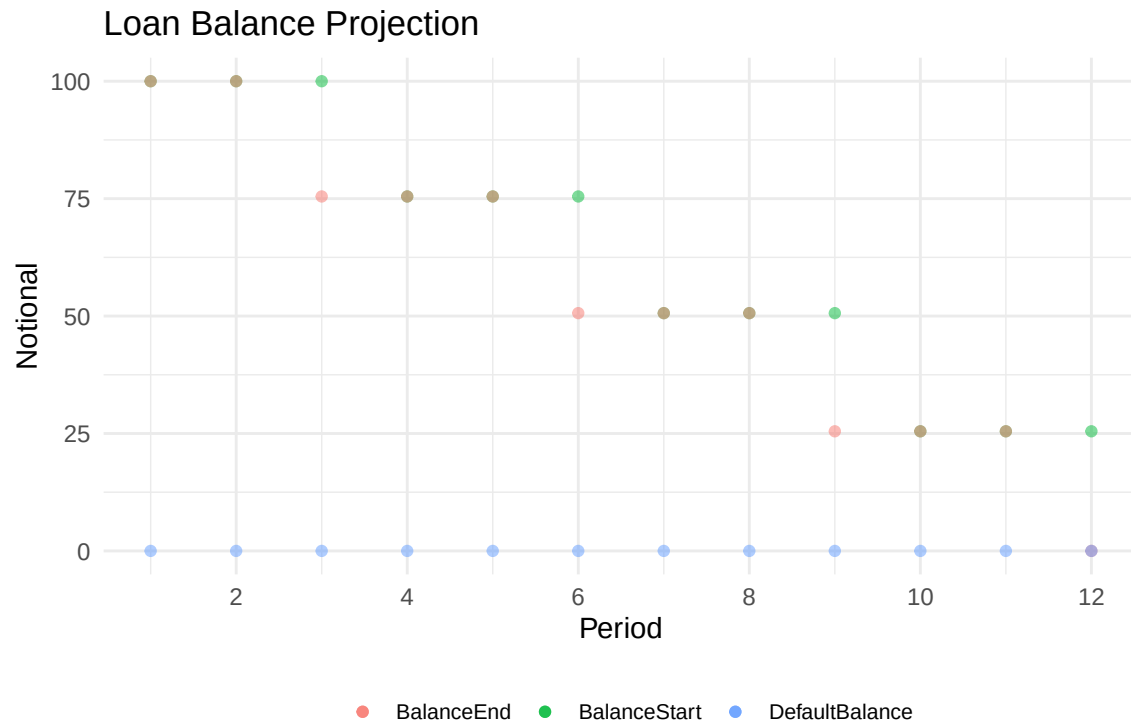
Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
3	100.00	0	0	0	0
4	75.46	0	0	0	0
5	75.46	0	0	0	0
6	75.46	0	0	0	0
7	50.62	0	0	0	0
8	50.62	0	0	0	0
9	50.62	0	0	0	0
10	25.47	0	0	0	0
11	25.47	0	0	0	0
12	25.47	0	0	0	0

Table 29: Table continues below

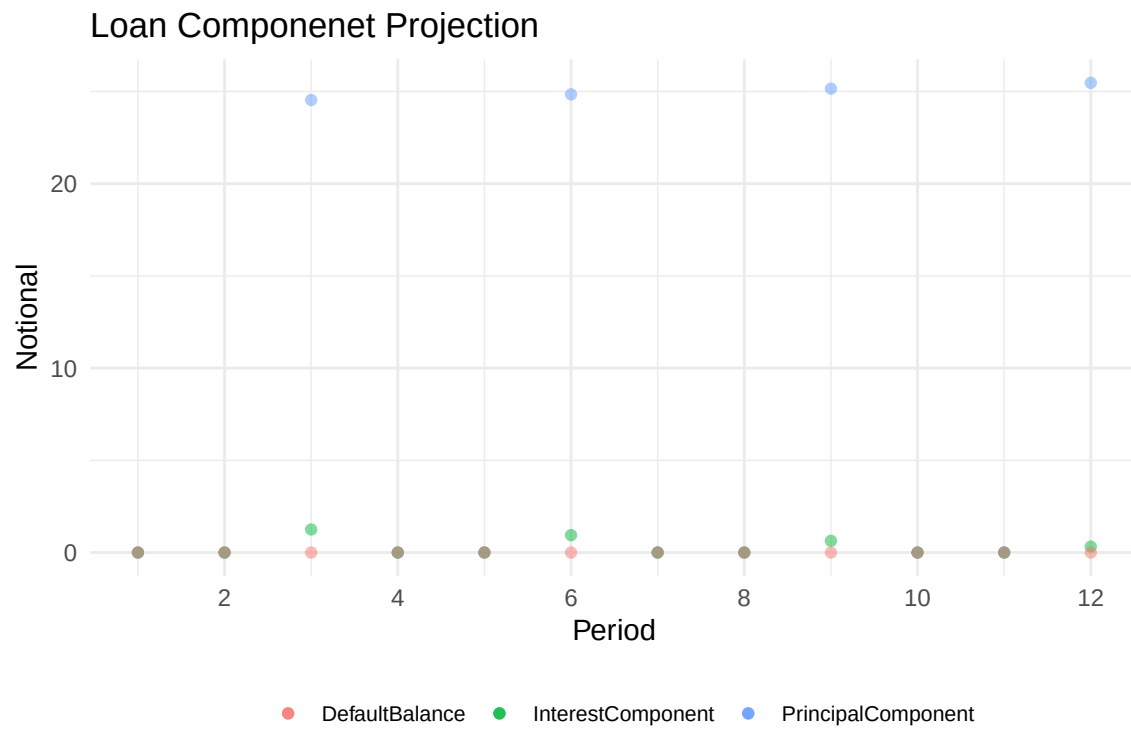
BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
100.00	0.00	0.0000	0.00
100.00	0.00	0.0000	0.00
100.00	25.79	1.2500	24.54
75.46	0.00	0.0000	0.00
75.46	0.00	0.0000	0.00
75.46	25.79	0.9433	24.84
50.62	0.00	0.0000	0.00
50.62	0.00	0.0000	0.00
50.62	25.79	0.6328	25.15
25.47	0.00	0.0000	0.00
25.47	0.00	0.0000	0.00
25.47	25.79	0.3183	25.47

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
100.0000000000000000	0	0.6	0.08
100.0000000000000000	0	0.6	0.08
75.46389766528429277	0	0.6	0.08
75.46389766528429277	0	0.6	0.08
75.46389766528429277	0	0.6	0.08
50.62109405138472340	0	0.6	0.08
50.62109405138472340	0	0.6	0.08
50.62109405138472340	0	0.6	0.08
25.46775539231155960	0	0.6	0.08
25.46775539231155960	0	0.6	0.08
25.46775539231155960	0	0.6	0.08
-0.00000000000000882	0	0.6	0.08

Visualizing the balances



Visualizing the payments



3.5 Portfolios

3.5.1 Portfolio Uploads

Portfolios are typically provided in investor reports in the reference registry section. To upload and evaluate a portfolio of loans it is usually necessary to perform some data wrangling to normalize the format to mgen's expected standard form. A example of a portfolio generated is included within the mgen distribution at location "c:\dev\mgen\files\1.5bil_saved.csv"

The portfolio file format requires fields as follows

Table 31: Table continues below

Field	Description
ID	Unique Identifier
Loan_ID	Loan Specific Identifier
Obligor_ID	Obligor specific identifier
AB	Amortising 0 or Bullet 1 (amortising is default if unspecified)
CAPITAL	Capital
CURRENCY	Currency
EAD	Exposure at default
EL	Expected Loss
FIXFLOAT	Fix or Float
INDUSTRY	Industry group identifier
KIRB	Regulatory KIRB
LGD	Loss given default
LOAN_RATE	Loan interest rate
LOAN_TYPE	Loan Description i.e. Shipping, Mortgage
MATURITY	Maturity as float in years
PD	Probability of Default
PD_REGULATORY	Probability of default regulatory
PEAD	Participation Exposure at default
RATING	Rating
REG_MATURITY	Regulatory maturity floorer at 1y capped at 5y'
RW	Risk Weight

Example
981231jff0dw9u314
Loan_1
Name_1
0
0.08
EUR
100000
0.025
Fix
Real Estate
0.105
0.5
0.05
Mortgage
5.2
0.05

Example

0.03
100000
A1
5
1.00

For performance reasons we use Numba wrapped Numpy implementation to project the flows of large portfolios on a loan by loan basis, this is run server side, currently on MMD06742 presently residing at ip 10.161.7.81 (running on Andrei Preda desktop), it will be migrated to a server once made available by ISD. - We have now made available an rcpp script that remove the need to have a python server.

The portfolio parameters used follow the following chain, if PD exists in the portfolio file then this is used, if absent then PD_REGULATORY will be used, if absent then the user defined default value is used.

Where a value for default is specified and PD or PD_REGULATORY is supplied then default is applied as a multiplier or stress factor.

```
# library(crul) b <- crul::HttpClient$new(url =
# 'http://10.161.7.81:5000') res <- b$get('status')
# res$parse(encoding = 'utf-8') start_time = Sys.time() # if
# default is zero then use file based PD, otherwise apply
# res_csv <- b$post('api/portfolios/amortization', body =
# list( default = 0.00, severity = 0.5, prepayment = 0.125,
# rw = 0.2, kirb = 0.6, csvfile =
# crul::upload('C:\\dev\\mgen\\files\\1.5bil_saved.csv'))
# df <- read.csv(text = res_csv$parse(encoding = 'utf-8'),
# row.names = 1) end_time = Sys.time() pander_split(head(df))

df_40k_loans <- read.csv("../files/1.5bil_saved.csv")
df_40k_loans$MATURITY_MONTH <- df_40k_loans$MATURITY * 12
# 0 -> A (Amortizing) or 1 -> B (Bullet)
df_40k_loans$LOAN_TYPE <- 0

start_time = Sys.time()

# stressing default and severity
res_df <- loans_amortization_cpp(df = df_40k_loans, loan_rate = 0.05,
  default = 0, severity = 0.5, prepayment = 0.125, loan_freq = 1,
  cumulative_default = F, recovery_lag = 0, keep_post_term_recoveries = F)

end_time = Sys.time()
pander_split(head(res_df))
```

Table 33: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	1,490,955,242	0	0	0	16,498,811
2	1,463,016,543	0	0	0	16,189,643
3	1,437,769,236	0	0	0	15,910,258
4	1,413,304,791	0	0	0	15,639,536
5	1,388,125,367	0	0	0	15,360,902

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
6	1,363,796,576	0	0	0	15,091,682

Table 34: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
1,474,456,431	0	0	0
1,446,826,899	0	0	0
1,421,858,978	0	0	0
1,397,665,255	0	0	0
1,372,764,464	0	0	0
1,348,704,894	0	0	0

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
0	0	0.9406	0.09650
0	0	0.9414	0.09656
0	0	0.9413	0.09652
0	0	0.9418	0.09654
0	0	0.9421	0.09653
0	0	0.9420	0.09648

It can be seen that the portfolio of 40,067 loans ran in 3.64134788513184 seconds.

We can run stress tests on the portfolio by applying a multiplier of 100% to the default rates

```
# library(crul) b <- crul::HttpClient$new(url =
# 'http://10.161.7.81:5000') res <- b$get('status')
# res$parse(encoding = 'utf-8') start_time = Sys.time() #
# stressing default and severity res_csv <-
# b$post('api/portfolios/amortization', body = list( default
# = 3, severity = 0.2, prepayment = 0.125, rw = 0.2, kirb =
# 0.6, csvfile =
# crul::upload('C:\\dev\\mgen\\files\\1.5bil_saved.csv'))
# df <- read.csv(text = res_csv$parse(encoding = 'utf-8'),
# row.names = 1) end_time = Sys.time() pander_split(head(df))

df_40k_loans <- read.csv("../files/1.5bil_saved.csv")
df_40k_loans$MATURITY_MONTH <- df_40k_loans$MATURITY * 12
# 0 -> A (Amortizing) or 1 -> B (Bullet)
df_40k_loans$LOAN_TYPE <- 0

# stressing default and severity
res_df <- loans_amortization_cpp(df = df_40k_loans, loan_rate = 0.05,
  default = 3, severity = 0.2, prepayment = 0.125, loan_freq = 1,
  cumulative_default = F, recovery_lag = 0, keep_post_term_recoveries = F)

pander_split(head(res_df))
```

Table 36: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	1,490,955,242	28,143,052	2,251,444	25,891,608	16,187,382
2	1,435,409,693	25,724,395	2,057,952	23,666,443	15,599,483
3	1,385,450,281	23,526,369	1,882,110	21,644,259	15,070,959
4	1,338,798,877	21,554,813	1,724,385	19,830,428	14,576,535
5	1,293,912,992	19,728,275	1,578,262	18,150,013	14,100,043
6	1,252,065,512	18,062,611	1,445,009	16,617,602	13,655,394

Table 37: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
1,446,624,808	0	0	0
1,394,085,816	0	0	0
1,346,852,953	0	0	0
1,302,667,529	0	0	0
1,260,084,674	0	0	0
1,220,347,507	0	0	0

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
0	25,891,608	0.9173	0.09363
0	23,666,443	0.8957	0.09092
0	21,644,259	0.8742	0.08823
0	19,830,428	0.8539	0.08570
0	18,150,013	0.8345	0.08327
0	16,617,602	0.8154	0.08090

Let us visualize the new portfolio balance projections

Let us visualize the portfolio components

The consistency of the projection to the R based implementation is verified in the unit test located in “c:\dev\mgen\tests\testthat\test-loan-by-loan.R”

Additional documentation of the numba implementation in separate notebook r markdown file located “c:\dev\mgen\notebooks\loan_by_loan_rpc_HTTP.rmd”

3.5.2 Portfolio Object Instantiation

The portfolio object is used to manage investments in loans, it is possible to add multiple loan data-frames to a portfolio however a portfolio must have at least one loan data-frame in order for a cash-flow model to be able to run. Here under a new portfolio object is created, the loan created above is added as the “seed” and the portfolio object subsequently exposed.

```
# create a new portfolio
myptf <- r_portfolio$new("test_ptf")

# seed a new portfolio with the new loan object created above
myptf$seed_portfolio(loan_test_1Y)
```

```
## INFO [2019-22-10 14:21:33] Portfolio seeded
## INFO [2019-22-10 14:21:33] hello, I am your R6 portfolio manager, my name is test_ptf
# expose the portfolio object
myptf
```

```
## <r_portfolio>
##   Public:
##     calc_principal_collection: function (period)
##     clone: function (deep = FALSE)
##     count_periods: function ()
##     default_stress: -1
##     df: NA
##     get_balance_after_default_and_prepayment: function (period)
##     get_balance_end: function (period)
##     get_balance_start: function (period)
##     get_default_balance: function (period)
##     get_interest_componenet: function (period)
##     get_loan_payment: function (period)
##     get_loss_given_loss: function (period)
##     get_prepayment: function (period)
##     get_principal_component: function (period)
##     get_reinvestment_balance: function (period)
##     get_severity_recovered: function (period)
##     get_wead_kirb: function (period)
##     get_wead_risk_weight: function (period)
##     greet: function ()
##     initialise_df: function (period)
##     initialize: function (name, periods)
##     loans: list
##     migration_matrix: NA
##     name: test_ptf
##     periods:
##     prepayment_stress: 0
##     reinvest_balance: function (period, val)
##     reinvest_balance_ptf: function (period, val)
##     reinvest_balance_replines: function (period, val)
##     reinvest_select_generate: function (val)
##     reinvest_select_replines: function (val)
##     reinvest_select_tape: function (val)
##     reinvest_test: function (period, val)
##     reinvestment_ptf: NA
##     reinvestment_rep_lines: NA
##     seed_portfolio: function (val = numeric())
##     seed_reinvestment_portfolio: function (val)
##     seed_reinvestment_replines: function (val)
##     set_principal_collection: function (period, val)
##     set_reinvestment_balance: function (period, val)
##     severity_stress: -1
```

The portfolio contains the list of loan data-frames added, the first loan being the portfolio at t0, it is possible to add the projection profile for a large volume of loans. We can directly access the first loan object by index into the list.

```
pander_split(head(myptf$loans[[1]]))
```

Table 39: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
1	100.00	0	0	0	0
2	91.86	0	0	0	0
3	83.68	0	0	0	0
4	75.47	0	0	0	0
5	67.22	0	0	0	0
6	58.94	0	0	0	0

Table 40: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
100.00	8.561	0.4167	8.144
91.86	8.561	0.3827	8.178
83.68	8.561	0.3487	8.212
75.47	8.561	0.3144	8.246
67.22	8.561	0.2801	8.281
58.94	8.561	0.2456	8.315

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
91.86	0	0.6	0.08
83.68	0	0.6	0.08
75.47	0	0.6	0.08
67.22	0	0.6	0.08
58.94	0	0.6	0.08
50.62	0	0.6	0.08

3.5.3 Reinvestments

A portfolio can be instructed to make reinvestment at a given period for the balance desired as illustrated below.

The portfolio is extensible to allow a range of reinvestment methods to be implemented

- `reinvest_test()` defines an arbitrary test loan object for reinvestment.
- `reinvest_balance_replines()` specify replines for reinvestment and pro-rata draw from these (similar to Bloomberg cash flow & Intex approach)

The portfolio object design permits future extension methods for;

- `reinvest_select_generate()`: generate hypothetical new loan production and re-select optimal reinvestment plan
- `reinvest_select_tape()`: select reinvestment from unused stock of loans from original tape
- `reinvest_self()`: reinvest into the same asset as that repaying
- `reinvest_worst()`: invest into weakest assets that comply with portfolio criteria

```
# initialise a buffer to handle reinvestments
myptf$initialise_df(120)
```

```
## INFO [2019-22-10 14:21:33] Portfolio initilalised reinvestmet and collections buffer
# reinvest a balance
myptf$reinvest_test(period = 1, 1)

# show the reinvested loan
pander_split(head(myptf$loans[[2]]))
```

Table 42: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
NA	NA	NA	NA	NA	NA
1	1	0	0	0	0.004265

Table 43: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
NA	NA	NA	NA
0.9957	0.9999	0.004167	0.9957

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
NA	NA	NA	NA
-0.0000000000000003553	0	0.6	0.08

It is noteworthy that the first loan record is NA this is because the reinvestment was instructed to commence at the end of period 1 and thus the balance becomes reinvested in period 2, the first period for the loan therefore is available in period 2.

The outstanding balance from the portfolio object for any given time period can be requested via an accessor function.

```
# get balance at the end of period 1
cat("balance at start of period 1: ", myptf$get_balance_start(1),
    "end of period 1: ", myptf$get_balance_end(1), "\n")
```

```
## balance at start of period 1: 100 end of period 1: 91.85591849
```

```
cat("balance at start of period 2: ", myptf$get_balance_start(2),
    "end of period 2: ", myptf$get_balance_end(2))
```

```
## balance at start of period 2: 92.85591849 end of period 2: 83.6779033
```

Period 1 shows an amortized balance of 91.8559184878 This is the amortization of loan 1 at the end of the first period.

3.5.4 Reinvestment replines

Let us now define reinvestment assets in the form of a set of representative line items.

```
# create a new set of reinvestment repline, 50% proportion to
# each
repline_1 <- r_repline$new(name = "repline1", proportion = 0.3,
```

```

    term = 120, balance = 1, rate = 0.05, default_vector = rep(0,
      120), prepayment_vector = rep(0, 120), severity_vector = rep(0.5,
      120), risk_weight = rep(0.6, 120), kirb = rep(0.08, 120),
    loan_type = "A", loan_freq = 3)

## INFO [2019-22-10 14:21:33] hello, from repline object repline1
repline_2 <- r_repline$new(name = "repline2", proportion = 0.7,
  term = 60, balance = 1, rate = 0.05, default_vector = rep(0.02,
    120), prepayment_vector = rep(0, 120), severity_vector = rep(0.5,
    120), risk_weight = rep(0.6, 120), kirb = rep(0.08, 120),
  loan_type = "A", loan_freq = 3)

## INFO [2019-22-10 14:21:33] hello, from repline object repline2
# create loan object for comparison
loan_repline_test <- loan_amortization_default_prepayment(Term = repline_1$term,
  LoanRate = repline_1$rate, OriginalBalance = repline_1$balance,
  DefaultVector = repline_1$default_vector, SeverityVector = repline_1$severity_vector,
  PrepaymentVector = repline_1$prepayment_vector, RiskWeight = repline_1$risk_weight,
  KIRB = repline_1$kirb, LoanType = repline_1$loan_type, LoanFreq = repline_1$loan_freq)

# create a new portfolio to illustrate reinvest replines
myptf_rep <- r_portfolio$new("test_ptf_reinvest_rep")

# Initialise for use outside cashflow engine
myptf_rep$initialise_df(120)

## INFO [2019-22-10 14:21:33] Portfolio initialised reinvestmet and collections buffer
# seed a new portfolio with the new loan object as created
# above
myptf_rep$seed_portfolio(loan_repline_test)

## INFO [2019-22-10 14:21:33] Portfolio seeded
## INFO [2019-22-10 14:21:33] hello, I am your R6 portfolio manager, my name is test_ptf_reinvest_rep
# create repline list
repline_list = list(repline_1, repline_2)

# seed the reinvestment replines with the repline list
myptf_rep$seed_reinvestment_replines(repline_list)

# define a reinvestment of notional 5 in the 3rd period into
# the two described reinvestment replines
myptf_rep$reinvest_balance_replines(3, 2)

## INFO [2019-22-10 14:21:33] reinvesting repline repline1 in period 3
## INFO [2019-22-10 14:21:33] reinvesting repline repline2 in period 3

Let us inspect the first reinvestment repline
pander_split(head(myptf_rep$loans[[2]]))

```

Table 45: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
NA	NA	NA	NA	NA	NA
NA	NA	NA	NA	NA	NA
NA	NA	NA	NA	NA	NA
1	0.6000	0	0	0	0
2	0.5961	0	0	0	0
3	0.5923	0	0	0	0

Table 46: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
NA	NA	NA	NA
NA	NA	NA	NA
NA	NA	NA	NA
0.6000	0.006364	0.002500	0.003864
0.5961	0.006364	0.002484	0.003880
0.5923	0.006364	0.002468	0.003896

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
NA	NA	NA	NA
NA	NA	NA	NA
NA	NA	NA	NA
0.5961	0	0.6	0.08
0.5923	0	0.6	0.08
0.5884	0	0.6	0.08

Let us inspect the second reinvestment repline

```
pander_split(head(myptf_rep$loans[[3]]))
```

Table 48: Table continues below

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment
NA	NA	NA	NA	NA	NA
NA	NA	NA	NA	NA	NA
NA	NA	NA	NA	NA	NA
1	1.400	0.002355	0.001177	0.001177	0
2	1.377	0.002316	0.001158	0.001158	0
3	1.354	0.002278	0.001139	0.001139	0

Table 49: Table continues below

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
NA	NA	NA	NA
NA	NA	NA	NA
NA	NA	NA	NA

BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent
1.398	0.02638	0.005833	0.02055
1.375	0.02633	0.005738	0.02060
1.352	0.02629	0.005642	0.02065

BalanceEnd	SeverityRecoveredNoLag	RiskWeight	KIRB
NA	NA	NA	NA
NA	NA	NA	NA
NA	NA	NA	NA
1.377	0.001177	0.6	0.08
1.354	0.001158	0.6	0.08
1.331	0.001139	0.6	0.08

3.6 Ratings Migration

Typically originating banks keep metrics such as the observed default frequency and ratings migration tables, these are reviewed periodically typically annually and can be incorporated into the portfolio object by splitting each loan into a series of sub loans, one for each rating bucket. At inception the loan is 100% notional in it's starting rating bucket, and 0% in all other rating buckets, with each migration operation performed the loan is probabilistically sub divided into each sub loan bucket.

TODO : Implement a working example

3.7 Tranche

SRT transactions liability side is defined by a series of tranches that comprise the capital structure.

The tranche object is the abstraction that hold tranche details, name, notional size, sequential/pro-rata, fix/float, coupon etc.

```
example_tranche <- r_tranche$new("equity", 2, "sequential", "flt",
  50, TRUE)
```

```
## INFO [2019-22-10 14:21:34] Hello, I am your R6 tranche , my name is equity
## INFO [2019-22-10 14:21:34] ... and my size is 2, my pay type is sequential
## INFO [2019-22-10 14:21:34]
```

```
example_tranche
```

```
## <r_tranche>
##   Public:
##     calculate_pvs: function ()
##     clone: function (deep = FALSE)
##     construct_df: function (periods = numeric())
##     coupon_dcf: NA
##     coupon_floor: TRUE
##     coupon_freq: 0
##     coupon_rate: 50
##     coupon_type: flt
##     curve_container: NA
##     df: NA
##     discount_factor_dm: NA
```

```
## discount_factor_riskless: NA
## discount_margin: NA
## fn_new_test: function ()
## greet: function ()
## initialize: function (name, size, pay_type, coupon_type, coupon_rate, coupon_floor,
## irr: function (call_period, px_date, px_buy)
## name: equity
## npv: function (ir, call_period, px_date)
## npv_dm: function (call_period, px_date)
## npv_solve: function (ir, call_period, px_date, px_buy)
## pay_type: sequential
## payment_date: NA
## principal_payment: NA
## reset_curve: NA
## reset_dates: NA
## reset_rates: NA
## size: 2
## to_labeled_df: function ()
## Private:
## display_columns: list
```

3.8 Triggers

The triggers object is used to control cash-flow switch logic, such as reinvestment requirements.

```
deal_trigger <- r_triggers$new(name = "cum_loss", periods = 120)
```

```
## INFO [2019-22-10 14:21:34] hello, I am your R6 trigger, my name is cum_loss
```

```
deal_trigger$set_threshold(threshold = 1.3, active_period = 24)
deal_trigger
```

```
## <r_triggers>
## Public:
## clone: function (deep = FALSE)
## df: data.frame
## greet: function ()
## initialise_df: function (periods)
## initialize: function (name, periods)
## name: cum_loss
## periods: 120
## set_threshold: function (threshold, active_period)
## set_threshold_range: function (threshold, active_period_start, active_period_end)
## set_threshold_vec: function (vec_thresholds, vec_active)
## verify: function (period, val)
```

3.9 Cashflow Engine

3.9.1 Waterfall

The assignment of losses and principal to the liabilities of a SRT are described by the waterfall. A sequential structure assigns principal first to the senior, then mezzanine and final equity. In a pro-rata structure the principal is assigned to all tranches concurrently in quantum in accordance with their notional size outstanding.

Losses are allocated from the bottom up. The equity tranche absorbing the initial loss until it's notional is exhausted, thereafter losses erode mezzanine and finally the senior tranche.

The cash-flow engine calculation process is as follows:

1. Pay Interest
2. Pay Losses
3. Calculate Principal and Reinvest
4. Pay Principal
5. Calculate new attachment/detachment
6. Calculate regulatory risk weights

3.9.2 Example

We now create a complete cash-flow engine using the objects previously introduced the steps are as follows:

- Create a loan representation (1 loan to many)
- Create a portfolio object to manage loans
- Create a set of tranches that form the liability structure
- Create a triggers set to control the waterfall
- Create a cash-flow engine to process the assets/liabilities

Let us now construct an illustration of a cash-flow model, we consider a sequential structure with three tranches

```
demo_cashflow_engine <- function(demo_equity = 2, demo_mezz = 8,
  demo_snr = 90, stress_cdr = 1, stress_cpr = 1) {
  # we initialise some tranches here - we can extend this to
  # create a tranche classess who have the data here as member
  # data and that have as member data their own cash registers

  # project a loan - for simpllicity assume a single rep line.
  loan_amortization_df_small <- loan_amortization_default_prepayment(Term = 120,
    LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.005 *
      stress_cdr, 120), SeverityVector = rep(0.5, 120),
    PrepaymentVector = rep(0 * stress_cpr, 120), RiskWeight = rep(0.6,
      120), KIRB = rep(0.08, 120), LoanType = "A")

  myptf <- r_portfolio$new("test_ptf")

  # create an initial portfolio
  myptf$seed_portfolio(loan_amortization_df_small)
  print("got here")

  # create a set of tranches to be included in the transaction
  # name, size, pay_type, periods, coupon_type, coupon_rate,
  # coupon_floor, coupon_freq, reset_curve, discount_margin
  equity_tranche <- r_tranche$new(name = "equity", size = demo_equity,
    pay_type = "sequential", coupon_type = "flt", coupon_rate = 50,
    coupon_floor = FALSE, coupon_freq = 3, reset_curve = "EUR3ML",
```

```

    curve_container = "EURYC", discount_margin = 0)

mezz_tranche <- r_tranche$new(name = "mezz", size = demo_mezz,
    pay_type = "sequential", coupon_type = "fix", coupon_rate = 800,
    coupon_floor = FALSE, coupon_freq = 3, reset_curve = "EUR3ML",
    curve_container = "EURYC", discount_margin = 0)

snr_tranche <- r_tranche$new(name = "senior", size = demo_snr,
    pay_type = "sequential", coupon_type = "flt", coupon_rate = 1500,
    coupon_floor = FALSE, coupon_freq = 3, reset_curve = "EUR3ML",
    curve_container = "EURYC", discount_margin = 0)

# bind the tranches into a capital structure
tranche_list <- list(equity_tranche, mezz_tranche, snr_tranche)

# triggers needed
deal_trigger <- r_triggers$new(name = "cum_loss", periods = 120)
deal_trigger$set_threshold(threshold = 1.3, active_period = 24)
triggers_list <- list(deal_trigger)

# instantiate a cashflow engine
my_engine <- r_cashflow_engine$new("myengine")

# mlib_adapter
mllib <- mgen::mllib_adapter$new()

# configure the cashflow engine TODO kirb, elgd, mt, n should
# come from portfolio - the loan object
my_engine$configure(portfolio = myptf, tranches = tranche_list,
    kirb = 0.07, elgd = 0.4, mt = 0.3, n = 500, supervisory_type = "old",
    pool_type = "Retail", pricing_lib = mllib, effective_date = as.Date("2019-05-15"),
    maturity_date = as.Date("2029-05-15"))

# run the cashflow model
my_engine$evaluate()

my_engine
}

```

3.9.3 Cash-flow output

The engine can be interrogated for flow dynamics

```

demo_engine <- demo_cashflow_engine()

## INFO [2019-22-10 14:21:34] Portfolio seeded
## INFO [2019-22-10 14:21:34] hello, I am your R6 portfolio manager, my name is test_ptf
## [1] "got here"
## INFO [2019-22-10 14:21:34] Hello, I am your R6 tranche , my name is equity
## INFO [2019-22-10 14:21:34] ... and my size is 2, my pay type is sequential

```

```

## INFO [2019-22-10 14:21:34]
## INFO [2019-22-10 14:21:34] Hello, I am your R6 tranche , my name is mezz
## INFO [2019-22-10 14:21:34] ... and my size is 8, my pay type is sequential
## INFO [2019-22-10 14:21:34]
## INFO [2019-22-10 14:21:34] Hello, I am your R6 tranche , my name is senior
## INFO [2019-22-10 14:21:34] ... and my size is 90, my pay type is sequential
## INFO [2019-22-10 14:21:34]
## INFO [2019-22-10 14:21:34] hello, I am your R6 trigger, my name is cum_loss
## Hello, from the cashflow engine, my name is myengine.
## INFO [2019-22-10 14:21:35] MLIB Successfully loaded and ready to GO!
## INFO [2019-22-10 14:21:37] Curve EUR_OIS_CURVE@24 loaded from 20190101!
## INFO [2019-22-10 14:21:37] Curve EUR_OIS_CURVE@30 loaded from 20190101!
## INFO [2019-22-10 14:21:37] Curve EUR_OIS_CURVE_MARKETDATA@23 loaded from 20190101!
## INFO [2019-22-10 14:21:37] Curve EUR_SWAP_3M_CURVE@18 loaded from 20190101!
## INFO [2019-22-10 14:21:37] Curve EUR_SWAP_3M_CURVE@22 loaded from 20190101!
## INFO [2019-22-10 14:21:37] Curve EUR_SWAP_3M_CURVE_MARKETDATA@21 loaded from 20190101!
## INFO [2019-22-10 14:21:37] mlib_adapter configured!
## INFO [2019-22-10 14:21:37] Configured
## INFO [2019-22-10 14:21:37] Tranche equity's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:37] Tranche mezz's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:37] Tranche senior's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:37] Portfolio initilalised reinvestmet and collections buffer
## INFO [2019-22-10 14:21:37] Getting df curve and forward rates
## INFO [2019-22-10 14:21:37] Using MLIB version MLIB
pander_split(head(demo_engine$liability_flowsdf, 9))

```

Table 51: Table continues below

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
1	100.00	99.31	0.6437	0
2	99.31	98.63	0.6461	0
3	98.63	97.94	0.6486	0
4	97.94	97.25	0.6510	0
5	97.25	96.55	0.6534	0
6	96.55	95.86	0.6559	0
7	95.86	95.16	0.6583	0
8	95.16	94.46	0.6608	0
9	94.46	93.75	0.6633	0

Table 52: Table continues below

SeverityRecovered	Prepayment	LossGivenLoss	InterestComponent	CumulativeDefaults
0.02088	0	0.02088	0.4167	0.04176
0.02074	0	0.02074	0.4138	0.08324
0.02059	0	0.02059	0.4109	0.12443
0.02045	0	0.02045	0.4081	0.16533
0.02031	0	0.02031	0.4052	0.20594
0.02016	0	0.02016	0.4023	0.24626
0.02002	0	0.02002	0.3994	0.28629
0.01987	0	0.01987	0.3965	0.32603
0.01972	0	0.01972	0.3936	0.36548

Table 53: Table continues below

CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated
0.02088	0.02088	2.000	1.979	0.02088
0.04162	0.04162	1.979	1.958	0.02074
0.06221	0.06221	1.958	1.938	0.02059
0.08266	0.08266	1.938	1.917	0.02045
0.10297	0.10297	1.917	1.897	0.02031
0.12313	0.12313	1.897	1.877	0.02016
0.14315	0.14315	1.877	1.857	0.02002
0.16302	0.16302	1.857	1.837	0.01987
0.18274	0.18274	1.837	1.817	0.01972

Table 54: Table continues below

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.01993	NA	NA
0	0	0.01986	NA	NA
0	0	0.01979	0.0025289	May 15 2019
0	0	0.01972	NA	NA
0	0	0.01965	NA	NA
0	0	0.01958	0.0024500	August 15 2019
0	0	0.01951	NA	NA
0	0	0.01945	NA	NA
0	0	0.01938	0.0003877	November 15 2019

Table 55: Table continues below

EquityResetRates	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0.0000	0.2556	0.9991	0.000	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0.0000	0.2556	1.0003	1.000	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
-0.0042	0.2611	1.0014	1.001	12.5

Table 56: Table continues below

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
NA	NA	NA	NA
0	0.0000000	0.0025266	August 15 2019
NA	NA	NA	NA
NA	NA	NA	NA
0	0.0024507	0.0024507	November 15 2019
NA	NA	NA	NA

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
0	0.0003883	0.0003883	February 17 2020

Table 57: Table continues below

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.979	0.0050000	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.917	0.0050000	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.857	0.0007997	sequential	NA	8	8

Table 58: Table continues below

MezzLossAllocated	MezzPrincipalAllocated	MezzAttach	MezzDettach	MezzCoupon
0	0	0.01993	0.1005	NA
0	0	0.01986	0.1010	NA
0	0	0.01979	0.1015	0.1636
0	0	0.01972	0.1020	NA
0	0	0.01965	0.1025	NA
0	0	0.01958	0.1030	0.1636
0	0	0.01951	0.1036	NA
0	0	0.01945	0.1041	NA
0	0	0.01938	0.1047	0.1583

Table 59: Table continues below

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
May 15 2019	0.0000	0.2556	0.9991	0.000
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
August 15 2019	0.0000	0.2556	1.0003	1.000
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
November 15 2019	-0.0042	0.2611	1.0014	1.001

Table 60: Table continues below

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDDiscountMargin	MezzPaymentDate
8.758	NA	NA	NA	NA

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDiscounMargin	MezzPaymentDate
8.709	NA	NA	NA	NA
8.659	0	0.0000	0.1634	August 15 2019
8.609	NA	NA	NA	NA
8.558	NA	NA	NA	NA
8.507	0	0.1636	0.1636	November 15 2019
8.455	NA	NA	NA	NA
8.403	NA	NA	NA	NA
8.350	0	0.1586	0.1586	February 17 2020

Table 61: Table continues below

MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart	SeniorEnd
NA	NA	sequential	NA	90.00	89.34
NA	NA	sequential	NA	89.34	88.67
8	0.0800	sequential	NA	88.67	88.00
NA	NA	sequential	NA	88.00	87.33
NA	NA	sequential	NA	87.33	86.65
8	0.0800	sequential	NA	86.65	85.98
NA	NA	sequential	NA	85.98	85.30
NA	NA	sequential	NA	85.30	84.62
8	0.0758	sequential	NA	84.62	83.94

Table 62: Table continues below

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.6646	0.1005	1	NA
0	0.6669	0.1010	1	NA
0	0.6691	0.1015	1	3.424
0	0.6714	0.1020	1	NA
0	0.6737	0.1025	1	NA
0	0.6760	0.1030	1	3.348
0	0.6783	0.1036	1	NA
0	0.6807	0.1041	1	NA
0	0.6830	0.1047	1	3.247

Table 63: Table continues below

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
NA	NA	NA	NA
NA	NA	NA	NA
May 15 2019	0.0000	0.2556	0.9991
NA	NA	NA	NA
NA	NA	NA	NA
August 15 2019	0.0000	0.2556	1.0003
NA	NA	NA	NA
NA	NA	NA	NA
November 15 2019	-0.0042	0.2611	1.0014

Table 64: Table continues below

SeniorRiskLessDiscount	SeniorRW	SeniorPrincipalPayment	SeniorPVRiskLess
NA	0.07	NA	NA
NA	0.07	NA	NA
0.000	0.07	2.001	0.000
NA	0.07	NA	NA
NA	0.07	NA	NA
1.000	0.07	2.021	5.370
NA	0.07	NA	NA
NA	0.07	NA	NA
1.001	0.07	2.042	5.297

Table 65: Table continues below

SeniorPVDiscountMargin	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate
NA	NA	NA	NA
NA	NA	NA	NA
5.420	August 15 2019	89.33	0.1500
NA	NA	NA	NA
NA	NA	NA	NA
5.370	November 15 2019	87.33	0.1500
NA	NA	NA	NA
NA	NA	NA	NA
5.297	February 17 2020	85.30	0.1458

SeniorPayType	SeniorNULL
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA

```
pander_split(tail(demo_engine$liability_flowsdf))
```

Table 67: Table continues below

	Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent
115	115	5.980	4.991972375615886470	0.9860
116	116	4.992	4.000191983758174530	0.9897
117	117	4.000	3.005117095422264129	0.9934
118	118	3.005	2.006735188895701238	0.9971
119	119	2.007	1.005033695616996070	1.0009
120	120	1.005	-0.0000000000000003553	1.0046

Table 68: Table continues below

	ReinvestmentBalance	SeverityRecovered	Prepayment	LossGivenLoss
115	0	0.0012488	0	0.0012488
116	0	0.0010424	0	0.0010424
117	0	0.0008353	0	0.0008353
118	0	0.0006275	0	0.0006275
119	0	0.0004190	0	0.0004190
120	0	0.0002099	0	0.0002099

Table 69: Table continues below

	InterestComponent	CumulativeDefaults	CumulativeLoss	CumulativeRecovery
115	0.024919	2.681	1.341	1.341
116	0.020800	2.683	1.342	1.342
117	0.016667	2.685	1.343	1.343
118	0.012521	2.686	1.343	1.343
119	0.008361	2.687	1.344	1.344
120	0.004188	2.688	1.344	1.344

Table 70: Table continues below

	EquityStart	EquityEnd	EquityLossAllocated	EquityPrincipalAllocated
115	0.6606	0.6593	0.0012488	0.0000
116	0.6593	0.6583	0.0010424	0.0000
117	0.6583	0.6575	0.0008353	0.0000
118	0.6575	0.6568	0.0006275	0.0000
119	0.6568	0.6564	0.0004190	0.0000
120	0.6564	0.0000	0.0002099	0.6562

Table 71: Table continues below

	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates	EquityResetRates
115	0	0.1321	NA	NA	NA
116	0	0.1646	NA	NA	NA
117	0	0.2188	0.001737	November 15 2028	0.005306
118	0	0.3273	NA	NA	NA
119	0	0.6531	NA	NA	NA
120	0	0.0000	0.001724	February 15 2029	0.005616

Table 72: Table continues below

	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
115	NA	NA	NA	7.231
116	NA	NA	NA	5.803
117	0.2556	1.006	1.006	4.365
118	NA	NA	NA	2.918
119	NA	NA	NA	1.462

	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
120	0.2472	1.005	1.005	0.000

Table 73: Table continues below

	EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin
115	NA	NA	NA
116	NA	NA	NA
117	0.0000	0.001748	0.001748
118	NA	NA	NA
119	NA	NA	NA
120	0.6562	0.661254	0.661254

Table 74: Table continues below

	EquityPaymentDate	EquityAverageBalance	EquityCouponRate	EquityPayType
115	NA	NA	NA	sequential
116	NA	NA	NA	sequential
117	February 15 2029	0.6594	0.01031	sequential
118	NA	NA	NA	sequential
119	NA	NA	NA	sequential
120	May 15 2029	0.6569	0.01062	sequential

Table 75: Table continues below

	EquityNULL	MezzStart	MezzEnd	MezzLossAllocated	MezzPrincipalAllocated
115	NA	5.3199	4.3326	0	0.9872
116	NA	4.3326	3.3419	0	0.9907
117	NA	3.3419	2.3477	0	0.9942
118	NA	2.3477	1.3499	0	0.9978
119	NA	1.3499	0.3486	0	1.0013
120	NA	0.3486	0.0000	0	0.3486

Table 76: Table continues below

	MezzAttach	MezzDettach	MezzCoupon	MezzResetDates	MezzResetRates
115	0.1321	1	NA	NA	NA
116	0.1646	1	NA	NA	NA
117	0.2188	1	0.09443	November 15 2028	0.005306
118	0.3273	1	NA	NA	NA
119	0.6531	1	NA	NA	NA
120	0.0000	0	0.02855	February 15 2029	0.005616

Table 77: Table continues below

	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount	MezzRW
115	NA	NA	NA	0.07
116	NA	NA	NA	0.07
117	0.2556	1.006	1.006	0.07
118	NA	NA	NA	0.07
119	NA	NA	NA	0.07
120	0.2472	1.005	1.005	0.00

Table 78: Table continues below

	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDiscounMargin	MezzPaymentDate
115	NA	NA	NA	NA
116	NA	NA	NA	NA
117	2.972	3.086	3.086	February 15 2029
118	NA	NA	NA	NA
119	NA	NA	NA	NA
120	2.348	2.388	2.388	May 15 2029

Table 79: Table continues below

	MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart
115	NA	NA	sequential	NA	0
116	NA	NA	sequential	NA	0
117	4.331	0.08531	sequential	NA	0
118	NA	NA	sequential	NA	0
119	NA	NA	sequential	NA	0
120	1.349	0.08562	sequential	NA	0

Table 80: Table continues below

	SeniorEnd	SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach
115	0	0	0	1
116	0	0	0	1
117	0	0	0	1
118	0	0	0	1
119	0	0	0	1
120	0	0	0	0

Table 81: Table continues below

	SeniorDettach	SeniorCoupon	SeniorResetDates	SeniorResetRates
115	1	NA	NA	NA
116	1	NA	NA	NA
117	1	0	November 15 2028	0.005306
118	1	NA	NA	NA
119	1	NA	NA	NA

	SeniorDettach	SeniorCoupon	SeniorResetDates	SeniorResetRates
120	0	0	February 15 2029	0.005616

Table 82: Table continues below

	SeniorCouponDCF	SeniorDiscount	SeniorRiskLessDiscount	SeniorRW
115	NA	NA	NA	0
116	NA	NA	NA	0
117	0.2556	1.006	1.006	0
118	NA	NA	NA	0
119	NA	NA	NA	0
120	0.2472	1.005	1.005	0

Table 83: Table continues below

	SeniorPrincipalPayment	SeniorPVRiskLess	SeniorPVDiscountMargin
115	NA	NA	NA
116	NA	NA	NA
117	0	0	0
118	NA	NA	NA
119	NA	NA	NA
120	0	0	0

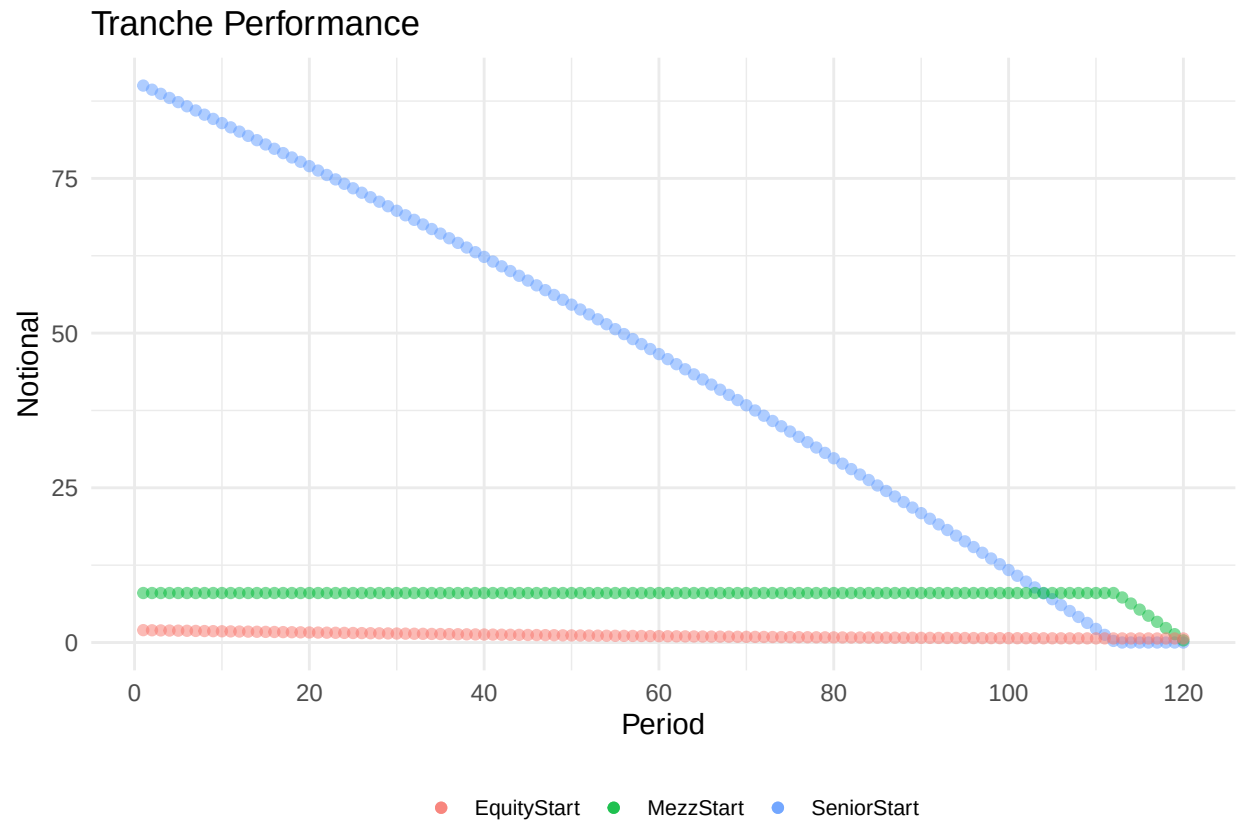
Table 84: Table continues below

	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate	SeniorPayType
115	NA	NA	NA	sequential
116	NA	NA	NA	sequential
117	February 15 2029	0	0.1553	sequential
118	NA	NA	NA	sequential
119	NA	NA	NA	sequential
120	May 15 2029	0	0.1556	sequential

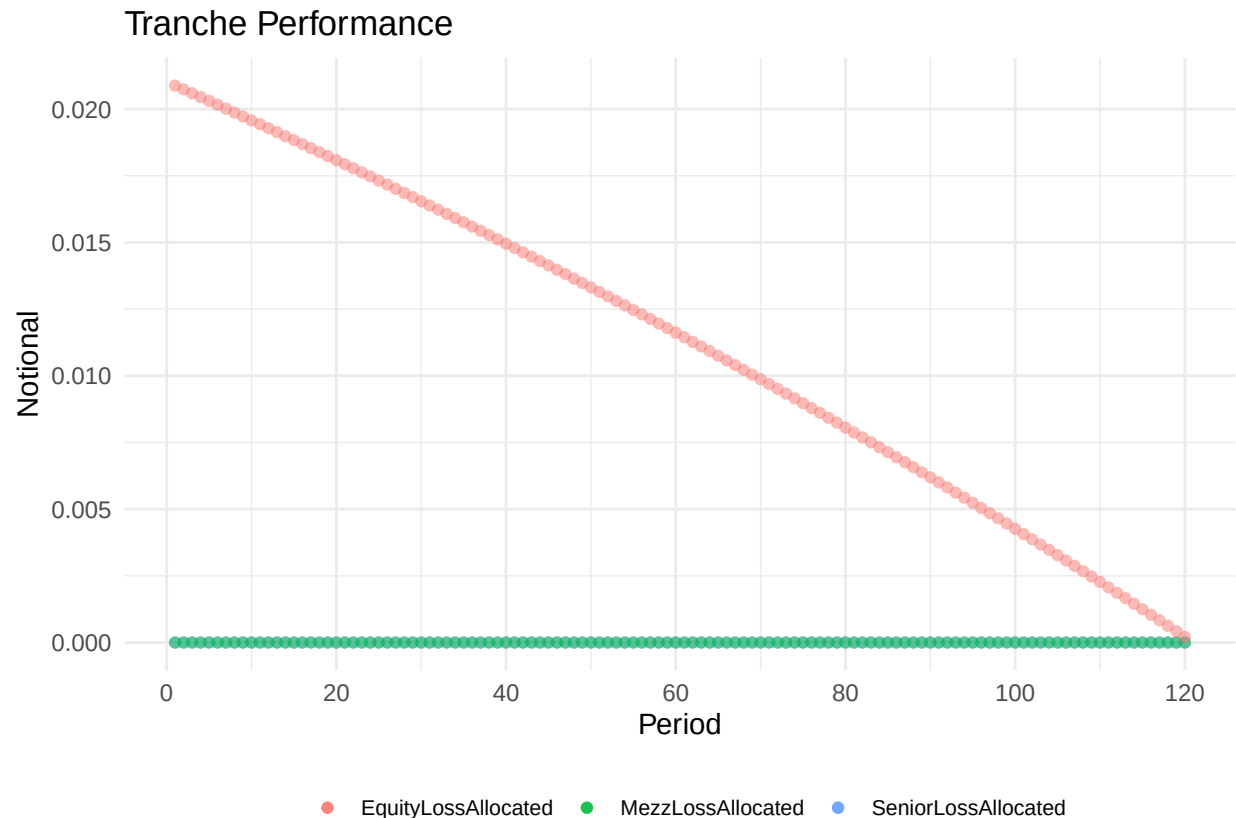
	SeniorNULL
115	NA
116	NA
117	NA
118	NA
119	NA
120	NA

3.9.4 Visualisations

Visualizing the tranche balances.



We can see in the sequential structure that the senior is repaid first then the Mezzanine and lastly the equity. Let us now consider the projected loss



The loss erodes the equity but is not sufficient to breach into the mezzanine.

####Stress scenarios

Let us now define a stress scenario to the portfolio with loss level at 300% of base level

```
demo_engine_stress <- demo_cashflow_engine(demo_equity = 2, demo_mezz = 8,
  demo_snr = 90, stress_cdr = 3, stress_cpr = 1)
```

```
## INFO [2019-22-10 14:21:41] Portfolio seeded
## INFO [2019-22-10 14:21:41] hello, I am your R6 portfolio manager, my name is test_ptf
## [1] "got here"
## INFO [2019-22-10 14:21:41] Hello, I am your R6 tranche , my name is equity
## INFO [2019-22-10 14:21:41] ... and my size is 2, my pay type is sequential
## INFO [2019-22-10 14:21:41]
## INFO [2019-22-10 14:21:41] Hello, I am your R6 tranche , my name is mezz
## INFO [2019-22-10 14:21:41] ... and my size is 8, my pay type is sequential
## INFO [2019-22-10 14:21:41]
## INFO [2019-22-10 14:21:41] Hello, I am your R6 tranche , my name is senior
## INFO [2019-22-10 14:21:41] ... and my size is 90, my pay type is sequential
## INFO [2019-22-10 14:21:41]
## INFO [2019-22-10 14:21:41] hello, I am your R6 trigger, my name is cum_loss
## Hello, from the cashflow engine, my name is myengine.
## INFO [2019-22-10 14:21:41] MLIB Successfully loaded and ready to GO!
## INFO [2019-22-10 14:21:44] Curve EUR_OIS_CURVE@24 loaded from 20190101!
## INFO [2019-22-10 14:21:44] Curve EUR_OIS_CURVE@30 loaded from 20190101!
## INFO [2019-22-10 14:21:44] Curve EUR_OIS_CURVE_MARKETDATA@23 loaded from 20190101!
## INFO [2019-22-10 14:21:44] Curve EUR_SWAP_3M_CURVE@18 loaded from 20190101!
```

```
## INFO [2019-22-10 14:21:44] Curve EUR_SWAP_3M_CURVE@22 loaded from 20190101!
## INFO [2019-22-10 14:21:44] Curve EUR_SWAP_3M_CURVE_MARKETDATA@21 loaded from 20190101!
## INFO [2019-22-10 14:21:44] mlib_adapter configured!
## INFO [2019-22-10 14:21:44] Configured
## INFO [2019-22-10 14:21:44] Tranche equity's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:44] Tranche mezz's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:44] Tranche senior's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:44] Portfolio initilalised reinvestmet and collections buffer
## INFO [2019-22-10 14:21:44] Getting df curve and forward rates
## INFO [2019-22-10 14:21:44] Using MLIB version MLIB
```

```
pander_split(head(demo_engine_stress$liability_flowsdf))
```

Table 86: Table continues below

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
1	100.00	99.23	0.6432	0
2	99.23	98.46	0.6450	0
3	98.46	97.69	0.6469	0
4	97.69	96.92	0.6488	0
5	96.92	96.15	0.6507	0
6	96.15	95.37	0.6526	0

Table 87: Table continues below

SeverityRecovered	Prepayment	LossGivenLoss	InterestComponent	CumulativeDefaults
0.06293	0	0.06293	0.4167	0.1259
0.06245	0	0.06245	0.4135	0.2508
0.06197	0	0.06197	0.4103	0.3747
0.06148	0	0.06148	0.4070	0.4977
0.06099	0	0.06099	0.4038	0.6196
0.06051	0	0.06051	0.4006	0.7407

Table 88: Table continues below

CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated
0.06293	0.06293	2.000	1.937	0.06293
0.12538	0.12538	1.937	1.875	0.06245
0.18735	0.18735	1.875	1.813	0.06197
0.24883	0.24883	1.813	1.751	0.06148
0.30982	0.30982	1.751	1.690	0.06099
0.37033	0.37033	1.690	1.630	0.06051

Table 89: Table continues below

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.01952	NA	NA
0	0	0.01904	NA	NA
0	0	0.01856	0.002475	May 15 2019
0	0	0.01807	NA	NA

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.01758	NA	NA
0	0	0.01709	0.002238	August 15 2019

Table 90: Table continues below

EquityResetRates	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	0.9991	0	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	1.0003	1	12.5

Table 91: Table continues below

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
NA	NA	NA	NA
0	0.000000	0.002473	August 15 2019
NA	NA	NA	NA
NA	NA	NA	NA
0	0.002238	0.002238	November 15 2019

Table 92: Table continues below

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.937	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.751	0.005	sequential	NA	8	8

Table 93: Table continues below

MezzLossAllocated	MezzPrincipalAllocated	MezzAttach	MezzDettach	MezzCoupon
0	0	0.01952	0.1001	NA
0	0	0.01904	0.1003	NA
0	0	0.01856	0.1004	0.1636
0	0	0.01807	0.1006	NA
0	0	0.01758	0.1008	NA
0	0	0.01709	0.1010	0.1636

Table 94: Table continues below

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991	0
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
August 15 2019	0	0.2556	1.0003	1

Table 95: Table continues below

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDDiscountMargin	MezzPaymentDate
8.813	NA	NA	NA	NA
8.819	NA	NA	NA	NA
8.824	0	0.0000	0.1634	August 15 2019
8.828	NA	NA	NA	NA
8.831	NA	NA	NA	NA
8.834	0	0.1636	0.1636	November 15 2019

Table 96: Table continues below

MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart	SeniorEnd
NA	NA	sequential	NA	90.00	89.29
NA	NA	sequential	NA	89.29	88.59
8	0.08	sequential	NA	88.59	87.88
NA	NA	sequential	NA	87.88	87.17
NA	NA	sequential	NA	87.17	86.46
8	0.08	sequential	NA	86.46	85.74

Table 97: Table continues below

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.7061	0.1001	1	NA
0	0.7075	0.1003	1	NA
0	0.7089	0.1004	1	3.423
0	0.7103	0.1006	1	NA
0	0.7117	0.1008	1	NA
0	0.7131	0.1010	1	3.341

Table 98: Table continues below

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
NA	NA	NA	NA
NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991
NA	NA	NA	NA
NA	NA	NA	NA

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
August 15 2019	0	0.2556	1.0003

Table 99: Table continues below

SeniorRiskLessDiscount	SeniorRW	SeniorPrincipalPayment	SeniorPVRiskLess
NA	0.07	NA	NA
NA	0.07	NA	NA
0	0.07	2.122	0.000
NA	0.07	NA	NA
NA	0.07	NA	NA
1	0.07	2.135	5.478

Table 100: Table continues below

SeniorPVDiscuntMargin	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate
NA	NA	NA	NA
NA	NA	NA	NA
5.540	August 15 2019	89.29	0.15
NA	NA	NA	NA
NA	NA	NA	NA
5.478	November 15 2019	87.17	0.15

SeniorPayType	SeniorNULL
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA

The flows toward maturity can be inspected as below;

```
pander_split(tail(demo_engine_stress$liability_flowsdf))
```

Table 102: Table continues below

	Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent
115	115	5.4333	4.53139086891717024	0.8950
116	116	4.5314	3.62806129842672176	0.8976
117	117	3.6281	2.72326314854125684	0.9002
118	118	2.7233	1.81699070439444643	0.9028
119	119	1.8170	0.90923823635870304	0.9055
120	120	0.9092	-0.00000000000000322	0.9081

Table 103: Table continues below

	ReinvestmentBalance	SeverityRecovered	Prepayment	LossGivenLoss
115	0	0.0034194	0	0.0034194
116	0	0.0028518	0	0.0028518
117	0	0.0022833	0	0.0022833
118	0	0.0017139	0	0.0017139
119	0	0.0011435	0	0.0011435
120	0	0.0005722	0	0.0005722

Table 104: Table continues below

	InterestComponent	CumulativeDefaults	CumulativeLoss	CumulativeRecovery
115	0.022639	7.811	3.905	3.905
116	0.018881	7.817	3.908	3.908
117	0.015117	7.821	3.911	3.911
118	0.011347	7.825	3.912	3.912
119	0.007571	7.827	3.913	3.913
120	0.003788	7.828	3.914	3.914

Table 105: Table continues below

	EquityStart	EquityEnd	EquityLossAllocated	EquityPrincipalAllocated
115	0	0	0	0
116	0	0	0	0
117	0	0	0	0
118	0	0	0	0
119	0	0	0	0
120	0	0	0	0

Table 106: Table continues below

	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates	EquityResetRates
115	0	0	NA	NA	NA
116	0	0	NA	NA	NA
117	0	0	0	November 15 2028	0.005306
118	0	0	NA	NA	NA
119	0	0	NA	NA	NA
120	0	0	0	February 15 2029	0.005616

Table 107: Table continues below

	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
115	NA	NA	NA	0
116	NA	NA	NA	0
117	0.2556	1.006	1.006	0
118	NA	NA	NA	0
119	NA	NA	NA	0

	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
120	0.2472	1.005	1.005	0

Table 108: Table continues below

	EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin
115	NA	NA	NA
116	NA	NA	NA
117	0	0	0
118	NA	NA	NA
119	NA	NA	NA
120	0	0	0

Table 109: Table continues below

	EquityPaymentDate	EquityAverageBalance	EquityCouponRate	EquityPayType
115	NA	NA	NA	sequential
116	NA	NA	NA	sequential
117	February 15 2029	0	0.01031	sequential
118	NA	NA	NA	sequential
119	NA	NA	NA	sequential
120	May 15 2029	0	0.01062	sequential

Table 110: Table continues below

	EquityNULL	MezzStart	MezzEnd	MezzLossAllocated	MezzPrincipalAllocated
115	NA	5.4333	4.5314	0.0034194	0.8984
116	NA	4.5314	3.6281	0.0028518	0.9005
117	NA	3.6281	2.7233	0.0022833	0.9025
118	NA	2.7233	1.8170	0.0017139	0.9046
119	NA	1.8170	0.9092	0.0011435	0.9066
120	NA	0.9092	0.0000	0.0005722	0.9087

Table 111: Table continues below

	MezzAttach	MezzDettach	MezzCoupon	MezzResetDates	MezzResetRates
115	0	1	NA	NA	NA
116	0	1	NA	NA	NA
117	0	1	0.09878	November 15 2028	0.005306
118	0	1	NA	NA	NA
119	0	1	NA	NA	NA
120	0	0	0.03845	February 15 2029	0.005616

Table 112: Table continues below

	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount	MezzRW
115	NA	NA	NA	0.955
116	NA	NA	NA	0.955
117	0.2556	1.006	1.006	0.955
118	NA	NA	NA	0.955
119	NA	NA	NA	0.955
120	0.2472	1.005	1.005	0.000

Table 113: Table continues below

	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDiscounMargin	MezzPaymentDate
115	NA	NA	NA	NA
116	NA	NA	NA	NA
117	2.701	2.818	2.818	February 15 2029
118	NA	NA	NA	NA
119	NA	NA	NA	NA
120	2.720	2.772	2.772	May 15 2029

Table 114: Table continues below

	MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart
115	NA	NA	sequential	NA	0
116	NA	NA	sequential	NA	0
117	4.531	0.08531	sequential	NA	0
118	NA	NA	sequential	NA	0
119	NA	NA	sequential	NA	0
120	1.816	0.08562	sequential	NA	0

Table 115: Table continues below

	SeniorEnd	SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach
115	0	0	0	1
116	0	0	0	1
117	0	0	0	1
118	0	0	0	1
119	0	0	0	1
120	0	0	0	0

Table 116: Table continues below

	SeniorDettach	SeniorCoupon	SeniorResetDates	SeniorResetRates
115	1	NA	NA	NA
116	1	NA	NA	NA
117	1	0	November 15 2028	0.005306
118	1	NA	NA	NA
119	1	NA	NA	NA

	SeniorDettach	SeniorCoupon	SeniorResetDates	SeniorResetRates
120	0	0	February 15 2029	0.005616

Table 117: Table continues below

	SeniorCouponDCF	SeniorDiscount	SeniorRiskLessDiscount	SeniorRW
115	NA	NA	NA	0
116	NA	NA	NA	0
117	0.2556	1.006	1.006	0
118	NA	NA	NA	0
119	NA	NA	NA	0
120	0.2472	1.005	1.005	0

Table 118: Table continues below

	SeniorPrincipalPayment	SeniorPVRiskLess	SeniorPVDiscountMargin
115	NA	NA	NA
116	NA	NA	NA
117	0	0	0
118	NA	NA	NA
119	NA	NA	NA
120	0	0	0

Table 119: Table continues below

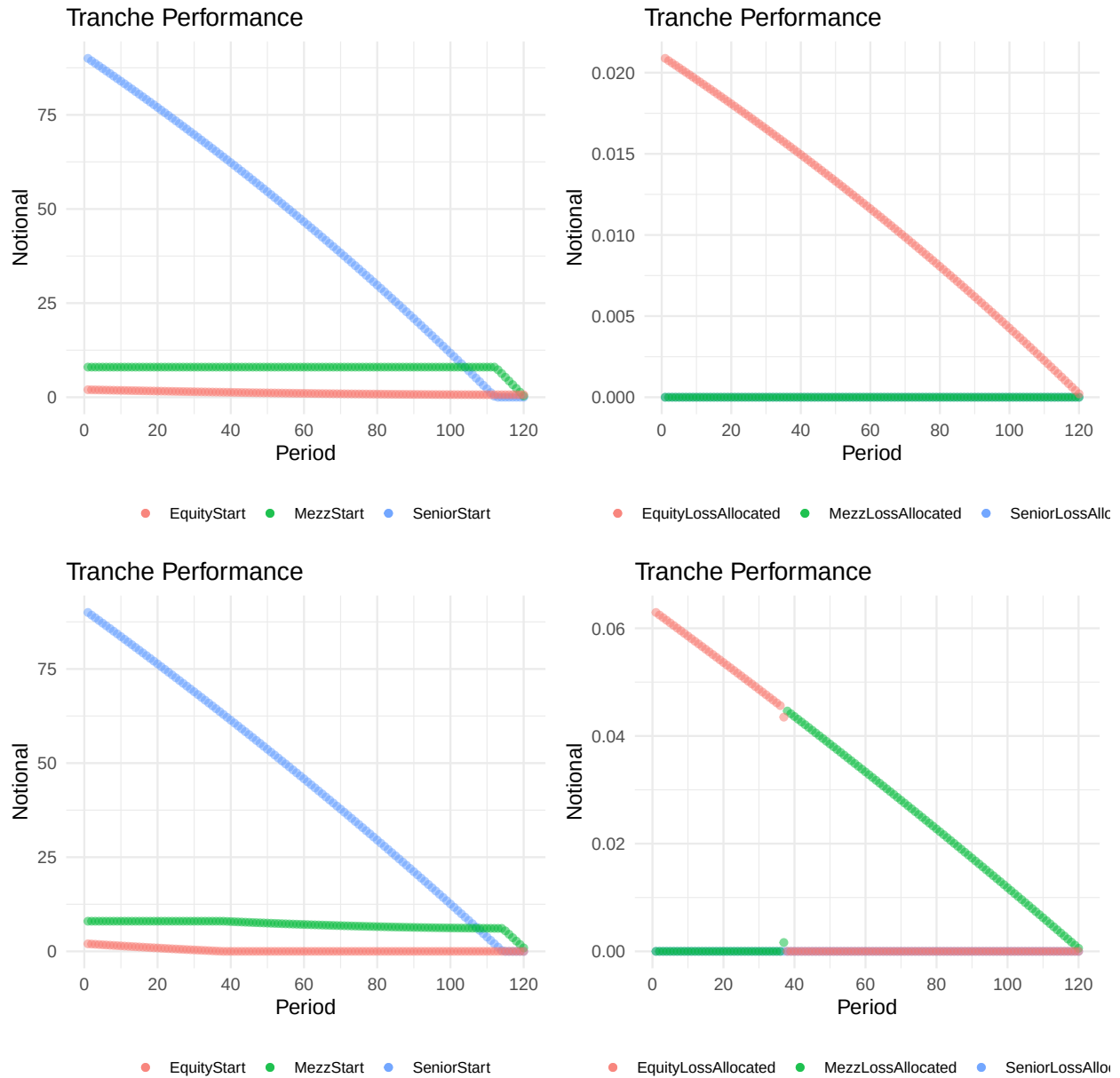
	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate	SeniorPayType
115	NA	NA	NA	sequential
116	NA	NA	NA	sequential
117	February 15 2029	0	0.1553	sequential
118	NA	NA	NA	sequential
119	NA	NA	NA	sequential
120	May 15 2029	0	0.1556	sequential

	SeniorNULL
115	NA
116	NA
117	NA
118	NA
119	NA
120	NA

3.9.4.1 Base Case vs Stress

Let us now visualize the impact of the stress

```
draw_tranche_grid(demo_engine$liability_flowsdf, demo_engine_stress$liability_flowsdf)
```



3.9.5 Reinvestments

Let us now consider a transaction which includes reinvestment of prepayments, recovered balances on default and scheduled principal. We can define the reinvestment period to run to any period, in this example we have set a 1Y reinvestment, that is the 12th period. For simplicity the asset selected for reinvestment has the same parameterisation as the seed portfolio loan, however it can have any form or multiple loans with a pro rata based reinvestment allocation that can be configured as required.

```
demo_cashflow_engine_3 <- function(demo_equity = 2, demo_mezz = 8,
  demo_snr = 90, stress_cdr = 1, stress_cpr = 1) {
  # we initialise some tranches here - we can extend this to
```

```

# create t ranche classess who have the data here as member
# data and that have as member data their own cash registers

# project a loan - for simpllicity assume a single rep line.
loan_amortization_df_small <- loan_amortization_default_prepayment(Term = 120,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.005 *
    stress_cdr, 120), SeverityVector = rep(0.5, 120),
  PrepaymentVector = rep(0 * stress_cpr, 120), RiskWeight = rep(0.6,
    120), KIRB = rep(0.08, 120), LoanType = "A")

myptf <- r_portfolio$new("test_ptf")

# create an initial portfolio
myptf$seed_portfolio(loan_amortization_df_small)

# create a set of tranches to be included in the transaction
# name, size, style, periods, fix/flt, coupon, coupon_floor
equity_tranche <- r_tranche$new(name = "equity", size = demo_equity,
  pay_type = "sequential", coupon_type = "flt", coupon_rate = 50,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

mezz_tranche <- r_tranche$new(name = "mezz", size = demo_mezz,
  pay_type = "sequential", coupon_type = "fix", coupon_rate = 800,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

snr_tranche <- r_tranche$new(name = "senior", size = demo_snr,
  pay_type = "sequential", coupon_type = "flt", coupon_rate = 1500,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

# bind the tranches into a capital structure
tranche_list <- list(equity_tranche, mezz_tranche, snr_tranche)

# triggers needed
deal_trigger <- r_triggers$new(name = "cum_loss", periods = 120)
deal_trigger$set_threshold(threshold = 1.3, active_period = 24)
triggers_list <- list(deal_trigger)

# reinvest into the same loan
repline_1 <- r_repline$new(name = "repline1", proportion = 1,
  term = 120, balance = 1, rate = 0.05, default_vector = rep(0,
    120), prepayment_vector = rep(0, 120), severity_vector = rep(0.5,
    120), risk_weight = rep(0.6, 120), kirb = rep(0.08,
    120), loan_type = "A", loan_freq = 3)

repline_list = list(repline_1)

# seed the reinvestment replines with the repline list
myptf$seed_reinvestment_replines(repline_list)

```

```

# instantiate a cashflow engine
my_engine <- r_cashflow_engine$new("myengine")

# mlib_adapter
mlib <- mgen::mlib_adapter$new()

# configure the cashflow engine TODO kirb, elgd, mt, n should
# come from portfolio - the loan object
my_engine$configure(portfolio = myptf, tranches = tranche_list,
  kirb = 0.07, elgd = 0.4, mt = 0.3, n = 500, supervisory_type = "old",
  pool_type = "Retail", pricing_lib = mlib, effective_date = as.Date("2019-05-15"),
  maturity_date = as.Date("2029-05-15"))

# set reinvestments to the 10th period
my_engine$reinvest_end_period = 12

# run the cashflow model
my_engine$evaluate()

my_engine
}

```

Calculate the cash-flows with reinvestment

```

demo_engine_reinvest <- demo_cashflow_engine_3(demo_equity = 2,
  demo_mezz = 8, demo_snr = 90, stress_cdr = 0, stress_cpr = 1)

```

```

## INFO [2019-22-10 14:21:48] Portfolio seeded
## INFO [2019-22-10 14:21:48] hello, I am your R6 portfolio manager, my name is test_ptf
## INFO [2019-22-10 14:21:48] Hello, I am your R6 tranche , my name is equity
## INFO [2019-22-10 14:21:48] ... and my size is 2, my pay type is sequential
## INFO [2019-22-10 14:21:48]
## INFO [2019-22-10 14:21:48] Hello, I am your R6 tranche , my name is mezz
## INFO [2019-22-10 14:21:48] ... and my size is 8, my pay type is sequential
## INFO [2019-22-10 14:21:48]
## INFO [2019-22-10 14:21:48] Hello, I am your R6 tranche , my name is senior
## INFO [2019-22-10 14:21:48] ... and my size is 90, my pay type is sequential
## INFO [2019-22-10 14:21:48]
## INFO [2019-22-10 14:21:48] hello, I am your R6 trigger, my name is cum_loss
## INFO [2019-22-10 14:21:48] hello, from repline object repline1
## Hello, from the cashflow engine, my name is myengine.
## INFO [2019-22-10 14:21:48] MLIB Successfully loaded and ready to GO!
## INFO [2019-22-10 14:21:51] Curve EUR_OIS_CURVE@24 loaded from 20190101!
## INFO [2019-22-10 14:21:51] Curve EUR_OIS_CURVE@30 loaded from 20190101!
## INFO [2019-22-10 14:21:51] Curve EUR_OIS_CURVE_MARKETDATA@23 loaded from 20190101!
## INFO [2019-22-10 14:21:51] Curve EUR_SWAP_3M_CURVE@18 loaded from 20190101!
## INFO [2019-22-10 14:21:51] Curve EUR_SWAP_3M_CURVE@22 loaded from 20190101!
## INFO [2019-22-10 14:21:51] Curve EUR_SWAP_3M_CURVE_MARKETDATA@21 loaded from 20190101!
## INFO [2019-22-10 14:21:51] mlib_adapter configured!
## INFO [2019-22-10 14:21:51] Configured
## INFO [2019-22-10 14:21:51] Tranche equity's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:51] Tranche mezz's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:51] Tranche senior's dataframe has been initialized with 120 periods

```

```

## INFO [2019-22-10 14:21:51] Portfolio initilalised reinvestmet and collections buffer
## INFO [2019-22-10 14:21:51] Getting df curve and forward rates
## INFO [2019-22-10 14:21:51] Using MLIB version MLIB
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 1 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:51] reinvesting repline repline1 in period 1
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 2 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:51] reinvesting repline repline1 in period 2
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 3 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:51] reinvesting repline repline1 in period 3
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 4 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:51] reinvesting repline repline1 in period 4
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 5 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:51] reinvesting repline repline1 in period 5
## INFO [2019-22-10 14:21:51] Reinvestment attempted in period 6 subject to triggers
## INFO [2019-22-10 14:21:51] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 6
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 7 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 7
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 8 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 8
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 9 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 9
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 10 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 10
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 11 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 11
## INFO [2019-22-10 14:21:52] Reinvestment attempted in period 12 subject to triggers
## INFO [2019-22-10 14:21:52] No triggers defined so reinvesting
## INFO [2019-22-10 14:21:52] reinvesting repline repline1 in period 12

```

Note from the above logs that re-investments are indicated in the log, let us now review the cash-flows

```
pander_split(head(demo_engine_reinvest$liability_flowsdf, 14))
```

Table 121: Table continues below

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
1	100.00	100.00	0.6440	0.6440
2	100.00	100.00	0.6508	0.6508
3	100.00	100.00	0.6577	0.6577
4	100.00	100.00	0.6647	0.6647
5	100.00	100.00	0.6717	0.6717
6	100.00	100.00	0.6789	0.6789
7	100.00	100.00	0.6861	0.6861
8	100.00	100.00	0.6934	0.6934

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
9	100.00	100.00	0.7007	0.7007
10	100.00	100.00	0.7081	0.7081
11	100.00	100.00	0.7156	0.7156
12	100.00	100.00	0.7232	0.7232
13	100.00	99.27	0.7309	0.0000
14	99.27	98.54	0.7340	0.0000

Table 122: Table continues below

SeverityRecovered	Prepayment	LossGivenLoss	InterestComponent	CumulativeDefaults
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4167	0
0	0	0	0.4136	0

Table 123: Table continues below

CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0
0	0	2	2	0

Table 124: Table continues below

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.02000	NA	NA
0	0	0.02000	NA	NA

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.02000	0.002556	May 15 2019
0	0	0.02000	NA	NA
0	0	0.02000	NA	NA
0	0	0.02000	0.002556	August 15 2019
0	0	0.02000	NA	NA
0	0	0.02000	NA	NA
0	0	0.02000	0.002611	November 15 2019
0	0	0.02000	NA	NA
0	0	0.02000	NA	NA
0	0	0.02000	0.002444	February 17 2020
0	0	0.02015	NA	NA
0	0	0.02030	NA	NA

Table 125: Table continues below

EquityResetRates	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	0.9991	0.000	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	1.0003	1.000	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2611	1.0014	1.001	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2444	1.0024	1.002	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5

Table 126: Table continues below

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
NA	NA	NA	NA
0	0.000000	0.002553	August 15 2019
NA	NA	NA	NA
NA	NA	NA	NA
0	0.002556	0.002556	November 15 2019
NA	NA	NA	NA
NA	NA	NA	NA
0	0.002615	0.002615	February 17 2020
NA	NA	NA	NA
NA	NA	NA	NA
0	0.002450	0.002450	May 15 2020
NA	NA	NA	NA
NA	NA	NA	NA

Table 127: Table continues below

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
2	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
2	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
2	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
2	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8

Table 128: Table continues below

MezzLossAllocated	MezzPrincipalAllocated	MezzAttach	MezzDettach	MezzCoupon
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	0.1636
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	0.1636
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	0.1671
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	NA
0	0	0.02000	0.1000	0.1564
0	0	0.02015	0.1007	NA
0	0	0.02030	0.1015	NA

Table 129: Table continues below

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991	0.000
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
August 15 2019	0	0.2556	1.0003	1.000
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
November 15 2019	0	0.2611	1.0014	1.001
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
February 17 2020	0	0.2444	1.0024	1.002

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA

Table 130: Table continues below

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDiscounMargin	MezzPaymentDate
8.806	NA	NA	NA	NA
8.806	NA	NA	NA	NA
8.806	0	0.0000	0.1634	August 15 2019
8.806	NA	NA	NA	NA
8.806	NA	NA	NA	NA
8.806	0	0.1636	0.1636	November 15 2019
8.806	NA	NA	NA	NA
8.806	NA	NA	NA	NA
8.806	0	0.1673	0.1673	February 17 2020
8.806	NA	NA	NA	NA
8.806	NA	NA	NA	NA
8.806	0	0.1568	0.1568	May 15 2020
8.720	NA	NA	NA	NA
8.633	NA	NA	NA	NA

Table 131: Table continues below

MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart	SeniorEnd
NA	NA	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
8	0.08	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
8	0.08	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
8	0.08	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	90.00
8	0.08	sequential	NA	90.00	90.00
NA	NA	sequential	NA	90.00	89.27
NA	NA	sequential	NA	89.27	88.54

Table 132: Table continues below

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	3.450
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	3.450

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	3.525
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	NA
0	0.0000	0.1000	1	3.300
0	0.7309	0.1007	1	NA
0	0.7340	0.1015	1	NA

Table 133: Table continues below

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
NA	NA	NA	NA
NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991
NA	NA	NA	NA
NA	NA	NA	NA
August 15 2019	0	0.2556	1.0003
NA	NA	NA	NA
NA	NA	NA	NA
November 15 2019	0	0.2611	1.0014
NA	NA	NA	NA
NA	NA	NA	NA
February 17 2020	0	0.2444	1.0024
NA	NA	NA	NA
NA	NA	NA	NA

Table 134: Table continues below

SeniorRiskLessDiscount	SeniorRW	SeniorPrincipalPayment	SeniorPVRiskLess
NA	0.07	NA	NA
NA	0.07	NA	NA
0.000	0.07	0	0.000
NA	0.07	NA	NA
NA	0.07	NA	NA
1.000	0.07	0	3.451
NA	0.07	NA	NA
NA	0.07	NA	NA
1.001	0.07	0	3.530
NA	0.07	NA	NA
NA	0.07	NA	NA
1.002	0.07	0	3.308
NA	0.07	NA	NA
NA	0.07	NA	NA

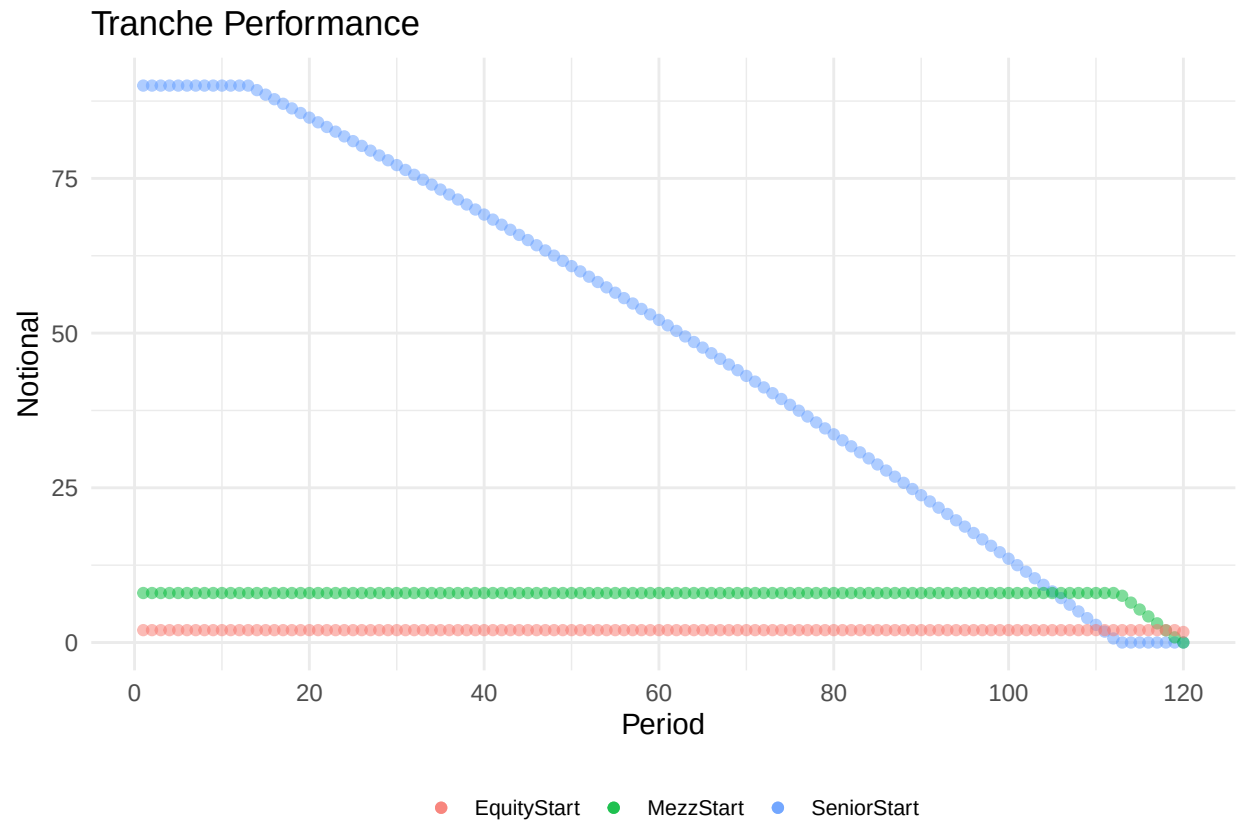
Table 135: Table continues below

SeniorPVDiscountMargin	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate
NA	NA	NA	NA
NA	NA	NA	NA
3.447	August 15 2019	90	0.15
NA	NA	NA	NA
NA	NA	NA	NA
3.451	November 15 2019	90	0.15
NA	NA	NA	NA
NA	NA	NA	NA
3.530	February 17 2020	90	0.15
NA	NA	NA	NA
NA	NA	NA	NA
3.308	May 15 2020	90	0.15
NA	NA	NA	NA
NA	NA	NA	NA

SeniorPayType	SeniorNULL
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA

We can visually inspect the re-investments by giving consideration to the tranches outstanding balances

```
draw_tranche_balances(demo_engine_reinvest$liability_flowsdf)
```



It can be observed in the plot above that the tranche Balance in a no loss scenario with replenishment shows amortization commencing only after the end of the replenishment period, consistent with the expectation.

3.9.6 Triggers

Let us now define a cumulative loss trigger that control the reinvestment eligibility, in this case if the cumulative loss exceeds 0.13% then no further reinvestment is permitted.

```
demo_cashflow_engine_4 <- function(demo_equity = 2, demo_mezz = 8,
  demo_snr = 90, stress_cdr = 1, stress_cpr = 1) {
  # we initialise some tranches here - we can extend this to
  # create t ranche classess who have the data here as member
  # data and that have as member data their own cash registers

  # project a loan - for simpllicity assume a single rep line.
  loan_amortization_df_small <- loan_amortization_default_prepayment(Term = 120,
    LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.005 *
      stress_cdr, 120), SeverityVector = rep(0.5, 120),
    PrepaymentVector = rep(0 * stress_cpr, 120), RiskWeight = rep(0.6,
      120), KIRB = rep(0.08, 120), LoanType = "A")

  myptf <- r_portfolio$new("test_ptf")

  # create an initial portfolio
  myptf$seed_portfolio(loan_amortization_df_small)
```

```

# create a set of tranches to be included in the transaction
# name, size, style, periods, fix/flt, coupon, coupon_floor
equity_tranche <- r_tranche$new(name = "equity", size = demo_equity,
  pay_type = "sequential", coupon_type = "flt", coupon_rate = 50,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

mezz_tranche <- r_tranche$new(name = "mezz", size = demo_mezz,
  pay_type = "sequential", coupon_type = "fix", coupon_rate = 800,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

snr_tranche <- r_tranche$new(name = "senior", size = demo_snr,
  pay_type = "sequential", coupon_type = "flt", coupon_rate = 1500,
  coupon_floor = TRUE, coupon_freq = 3, reset_curve = "EUR3ML",
  curve_container = "EURYC", discount_margin = 0)

# bind the tranches into a capital structure
tranche_list <- list(equity_tranche, mezz_tranche, snr_tranche)

# triggers needed
deal_trigger <- r_triggers$new(name = "cum_loss", periods = 120)

deal_trigger$set_threshold(threshold = 0.0013, active_period = 24)
triggers_list <- list(deal_trigger)

# reinvest into the same loan
repline_1 <- r_repline$new(name = "repline1", proportion = 1,
  term = 120, balance = 1, rate = 0.05, default_vector = rep(0,
  120), prepayment_vector = rep(0, 120), severity_vector = rep(0.5,
  120), risk_weight = rep(0.6, 120), kirb = rep(0.08,
  120), loan_type = "A", loan_freq = 3)

repline_list = list(repline_1)

# seed the reinvestment replines with the repline list
myptf$seed_reinvestment_replines(repline_list)

# instantiate a cashflow engine
my_engine <- r_cashflow_engine$new("myengine")

# mlib_adapter
mlib <- mgen::mlib_adapter$new()

# configure the cashflow engine TODO kirb, elgd, mt, n should
# come from portfolio - the loan object
my_engine$configure(portfolio = myptf, tranches = tranche_list,
  kirb = 0.07, elgd = 0.4, mt = 0.3, n = 500, supervisory_type = "old",
  pool_type = "Retail", pricing_lib = mlib, effective_date = as.Date("2019-05-15"),
  maturity_date = as.Date("2029-05-15"), triggers = triggers_list)

```

```

# set reinvestments to the 10th period
my_engine$reinvest_end_period = 12

# run the cashflow model
my_engine$evaluate()

my_engine
}

```

We now evaluate the cashflow engine, note in the log indication is given where the defined trigger is passed as TRUE or failed as FALSE. If the trigger is breached then no reinvestment is permitted..

```
my_engine_4 <- demo_cashflow_engine_4()
```

```

## INFO [2019-22-10 14:21:55] Portfolio seeded
## INFO [2019-22-10 14:21:55] hello, I am your R6 portfolio manager, my name is test_ptf
## INFO [2019-22-10 14:21:55] Hello, I am your R6 tranche , my name is equity
## INFO [2019-22-10 14:21:55] ... and my size is 2, my pay type is sequential
## INFO [2019-22-10 14:21:55]
## INFO [2019-22-10 14:21:55] Hello, I am your R6 tranche , my name is mezz
## INFO [2019-22-10 14:21:55] ... and my size is 8, my pay type is sequential
## INFO [2019-22-10 14:21:55]
## INFO [2019-22-10 14:21:55] Hello, I am your R6 tranche , my name is senior
## INFO [2019-22-10 14:21:55] ... and my size is 90, my pay type is sequential
## INFO [2019-22-10 14:21:55]
## INFO [2019-22-10 14:21:55] hello, I am your R6 trigger, my name is cum_loss
## INFO [2019-22-10 14:21:55] hello, from repline object repline1
## Hello, from the cashflow engine, my name is myengine.

## in method for 'copyToR' with signature '"_p_swig::NelsonSiegelSvenssonParameters"': no definition for
## in method for 'copyToR' with signature '"_p_swig::NelsonSiegelSvenssonCalibrationResults"': no defin
## in method for 'copyToR' with signature '"_p_swig::PolynomialCalibrationResults"': no definition for

## INFO [2019-22-10 14:21:56] MLIB Successfully loaded and ready to GO!
## INFO [2019-22-10 14:21:58] Curve EUR_OIS_CURVE@24 loaded from 20190101!
## INFO [2019-22-10 14:21:58] Curve EUR_OIS_CURVE@30 loaded from 20190101!
## INFO [2019-22-10 14:21:58] Curve EUR_OIS_CURVE_MARKETDATA@23 loaded from 20190101!
## INFO [2019-22-10 14:21:58] Curve EUR_SWAP_3M_CURVE@18 loaded from 20190101!
## INFO [2019-22-10 14:21:58] Curve EUR_SWAP_3M_CURVE@22 loaded from 20190101!
## INFO [2019-22-10 14:21:58] Curve EUR_SWAP_3M_CURVE_MARKETDATA@21 loaded from 20190101!
## INFO [2019-22-10 14:21:58] mlib_adapter configured!
## INFO [2019-22-10 14:21:58] Configured
## INFO [2019-22-10 14:21:58] Tranche equity's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:58] Tranche mezz's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:58] Tranche senior's dataframe has been initialized with 120 periods
## INFO [2019-22-10 14:21:58] Portfolio initilalised reinvestmet and collections buffer
## INFO [2019-22-10 14:21:58] Getting df curve and forward rates
## INFO [2019-22-10 14:21:58] Using MLIB version MLIB
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 1 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 2 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 3 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 4 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 5 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 6 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 7 subject to triggers

```

```
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 8 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 9 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 10 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 11 subject to triggers
## INFO [2019-22-10 14:21:58] Reinvestment attempted in period 12 subject to triggers
```

Inspecting the cashflows we observe that the replenishment is only performed up to the 3rd period

```
pander_split(head(my_engine_4$liability_flowsdf, 5))
```

Table 137: Table continues below

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
1	100.00	99.31	0.6437	0
2	99.31	98.63	0.6461	0
3	98.63	97.94	0.6486	0
4	97.94	97.25	0.6510	0
5	97.25	96.55	0.6534	0

Table 138: Table continues below

SeverityRecovered	Prepayment	LossGivenLoss	InterestComponent	CumulativeDefaults
0.02088	0	0.02088	0.4167	0.04176
0.02074	0	0.02074	0.4138	0.08324
0.02059	0	0.02059	0.4109	0.12443
0.02045	0	0.02045	0.4081	0.16533
0.02031	0	0.02031	0.4052	0.20594

Table 139: Table continues below

CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated
0.02088	0.02088	2.000	1.979	0.02088
0.04162	0.04162	1.979	1.958	0.02074
0.06221	0.06221	1.958	1.938	0.02059
0.08266	0.08266	1.938	1.917	0.02045
0.10297	0.10297	1.917	1.897	0.02031

Table 140: Table continues below

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.01993	NA	NA
0	0	0.01986	NA	NA
0	0	0.01979	0.002529	May 15 2019
0	0	0.01972	NA	NA
0	0	0.01965	NA	NA

Table 141: Table continues below

EquityResetRates	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	0.9991	0	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5

Table 142: Table continues below

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
NA	NA	NA	NA
0	0	0.002527	August 15 2019
NA	NA	NA	NA
NA	NA	NA	NA

Table 143: Table continues below

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.979	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8

Table 144: Table continues below

MezzLossAllocated	MezzPrincipalAllocated	MezzAttach	MezzDettach	MezzCoupon
0	0	0.01993	0.1005	NA
0	0	0.01986	0.1010	NA
0	0	0.01979	0.1015	0.1636
0	0	0.01972	0.1020	NA
0	0	0.01965	0.1025	NA

Table 145: Table continues below

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991	0
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA

Table 146: Table continues below

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDDiscountMargin	MezzPaymentDate
8.758	NA	NA	NA	NA
8.709	NA	NA	NA	NA
8.659	0	0	0.1634	August 15 2019
8.609	NA	NA	NA	NA
8.558	NA	NA	NA	NA

Table 147: Table continues below

MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart	SeniorEnd
NA	NA	sequential	NA	90.00	89.34
NA	NA	sequential	NA	89.34	88.67
8	0.08	sequential	NA	88.67	88.00
NA	NA	sequential	NA	88.00	87.33
NA	NA	sequential	NA	87.33	86.65

Table 148: Table continues below

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.6646	0.1005	1	NA
0	0.6669	0.1010	1	NA
0	0.6691	0.1015	1	3.424
0	0.6714	0.1020	1	NA
0	0.6737	0.1025	1	NA

Table 149: Table continues below

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
NA	NA	NA	NA
NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991
NA	NA	NA	NA
NA	NA	NA	NA

Table 150: Table continues below

SeniorRiskLessDiscount	SeniorRW	SeniorPrincipalPayment	SeniorPVRiskLess
NA	0.07	NA	NA
NA	0.07	NA	NA
0	0.07	2.001	0
NA	0.07	NA	NA
NA	0.07	NA	NA

Table 151: Table continues below

SeniorPVDiscountMargin	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate
NA	NA	NA	NA
NA	NA	NA	NA
5.42	August 15 2019	89.33	0.15
NA	NA	NA	NA
NA	NA	NA	NA

SeniorPayType	SeniorNULL
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA

Let us illustrate more clearly by giving consideration to PoolBalanceEnd and LossGivenLoss

```
# reshape with dplyr
pool_exp <- my_engine_4$liability_flowsdf %>% select(PoolBalanceEnd,
  LossGivenLoss, CumulativeLoss)
pool_exp$pool_gross_loss <- pool_exp$PoolBalanceEnd + pool_exp$CumulativeLoss
pander_split(head(pool_exp, 10))
```

PoolBalanceEnd	LossGivenLoss	CumulativeLoss	pool_gross_loss
99.31	0.02088	0.02088	99.34
98.63	0.02074	0.04162	98.67
97.94	0.02059	0.06221	98.00
97.25	0.02045	0.08266	97.33
96.55	0.02031	0.10297	96.65
95.86	0.02016	0.12313	95.98
95.16	0.02002	0.14315	95.30
94.46	0.01987	0.16302	94.62
93.75	0.01972	0.18274	93.94
93.05	0.01958	0.20232	93.25

We can output the analysis to excel for further investigations

```
# write.xlsx(my_engine_4$liability_flowsdf,
# file='c:/dev/mgen/files/my_engine4.xlsx', sheetName =
# 'flows')
```

3.9.7 Portfolio Upload

The model as described previously for the simulated loan can also run directly against the portfolio of loans by simply setting the seed portfolio to be equal to the loan cash-flow dataframe as produced by the curl numba script.

3.9.8 Pricing

3.9.8.1 PX to Term, DM

We can now collate the pv components across all time periods to arrive at a valuation using a discount margin

```
# cashflow ledger calc
px_dm <- 100 * sum(demo_engine$liability_flowsdf$MezzPVDiscoutMargin,
  na.rm = TRUE)/demo_engine$liability_flowsdf$MezzStart[1]

# tranche embededfunction
px_dm_t <- 100 * demo_engine$tranches[[2]]$npv_dm(call_period = 120,
  px_date = "2019-06-28")/demo_engine$liability_flowsdf$MezzStart[1]
```

The price to term is 179.2707704611 when calculated form flows or 179.2707704611 when calculated using tranche function

3.9.8.2 PX to Term, CDR

PV to term with fitted cdr and riskless discount factor

The price of par can be solved for in terms of the stress multiple of cdr

```
100 * sum(stress_px_impact$liability_flowsdf$MezzPVRiskLess,
  na.rm = TRUE)/stress_px_impact$liability_flowsdf$MezzStart[1]
```

```
## [1] 98.27640852
```

We can then visualize the flow that is projected that corresponds to this stressed level of cdr

```
pander_split(head(demo_engine_stress$liability_flowsdf))
```

Table 154: Table continues below

Period	PoolBalanceStart	PoolBalanceEnd	PrincipalComponent	ReinvestmentBalance
1	100.00	99.23	0.6432	0
2	99.23	98.46	0.6450	0
3	98.46	97.69	0.6469	0
4	97.69	96.92	0.6488	0
5	96.92	96.15	0.6507	0
6	96.15	95.37	0.6526	0

Table 155: Table continues below

SeverityRecovered	Prepayment	LossGivenLoss	InterestComponent	CumulativeDefaults
0.06293	0	0.06293	0.4167	0.1259
0.06245	0	0.06245	0.4135	0.2508
0.06197	0	0.06197	0.4103	0.3747
0.06148	0	0.06148	0.4070	0.4977
0.06099	0	0.06099	0.4038	0.6196
0.06051	0	0.06051	0.4006	0.7407

Table 156: Table continues below

CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated
0.06293	0.06293	2.000	1.937	0.06293
0.12538	0.12538	1.937	1.875	0.06245
0.18735	0.18735	1.875	1.813	0.06197
0.24883	0.24883	1.813	1.751	0.06148
0.30982	0.30982	1.751	1.690	0.06099
0.37033	0.37033	1.690	1.630	0.06051

Table 157: Table continues below

EquityPrincipalAllocated	EquityAttach	EquityDettach	EquityCoupon	EquityResetDates
0	0	0.01952	NA	NA
0	0	0.01904	NA	NA
0	0	0.01856	0.002475	May 15 2019
0	0	0.01807	NA	NA
0	0	0.01758	NA	NA
0	0	0.01709	0.002238	August 15 2019

Table 158: Table continues below

EquityResetRates	EquityCouponDCF	EquityDiscount	EquityRiskLessDiscount	EquityRW
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	0.9991	0	12.5
NA	NA	NA	NA	12.5
NA	NA	NA	NA	12.5
0	0.2556	1.0003	1	12.5

Table 159: Table continues below

EquityPrincipalPayment	EquityPVRiskLess	EquityPVDDiscountMargin	EquityPaymentDate
NA	NA	NA	NA
NA	NA	NA	NA
0	0.000000	0.002473	August 15 2019
NA	NA	NA	NA
NA	NA	NA	NA
0	0.002238	0.002238	November 15 2019

Table 160: Table continues below

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8
1.937	0.005	sequential	NA	8	8
NA	NA	sequential	NA	8	8
NA	NA	sequential	NA	8	8

EquityAverageBalance	EquityCouponRate	EquityPayType	EquityNULL	MezzStart	MezzEnd
1.751	0.005	sequential	NA	8	8

Table 161: Table continues below

MezzLossAllocated	MezzPrincipalAllocated	MezzAttach	MezzDettach	MezzCoupon
0	0	0.01952	0.1001	NA
0	0	0.01904	0.1003	NA
0	0	0.01856	0.1004	0.1636
0	0	0.01807	0.1006	NA
0	0	0.01758	0.1008	NA
0	0	0.01709	0.1010	0.1636

Table 162: Table continues below

MezzResetDates	MezzResetRates	MezzCouponDCF	MezzDiscount	MezzRiskLessDiscount
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991	0
NA	NA	NA	NA	NA
NA	NA	NA	NA	NA
August 15 2019	0	0.2556	1.0003	1

Table 163: Table continues below

MezzRW	MezzPrincipalPayment	MezzPVRiskLess	MezzPVDDiscountMargin	MezzPaymentDate
8.813	NA	NA	NA	NA
8.819	NA	NA	NA	NA
8.824	0	0.0000	0.1634	August 15 2019
8.828	NA	NA	NA	NA
8.831	NA	NA	NA	NA
8.834	0	0.1636	0.1636	November 15 2019

Table 164: Table continues below

MezzAverageBalance	MezzCouponRate	MezzPayType	MezzNULL	SeniorStart	SeniorEnd
NA	NA	sequential	NA	90.00	89.29
NA	NA	sequential	NA	89.29	88.59
8	0.08	sequential	NA	88.59	87.88
NA	NA	sequential	NA	87.88	87.17
NA	NA	sequential	NA	87.17	86.46
8	0.08	sequential	NA	86.46	85.74

Table 165: Table continues below

SeniorLossAllocated	SeniorPrincipalAllocated	SeniorAttach	SeniorDettach	SeniorCoupon
0	0.7061	0.1001	1	NA
0	0.7075	0.1003	1	NA
0	0.7089	0.1004	1	3.423
0	0.7103	0.1006	1	NA
0	0.7117	0.1008	1	NA
0	0.7131	0.1010	1	3.341

Table 166: Table continues below

SeniorResetDates	SeniorResetRates	SeniorCouponDCF	SeniorDiscount
NA	NA	NA	NA
NA	NA	NA	NA
May 15 2019	0	0.2556	0.9991
NA	NA	NA	NA
NA	NA	NA	NA
August 15 2019	0	0.2556	1.0003

Table 167: Table continues below

SeniorRiskLessDiscount	SeniorRW	SeniorPrincipalPayment	SeniorPVRiskLess
NA	0.07	NA	NA
NA	0.07	NA	NA
0	0.07	2.122	0.000
NA	0.07	NA	NA
NA	0.07	NA	NA
1	0.07	2.135	5.478

Table 168: Table continues below

SeniorPVDiscountMargin	SeniorPaymentDate	SeniorAverageBalance	SeniorCouponRate
NA	NA	NA	NA
NA	NA	NA	NA
5.540	August 15 2019	89.29	0.15
NA	NA	NA	NA
NA	NA	NA	NA
5.478	November 15 2019	87.17	0.15

SeniorPayType	SeniorNULL
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA
sequential	NA

3.9.8.3 PV to Call

To consider cash-flows only up to call date we need to truncate all flows outstanding at this point and return the tranche balance outstanding at this point in time.

```
calc_prices <- function(engine) {  
  
  # my_engine$get_pv_to_call() map calld ate to call period  
  # assume call date is say period 36 there is still dependance  
  # on mlib delivering a discount margin curve as requested by  
  # Manos  
  lf_df <- engine$liability_flowsdf  
  
  px_to_term <- 100 * sum(lf_df$MezzPVDiscntMargin, na.rm = TRUE)/lf_df$MezzStart[1]  
  
  px_to_call <- 100 * (sum(lf_df$MezzPVDiscntMargin[1:36],  
    na.rm = TRUE) + lf_df$MezzEnd[36] * lf_df$MezzDiscount[36])/lf_df$MezzStart[1]  
  
  list(px_to_term = px_to_term, px_to_call = px_to_call)  
}
```

The price to call is 124.5353035685 and the price to term 179.2707704611, the higher price of the price to term gives insight why following replenishment the issuer is axed to call.

=====

3.9.9 PV Cashflows

3.9.9.1 PV to term

We can now colalte the pv components accross all time periods to arrive at a valuation

```
# demo_engine$get_pv_to_term() demo_engine$tranches[[2]]$np
```

3.9.9.2 PV to Call

```
# demo_engine$get_pv_to_call()  
pv_dm_setlevel <- demo_engine$tranches[[2]]$npv_dm(call_period = 60,  
  px_date = "2019-06-28")  
print(pv_dm_setlevel)
```

```
## [1] 11.27934884
```

```
demo_engine$tranches[[2]]$discount_margin = 900/10000
```

```
demo_engine$set_yc()
```

```
## INFO [2019-22-10 14:22:08] Getting df curve and forward rates
```

```
## INFO [2019-22-10 14:22:08] Using MLIB version MLIB
```

```
demo_engine$evaluate()
```

```
## INFO [2019-22-10 14:22:08] Getting df curve and forward rates
```

```
## INFO [2019-22-10 14:22:08] Using MLIB version MLIB
```

```
pv_dm_override <- demo_engine$tranches[[2]]$npv_dm(call_period = 60,
  px_date = "2019-06-28")
print(pv_dm_override)
```

```
## [1] 8.019687303
```

3.9.10 IRR

We can calculate the Internal Rate of Return using the uni-root function that is conveniently wrapped into the tranche object.

```
# lets assume we buy the bond at face and it is called in
# 60th period
irr_tranche <- demo_engine$tranches[[2]]$irr(call_period = 60,
  px_date = as.Date("2019-06-28"), px_buy = demo_engine$tranches[[2]]$df$balance_start[1])
print(irr_tranche)
```

```
## [1] 0.08379972888
```

3.9.11 Portfolio Data

The model as described previously for the simulated loan can also run directly against the portfolio of loans by simply setting the seed portfolio to be equal to the loan cash-flow dataframe as produced by the curl numba script.

4 SRT Structuring

4.1 Data Tapes

The reference portfolio data delivery is dependent upon whether MHI is structuring the transaction or pricing the transaction. Where MHI is advising on the transaction origination a data type of the entire book is provided from which a subset of loans may be selected for inclusion. this process of selection is performed by simplex optimization or greedy algorithm. Where MHI is pricing a transaction the reference registry is provided as part of the investor reports, from this the portfolio can be constructed within mgen.

4.1.1 Client tape

Typically the client tape is provided in the form of a CSV file that conforms to the template provided.

Parameter	Description
Term	Term of the loan in months
LoanRate	Coupon payable on the loan
OriginalBalance	Loan balance outstanding at observation date
DefaultVector	Default rates vector, element per month, expressed as an annualised rate
SeverityVector	Recover rate vector, element per month, expressed as a %
PrepaymentVector	Prepayments rates vector, element per month, expressed as an annualised rate
RiskWeight	Regulatory risk weight parameter
KIRB	Regulatory CRR term (8% RWA + EL)
LoanType	Amortising 'A' or Bullet 'B'
LoanFreq	Loan frequency in months

4.1.2 Generate tape

In the case where MHI is structuring the transaction and the client does not have readily available means to extract the data tape from their systems they may instead choose to specify some summary statistics, Rating scale distribution, PD's, RW's etc. From these summary statistics it is possible in MGEN to synthesize a data tape from which a portfolio can be selected.

Let us now consider a simulated hypothetical data-tape

```
# Create a ratings distribution
categories_input <- list(list(list(c("RATING")), list(c("AAA",
  "AA", "A", "BBB", "BB", "B", "CCC", "D")), list(sample(1:4,
  8, replace = T))), list(list(c("CURRENCY")), list(c("USD",
  "EUR", "GBP"))), list(sample(1:3, 3, replace = T)))

# generate a hypothetical data tape
gen_df <- mgen::generate_tape_retry(categories = categories_input,
  est = 3e+05)

## WARN [2019-22-10 14:22:09] Generator tape failed in attempt 1, retrying!

names_df <- names(gen_df)

df_names <- c("LOAN_UUID", "Obligor_ID", "EAD", "RATING", "CURRENCY",
  "LOAN_ID", "MATURITY", "REG_MATURITY", "PD_REGULATORY", "LGD",
  "RW", "EL", "CAPITAL", "KIRB", "MATURITY_MONTH", "MATURITY_DATE")
pander_split(head(gen_df))
```

Table 170: Table continues below

LOAN_UUID	Obligor_ID	EAD	RATING	CURRENCY	LOAN_ID
f3288a78-f4ce-11e9-bc21-0b3f4f23fbc2	1	166.6	AAA	GBP	1
f3288a79-f4ce-11e9-bc21-0b3f4f23fbc2	1	19,137.0	AAA	GBP	2
f3288a7a-f4ce-11e9-bc21-0b3f4f23fbc2	1	12,286.4	AAA	GBP	3
f3288a7b-f4ce-11e9-bc21-0b3f4f23fbc2	2	48,974.6	D	GBP	4
f3288a7c-f4ce-11e9-bc21-0b3f4f23fbc2	2	5,236.3	D	GBP	5
f3288a7d-f4ce-11e9-bc21-0b3f4f23fbc2	3	8,909.1	D	GBP	6

Table 171: Table continues below

MATURITY	REG_MATURITY	PD_REGULATORY	LGD	RW	EL	CAPITAL
8.474	5.000	0.00003249	0.004	0.0007999	0.00000013	0.00006400
7.007	5.000	0.00003249	0.004	0.0007999	0.00000013	0.00006400
1.405	1.405	0.00003249	0.004	0.0001792	0.00000013	0.00001434
7.222	5.000	1.00000000	0.004	0.0000000	0.00400000	0.00000000
8.303	5.000	1.00000000	0.004	0.0000000	0.00400000	0.00000000
6.599	5.000	1.00000000	0.004	0.0000000	0.00400000	0.00000000

KIRB	MATURITY_MONTH	MATURITY_DATE
0.00006413	102	2028-04-01
0.00006413	84	2026-10-01
0.00001447	17	2021-03-01
0.00400000	87	2027-01-01
0.00400000	100	2028-02-01
0.00400000	79	2026-05-01

4.2 Portfolio Optimisation

Portfolio optimization can be performed by simplex algorithm for portfolios of less than 20,000 line items, for more granular portfolios a greedy algorithm is appropriate.

4.2.1 Simplex Algorithm

In the example below we find the optimal portfolio subject to specified constraints from a previously generated data tape using the simplex algorithm.

```
# load example tape
dataTape <- read.csv("../tests/testthat/test_files/vigorous_beaver.csv")

# create constraints
si <- list(list(c("RATING")), list(c("AAA", "AA", "A", "BBB",
  "BB", "B", "CCC", "D")), list(c(1, 0, 1, 1, 1, 0.3, 0.1,
  0)))
sc <- list(list(c("CURRENCY")), list(c("USD", "EUR", "GBP")),
  list(c(0.5, 0.5, 0.5)))
con <- mgen::make_simplex_constrains(dataTape, target = 10000,
  criterias = list(si, sc))
```

```
s_ptf = mgen::calculate_simplex(tape_df = dataTape, target = 1e+05,
                                criterias = list(si, sc))
pander_split(head(s_ptf))
```

Table 173: Table continues below

	X	Loan_ID	Obligor_ID	EAD	RATING	SNP_INDUSTRY_CODE	CURRENCY
515	514	515	267	5,980	CCC	Ind10	USD
516	515	516	267	10,509	CCC	Ind10	USD
836	835	836	431	13,733	BB	Ind01	USD
837	836	837	432	88,258	BB	Ind01	USD
1152	1,151	1,152	587	17,682	BB	Ind01	EUR
1155	1,154	1,155	588	37,201	BB	Ind01	EUR

Table 174: Table continues below

	LOAN_TYPE	MATURITY	REG_MATURITY	PD_REGULATORY	LGD	RW	EL
515	Lev	5.626	5	0.20495	0.4	2.352	0.081980
516	Lev	5.745	5	0.20495	0.4	2.352	0.081980
836	Lev	6.328	5	0.02109	0.4	1.318	0.008438
837	Lev	5.661	5	0.02109	0.4	1.318	0.008438
1152	ShipFin	6.085	5	0.02109	0.4	1.318	0.008438
1155	ShipFin	6.132	5	0.02109	0.4	1.318	0.008438

	CAPITAL	KIRB	ID	PEAD
515	0.1882	0.2702	515	5,980
516	0.1882	0.2702	516	4,020
836	0.1055	0.1139	836	13,733
837	0.1055	0.1139	837	26,267
1152	0.1055	0.1139	1,152	17,682
1155	0.1055	0.1139	1,155	2,318

```
summary(s_ptf)
```

```
##           X           Loan_ID      Obligor_ID          EAD
##  Min.      : 514.000   Min.      : 515.000   Min.      :267.00   Min.      : 5979.81
##  1st Qu.: 755.000   1st Qu.: 756.000   1st Qu.:390.00   1st Qu.:12927.07
##  Median : 993.500   Median : 994.500   Median :509.50   Median :19516.31
##  Mean   :1010.125   Mean   :1011.125   Mean   :515.25   Mean   :28612.88
##  3rd Qu.:1249.750   3rd Qu.:1250.750   3rd Qu.:634.75   3rd Qu.:34942.56
##  Max.    :1539.000   Max.    :1540.000   Max.    :775.00   Max.    :88258.13
##
##           RATING  SNP_INDUSTRY_CODE  CURRENCY  LOAN_TYPE  MATURITY
##  BB           :4    Ind01           :4          EUR:4    IG           :0    Min.      :5.626001
##  B            :2    Ind02           :2          GBP:0    Lev           :4    1st Qu.:5.724229
##  CCC          :2    Ind10           :2          USD:4    ShipFin:4    Median :5.931402
##  A            :0    Ind03           :0                               Mean   :6.092762
##  AA           :0    Ind04           :0                               3rd Qu.:6.181186
##  AAA          :0    Ind05           :0                               Max.    :7.386739
```

```

## (Other):0 (Other):0
## REG_MATURITY PD_REGULATORY LGD RW
## Min. :5 Min. :0.02109451 Min. :0.4 Min. :1.318184
## 1st Qu.:5 1st Qu.:0.02109451 1st Qu.:0.4 1st Qu.:1.318184
## Median :5 Median :0.04978752 Median :0.4 Median :1.572827
## Mean :5 Mean :0.08140485 Mean :0.4 Mean :1.704011
## 3rd Qu.:5 3rd Qu.:0.11009785 3rd Qu.:0.4 3rd Qu.:1.958655
## Max. :5 Max. :0.20494984 Max. :0.4 Max. :2.352208
##
## EL CAPITAL KIRB
## Min. :0.008437804 Min. :0.1054547 Min. :0.1138925
## 1st Qu.:0.008437804 1st Qu.:0.1054547 1st Qu.:0.1138925
## Median :0.019915006 Median :0.1258262 Median :0.1457412
## Mean :0.032561938 Mean :0.1363209 Mean :0.1688829
## 3rd Qu.:0.044039140 3rd Qu.:0.1566924 3rd Qu.:0.2007315
## Max. :0.081979935 Max. :0.1881767 Max. :0.2701566
##
## ID PEAD
## Min. : 515.000 Min. : 2317.520
## 1st Qu.: 756.000 1st Qu.: 5489.905
## Median : 994.500 Median :11191.520
## Mean :1011.125 Mean :12500.000
## 3rd Qu.:1250.750 3rd Qu.:18599.393
## Max. :1540.000 Max. :26266.830
##

```

4.2.2 Greedy Algorithm

The greedy algorithm works by sorting by Risk Weight contribution and then filling constraint requirements sequentially until full.

4.3 Capital Relief

SRT are primarily created for regulatory capital relief, to calculate the effectiveness of the transaction a forward projection of cash flows and resultant composition of capital structure can be calculated.

4.3.1 SEC_IRBA

The capital relief achieved is prescriptive as per the regulations

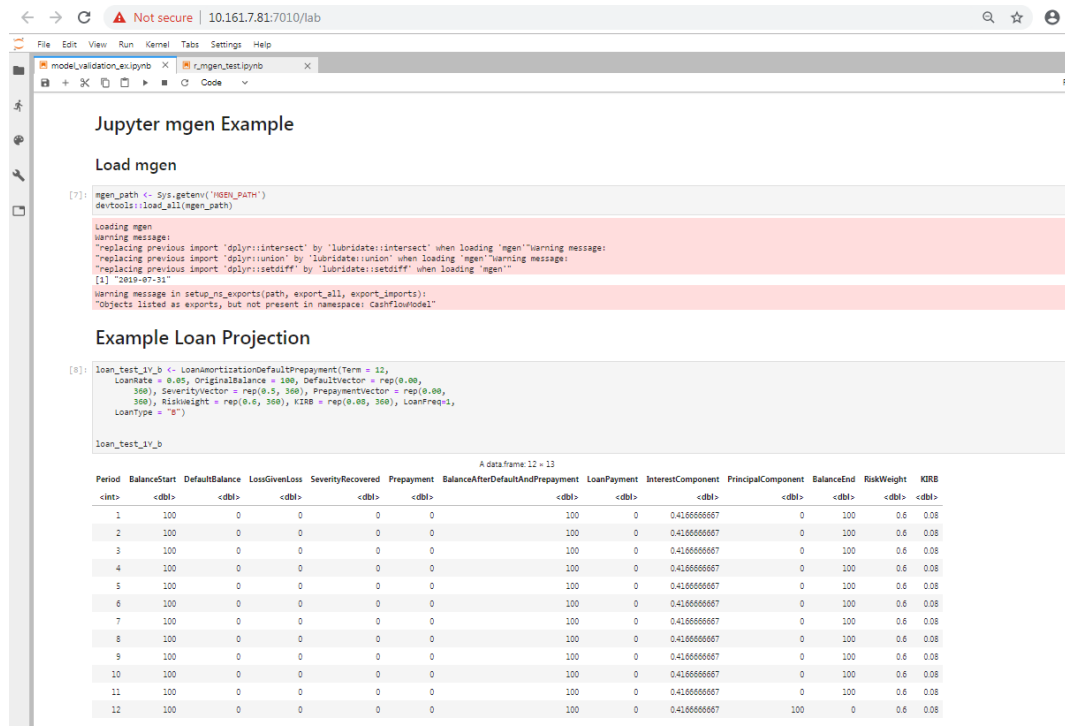
5 Jupyter Platform

The Jupyter platform offers an alternative way to access mgen, it strikes a balance between the ease of use of the SmARt portal and the low level control of RStudio.

5.1 mgen on Jupyter

5.1.1 Loans

Let us now illustrate how to load mgen on Jupyter and then make a loan cash-flow projection.



The screenshot shows a Jupyter Notebook with two code cells. The first cell, titled "Jupyter mgen Example", loads the mgen package and displays several warning messages related to namespace conflicts. The second cell, titled "Example Loan Projection", creates a loan object and displays its cash flow projection as a data frame.

```
[7]: mgen_path <- Sys.getenv("MGEN_PATH")
devtools::load_all(mgen_path)

Loading mgen
Warning message:
"replacing previous import 'dplyr::intersect' by 'lubridate::intersect' when loading 'mgen'"
Warning message:
"replacing previous import 'dplyr::union' by 'lubridate::union' when loading 'mgen'"
Warning message:
"replacing previous import 'dplyr::setdiff' by 'lubridate::setdiff' when loading 'mgen'"
[1] "2019-07-31"
Warning message in setup_ns_exports(path, export_all, export_imports):
"Objects listed as exports, but not present in namespace: CashFlowModel"
```

```
[8]: loan_test_IV_b <- LoanAmortizationDefaultPrepayment(Term = 12,
  LoanRate = 0.05, OriginalBalance = 100, DefaultVector = rep(0.00,
  360), SeverityVector = rep(0.5, 360), PrepaymentVector = rep(0.00,
  360), RiskWeight = rep(0.0, 360), K1RB = rep(0.00, 360), LoanFreq=1,
  LoanType = "R")

loan_test_IV_b
```

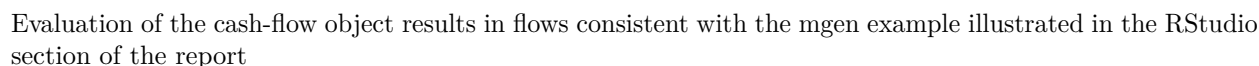
A data frame: 12 x 13

Period	BalanceStart	DefaultBalance	LossGivenLoss	SeverityRecovered	Prepayment	BalanceAfterDefaultAndPrepayment	LoanPayment	InterestComponent	PrincipalComponent	BalanceEnd	RiskWeight	K1RB
<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
2	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
3	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
4	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
5	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
6	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
7	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
8	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
9	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
10	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
11	100	0	0	0	0	100	0	0.4166666667	0	100	0.6	0.08
12	100	0	0	0	0	100	0	0.4166666667	100	0	0.6	0.08

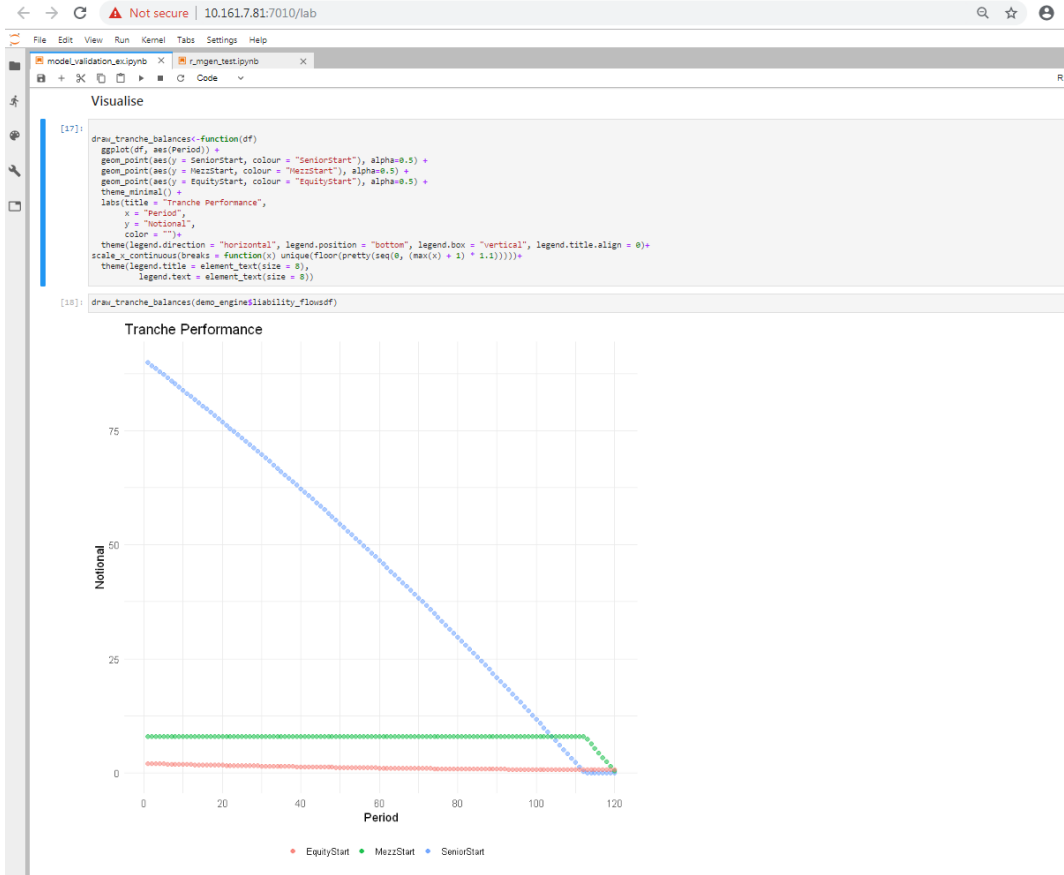
It observed that this loan projection is identical to that produced in RStudio, as expected.

5.1.2 Cashflow Engine

A cash-flow instance can be defined on Jupyter with mgen in exactly the same manner as is performed in RStudio as outlined previously.



The tranche balances can be visualized in the same manner and again are consistent with RStudio example given previously



6 SmaRT Portal

The functionality of mgen can be accessed via a higher level platform, the SmaRT portal. This platform has been specifically designed to be client friendly. The target audience spans prospective new clients considering an SRT transaction but with little or no prior knowledge through to knowledgeable users that wish to test data tapes for prospective transactions efficiency.

The workflow of the portal is similar to that outlined in the notebook previously.

1. Upload or generate data tape
2. Specify portfolio constraints
3. Select a portfolio using an optimization technique, simplex or greedy, subject to defined constraints.
4. Define a capital structure
5. Assess the structure for capital efficiency
6. Summarize transaction in a report

6.1 Upload or generate tape

Data tapes can be either uploaded or generated depending upon data availability

6.1.1 Upload

To upload a data tape, simply click on the upload tape button and select a csv data tape that conforms to the template specification provided in the Portfolio Uploads section of this report, found under SRT Modelling->Portfolios

 SmaRT
 Andrew

Tapes

Showing 1 of 1 total tapes

Sort by:
 Tape Size
Number of Loans
Created Date
Tape Name

Display:
 All
Generated
From CSV's

Upload Tape
Generate Tape

100M

lucid_turing_2688.csv

generated

From: Jun 4, 2019 | No. of loans: 2688

EL	1.520%
KIRB	7.924%
LGD	40.000%
MATURITY	5.091
N	1,813.084
PD_REGULATORY	3.801%
RW	80.046%

Details


6.1.2 Generate

When generating a portfolio it is necessary to specify both the size of the portfolio that is desired and the stratification over rating, industry, currency, loan type and PD. The loans maturity range is specified and the loan mean size and standard deviation, this allows both highly granular portfolios and smaller lumpy portfolios to be created with the same interface.

SmaRT

Andrew

Tape Generator

Categories

RATING, INDUSTRY, CURRENCY, LOAN_TYPE

RATING

AAA	AA	A	BBB	BB	B	CCC	D
57 %	93 %	95 %	41 %	18 %	75 %	76 %	78 %

INDUSTRY

Ind01	Ind02	Ind03	Ind04	Ind05	Ind06	Ind07	Ind08
42 %	56 %	13 %	25 %	19 %	16 %	25 %	84 %
Ind09	Ind10						
81 %	17 %						

CURRENCY

USD	EUR	GBP
25 %	16 %	83 %

LOAN_TYPE

IG	Lev	ShipFin
59 %	75 %	79 %

PD Input Map

1	2	3	4	5	6	7	8
0.00324 %	0.0176 %	0.198 %	0.461 %	2.109 %	7.848 %	20.494 %	100 %

Loans size standard dev.

EAD size target

Loans size mean

25,000.00	100,000,000.00	75,000.00
-----------	----------------	-----------

Maturity Min

Maturity Max

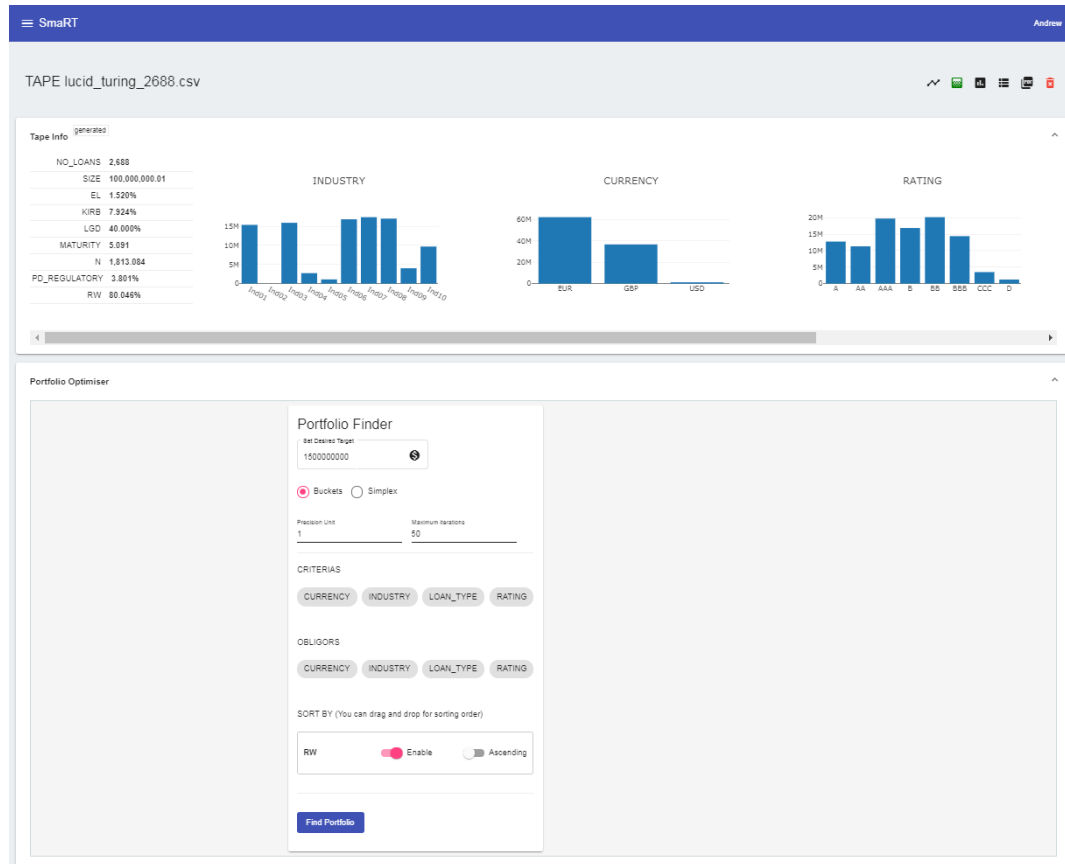
LGD %

0.01	10	40
------	----	----

Create Tape

6.1.3 Tape stratifications

Once a data tape has been either generated or uploaded summary stratification reports are produced giving a quick insight into the make up of the data tape.



6.2 Portfolio Creation

6.2.1 Specify constraints

When a transaction is being structured it is usual for the investors to specify constraints for concentration limits on a range of parameters of the portfolio. An example would be no more than 5% in ratings bucket C1, for example. Similar limitations may be requested for Industry concentration, single obligor maximum concentration etc. These constraints are specified in the portal as below.

Portfolio Optimiser

Portfolio Finder

Get Desired Target: 1500000000

☐ Buckets
☒ Simplex

CRITERIAS

CURRENCY

INDUSTRY

LOAN_TYPE

RATING

OBLIGORS

CURRENCY

INDUSTRY

LOAN_TYPE

RATING

Find Portfolio

CURRENCY

EUR 100 %

GBP 100 %

USD 100 %

Set all to 100 %

INDUSTRY

IND1 100 %

IND2 100 %

IND3 100 %

IND4 100 %

IND5 100 %

IND6 100 %

IND7 100 %

IND8 100 %

Set all to 100 %

6.2.2 Select a portfolio

A portfolio can be found either by simplex optimization or greedy algorithm by selecting the radio button. The constraints specified are applied to the portfolio.

Portfolio Optimiser

Portfolio Finder

Get Desired Target: 1500000000

☐ Buckets
☒ Simplex

CRITERIAS

CURRENCY

INDUSTRY

LOAN_TYPE

RATING

OBLIGORS

CURRENCY

INDUSTRY

LOAN_TYPE

RATING

Remove and find another! Keep found portfolio

SGM ecstatic_goodall_80 from_type simplex

From: Jul 30, 2019 |

NO_LOANS 2.650

SIZE 98,760,000.01

EL 1.037%

KIRB 7.521%

LGD 40.000%

MATURITY 3.09

N 1,787.885

PD_REGULATORY 2.593%

RW 81.051%

Details Criteria Check

CURRENCY

EUR 100 %

GBP 100 %

USD 100 %

Set all to 100 %

INDUSTRY

IND1 100 %

IND2 100 %

IND3 100 %

IND4 100 %

IND5 100 %

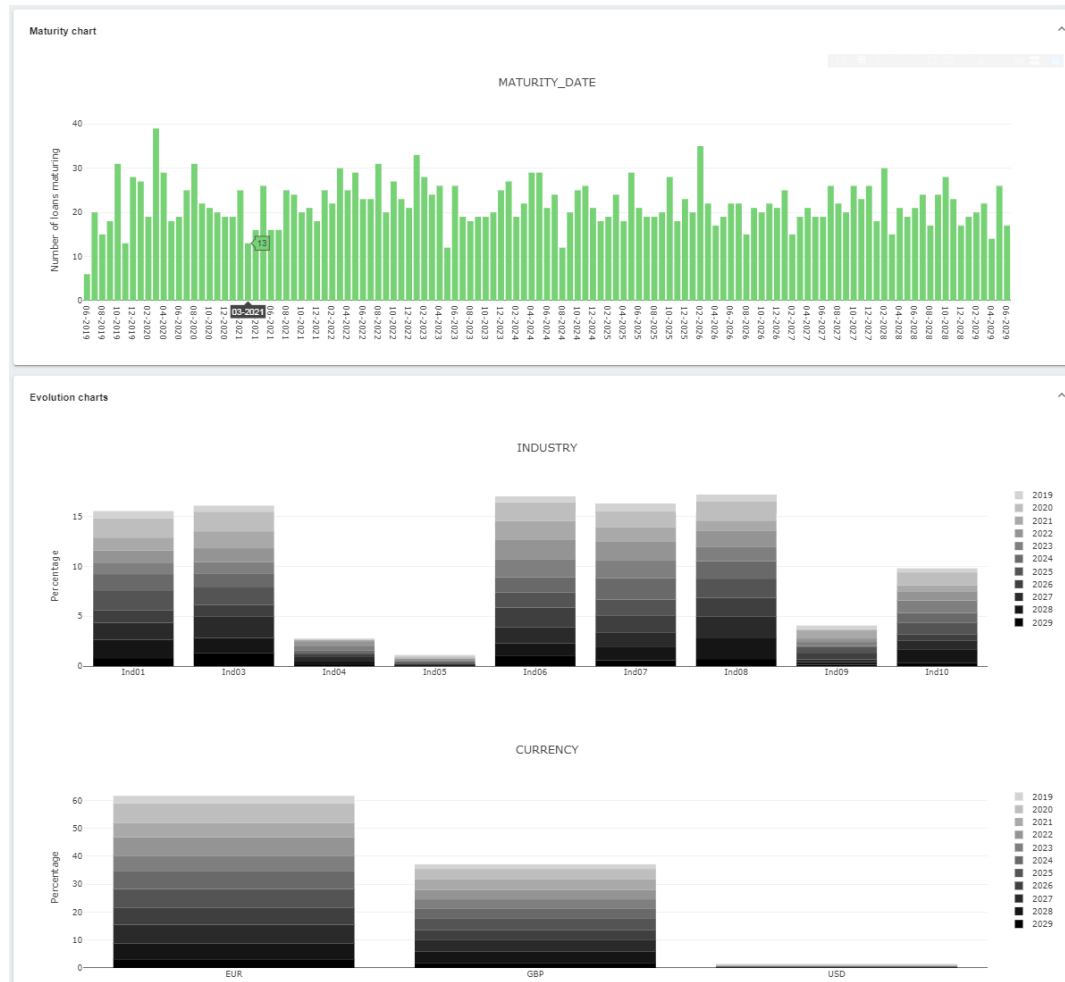
IND6 100 %

IND7 100 %

IND8 100 %

Set all to 100 %

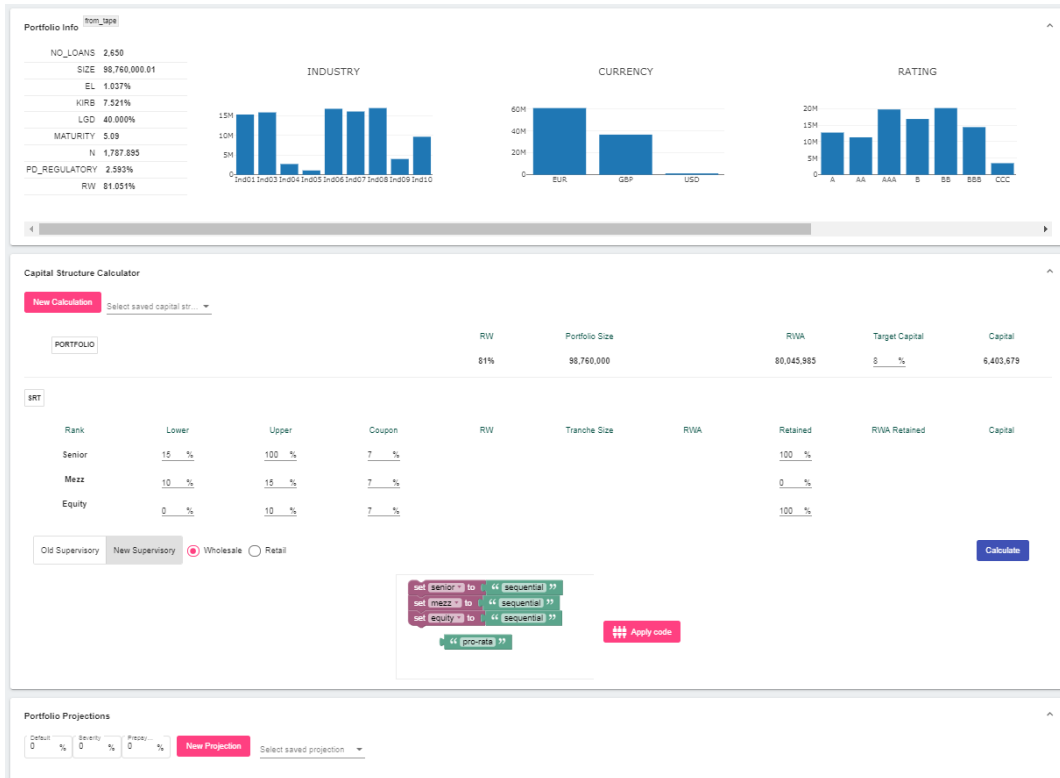
Once a portfolio has been created it is possible to show evolution charts, these show the breakdown of the portfolio with the added dimension of time.



6.3 Liabilities

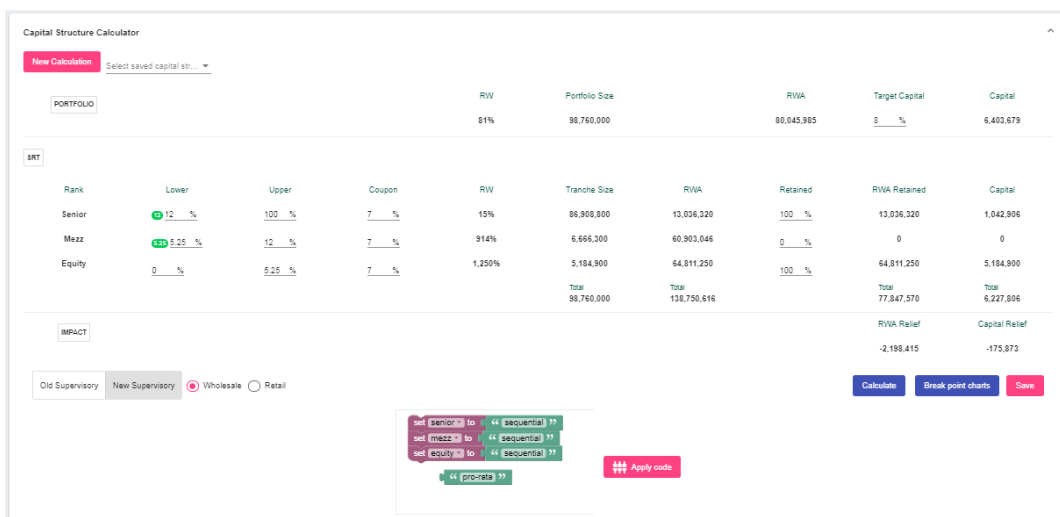
6.3.1 Define a capital structure

To create an SRT structure it is necessary to define a capital structure, typically equity, mezzanine and senior tranches require definition. Once an initial structure has been defined a calculation may be run that will indicate the capital relief created by the transaction.



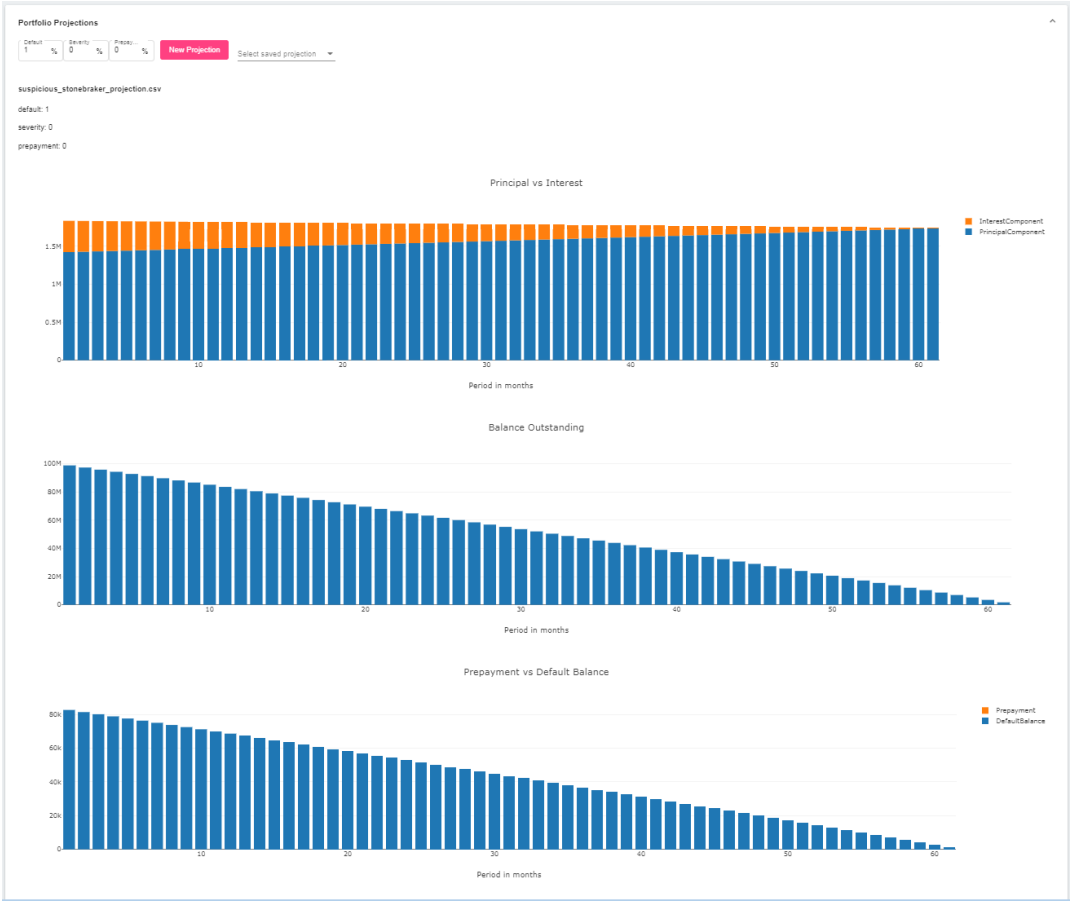
6.3.2 Calculate Capital Efficiency

The Reg capital formula is applied to the capital structure. The calculator solves for the optimal attachment/detachment points of the tranches, these can be selected by simply clicking on the figure highlighted in green.



6.3.3 Cash-flow Projections

Now that a valid transaction has been defined the cash-flows projected can be inspected either graphically or directly by a grid.



The cash-flows from the grid may simply be exported to excel.

Cash Flow

Charts Raw Table

Period	PoolBalance	PrincipalComponent	SeverityRecovered	Prepayment	LossGivenLoss	Coupons	CumulativeDefault	CumulativeLoss	CumulativeRecovery	EquityStart	EquityEnd	EquityLossAllocated	EquityPrincipalAllocated	EquityCoupon
1	97334074.62359673	1425325.3854432601	0	0	0	411500.0000419886	0	0	0	5184900.000525	5184900.000525	0	0	0
2	95903410.38133863	1431284.2422201080	0	0	0	405581.14420481974	0	0	0	5184900.000525	5184900.000525	0	0	0
3	94486182.53810728	1437227.8432203987	0	0	0	399597.5432555963	0	0	0	5184900.000525	5184900.000525	0	0	0
4	93022965.24553113	1443216.3925781458	0	0	0	393609.0939087803	0	0	0	5184900.000525	5184900.000525	0	0	0
5	91573730.55173591	1449229.6937952137	0	0	0	387695.56206871303	0	0	0	5184900.000525	5184900.000525	0	0	0
6	90118498.40088321	1455288.100826937	0	0	0	381657.236922233	0	0	0	5184900.000525	5184900.000525	0	0	0
7	8867138.9327363	1461331.768147913	0	0	0	375493.6183370134	0	0	0	5184900.000525	5184900.000525	0	0	0
8	87189715.6822201	1467420.8508191054	0	0	0	369404.7359687304	0	0	0	5184900.000525	5184900.000525	0	0	0
9	85716181.0789442	1473534.6032268797	0	0	0	363290.4832562054	0	0	0	5184900.000525	5184900.000525	0	0	0
10	84258595.4470953	1479874.6516891163	0	0	0	357150.754468101	0	0	0	5184900.000525	5184900.000525	0	0	0
11	82750895.50454957	1486336.842695736	0	0	0	350985.44352918875	0	0	0	5184900.000525	5184900.000525	0	0	0
12	81258935.66133151	1492830.8427180828	0	0	0	344784.4437688732	0	0	0	5184900.000525	5184900.000525	0	0	0
13	79780387.8230188	1499247.7383127103	0	0	0	338577.6481722146	0	0	0	5184900.000525	5184900.000525	0	0	0
14	78258897.38570548	1504490.437222347	0	0	0	332334.94926257833	0	0	0	5184900.000525	5184900.000525	0	0	0
15	76745138.23841901	1510759.1473774402	0	0	0	326066.2391074852	0	0	0	5184900.000525	5184900.000525	0	0	0
16	75228084.28126084	1517053.071158179	0	0	0	319771.4093287459	0	0	0	5184900.000525	5184900.000525	0	0	0
17	73704709.2258945	1523375.0383963373	0	0	0	313450.3510885895	0	0	0	5184900.000525	5184900.000525	0	0	0

6.4 Transaction Report

When a transaction is finalized a report in pdf can be produced that contains all the details of the transaction.